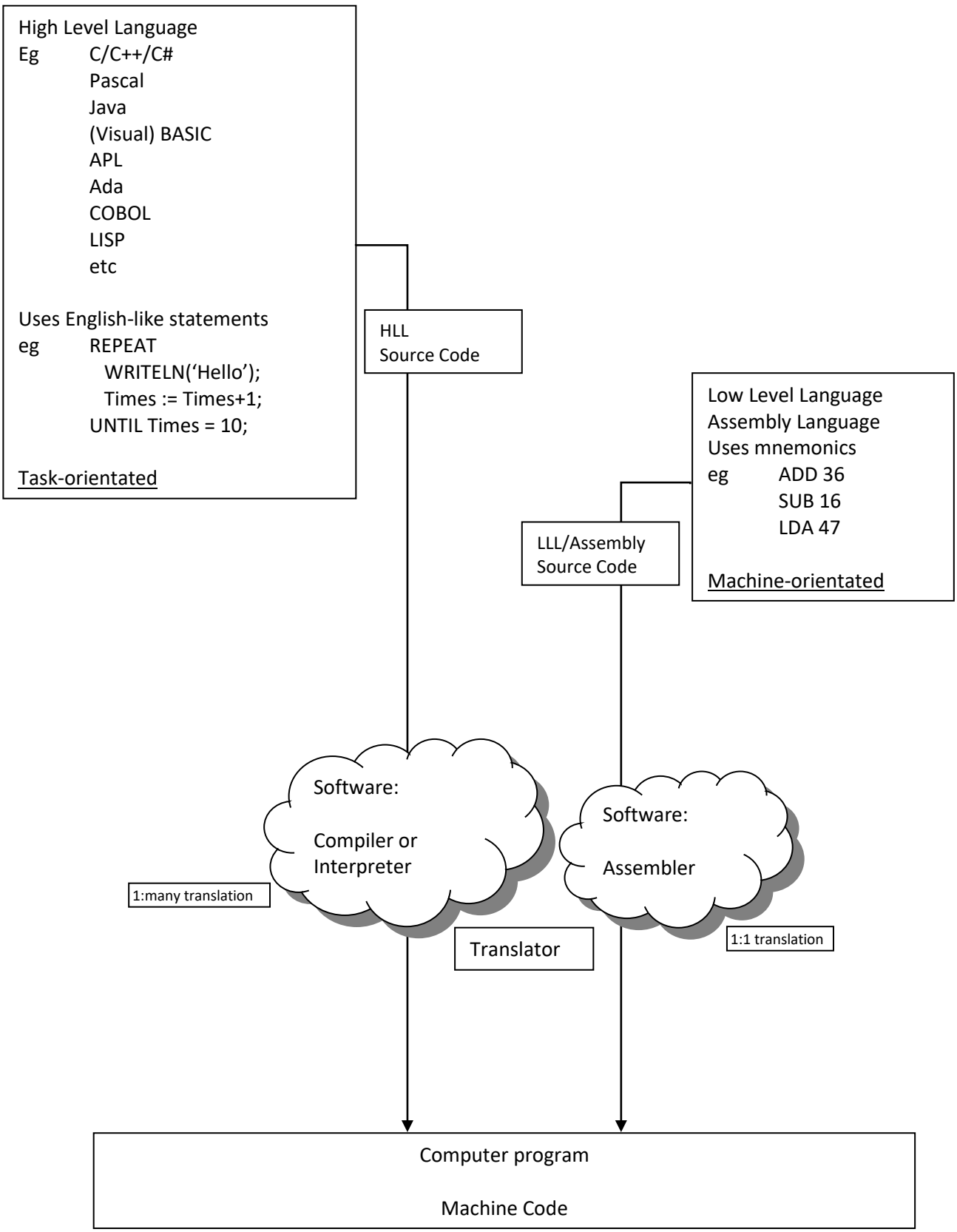
Pascal

Contents

Programming into machine code 2
Designing a solution 3
Program to add two numbers 4
Terminology/concepts to understand: 5
Selection statements in Pascal 11
Selection questions 12
Iteration/Repetition 13
REPEAT … UNTIL 13
FOR … DO 16
WHILE…DO 17
Iteration worksheet 23
Procedures to functions via rogue (sentinel) values and loops (iteration) 25
Arrays 28
Data Types and Data Structures 31
Sorting and Searching (Linear Search, Binary Search, Bubble Sort and Insertion Sort) 35
Linear Search 36
Bubble Sort 39
Binary Search 44
Insertion Sort 47
Files and Records 49
Permanently deleting a record from a file 57
Useful Pascal Library Routines 63
Date and Time features in Virtual Pascal 65
Validation routines in Pascal 68
Keeping a sorted file 69
Using Linked Files 71
File examples of Amazon 75
Entity-Relationship Diagram (ERD) 77
Output too long for screen display 79
Pointers 80
Maintaining an ordered linked list using pointers 82
OCR Programming Project 87
PLANNING WITH STRUCTURE DIAGRAMS AND PSEUDOCODE 93
A FIRST PASCAL PROGRAM 95
WORKSHEET – First program 102
Simple Arithmetic Operators 103

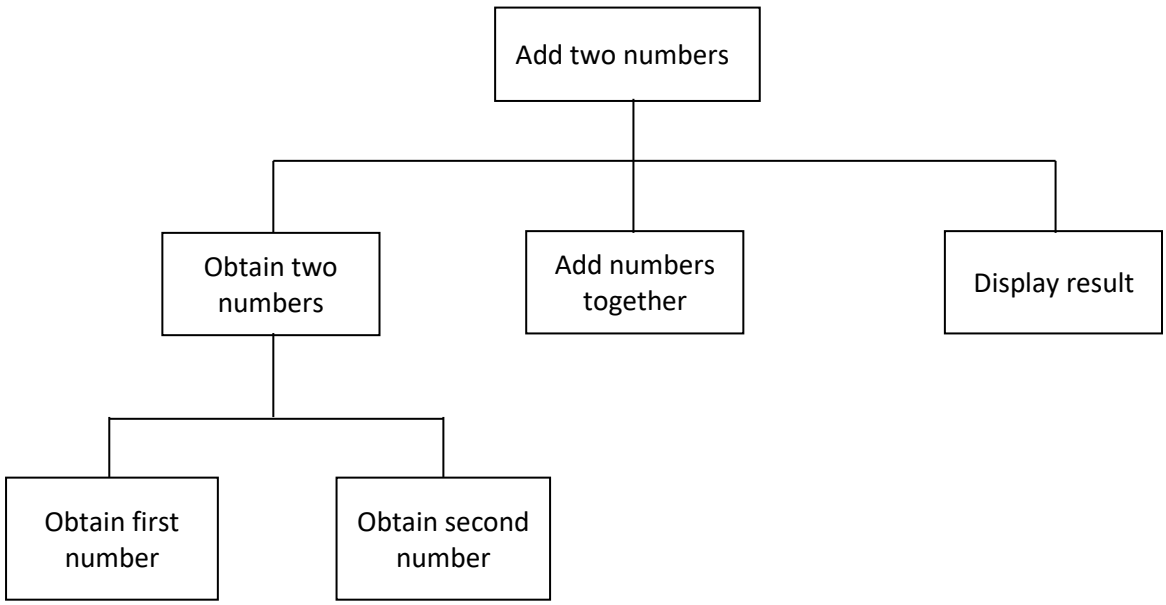
Programming into machine code



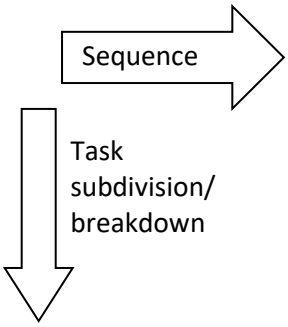
Designing a solution.

Before starting work on any problem, computer-based or not, you should always plan your strategy before attempting to carry out the solution. This can be done by breaking the problem down into its constituent parts. This is a top-down approach, or functional decomposition and can be represented as a structure diagram.

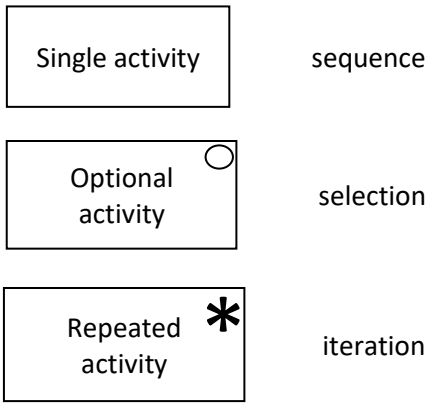
To add two numbers together, we can plan the approach as:



Reading a structure diagram:



Structure diagram boxes:



For more information about structure diagrams, see separate section.

Program to add two numbers

```
PROGRAM AddTwoNumbers;
  USES
    CRT, SYSUTILS;

  VAR
    Number1, Number2, Total : LONGINT;

  PROCEDURE Obtain;
    BEGIN
      WRITELN('Enter first value');
      READLN(Number1);
      WRITELN('Enter second value');
      READLN(Number2);
    END;

  PROCEDURE Add;
    BEGIN
      Total := Number1+Number2;
    END;

  PROCEDURE Display;
    BEGIN
      WRITELN('The answer is');
      WRITELN(Total);
    END;


  BEGIN {of main program}
    Obtain;
    Add;
    Display;

    READLN; {Keeps display on screen until <return> is pressed}
  END.
```

CRT, SYSUTILS?
Line space?
Indentation?

What are the semi-colons?
On every line?

```
PROCEDURE Obtain;
  BEGIN
    WRITE('Enter first value: ');
    READLN(Number1);
    WRITE('Enter second value: ');
    READLN(Number2);
  END;
```

WRITE,
WRITELN?

```
PROCEDURE Display;
  BEGIN
    WRITE('The answer is ');
    WRITELN(Total)
  END;
```

WRITELN('The answer is ',Total);

Terminology/concepts to understand:	
• Programming language	• Case-sensitive
• A program	• Comments
• Modularisation	• Variables
• Input-process-output	• Data types ○ Integer (long integer) ○ Real
• Source code	• Procedures/subprograms (and functions) ○ Defining ○ Calling
• Object code	• Program input/output
• Translation – compiler	• Statement
• Reserved words	• Assignment

Other data types are character (and string), and Boolean (TRUE/FALSE; Yes/No).
The four primitive data types are:

- Integer in Pascal: `INTEGER` or `LONGINT`
- Real `REAL`
- Character `CHAR`
- Boolean `BOOLEAN`

A string data type is a collection (an array) of character. (in Pascal: `STRING`)

In Pascal, we can define a variable to be any of the above data types, eg:

```
VAR
  NoOfPeople : LONGINT; {or INTEGER}
  Salary : REAL;
  DeptCode : CHAR; {P – personnel, M – management etc}
  PaidYet : BOOLEAN;
  Surname : STRING;
```

etc.

A program can consist of three constructs: Sequence, Selection and Iteration.

Sequence is the order that statements are executed.

Selection is when the order of execution is based on a Boolean test (true/false):

- if...then `IF…THEN` IF…THEN
- if...then…else `IF…THEN…ELSE` IF…THEN…ELSE
- multiple if/Case `CASE /IF…THEN…ELSEIF`

Iteration is when a section of code can be executed more than once – a loop:

- for…do `FOR…DO` FOR…DO
- repeat…until `REPEAT…UNTIL` REPEAT…UNTIL
- while…do `WHILE…DO` WHILE…DO

1. Read the notes "A First pascal program" at the end of this booklet and complete the associated worksheet.
2. Read the notes "Simple Arithmetic Operators"
   a. Design (with Structure Diagrams and pseudocode – see notes); and
   b. write a program to request that the user enters his/her name, the name of a subject in which a test was taken; the total possible marks available and the actual marks achieved.

   The output should be the percentage gained in the test.
   A typical screen showing the program execution could look like (italics show user input):

> Please enter your name: *Hermoust*
>
> Thank you Hermoust. Which subject did you take the test in? *geography*
>
> What was the maximum mark available in the geography test? *70*
> What mark did you achieve Hermoust? *23*
>
> Hermoust, you gained 32.86% in the geography test.

   Note: to get a `REAL` number displayed to a set number of decimal places you need to add this to the `WRITE/LN` statement. For example, if `value` is declared as a `REAL` within the `VAR` section, and contains 176.3456; the following are true:

`WRITELN(value);`	will output: 1.76345600000000E+0002 (ie in floating point format)
`WRITELN(value:12:2);`	will output: ▿▿▿▿▿▿176.35 [▿=space] (ie 12 spaces in total including the decimal point, with 2 digits after the point)
`WRITELN(value:4:2);`	will output: 176.35 (ie gives correct value even though 4 spaces is not enough [it should be 6 minimum]!)

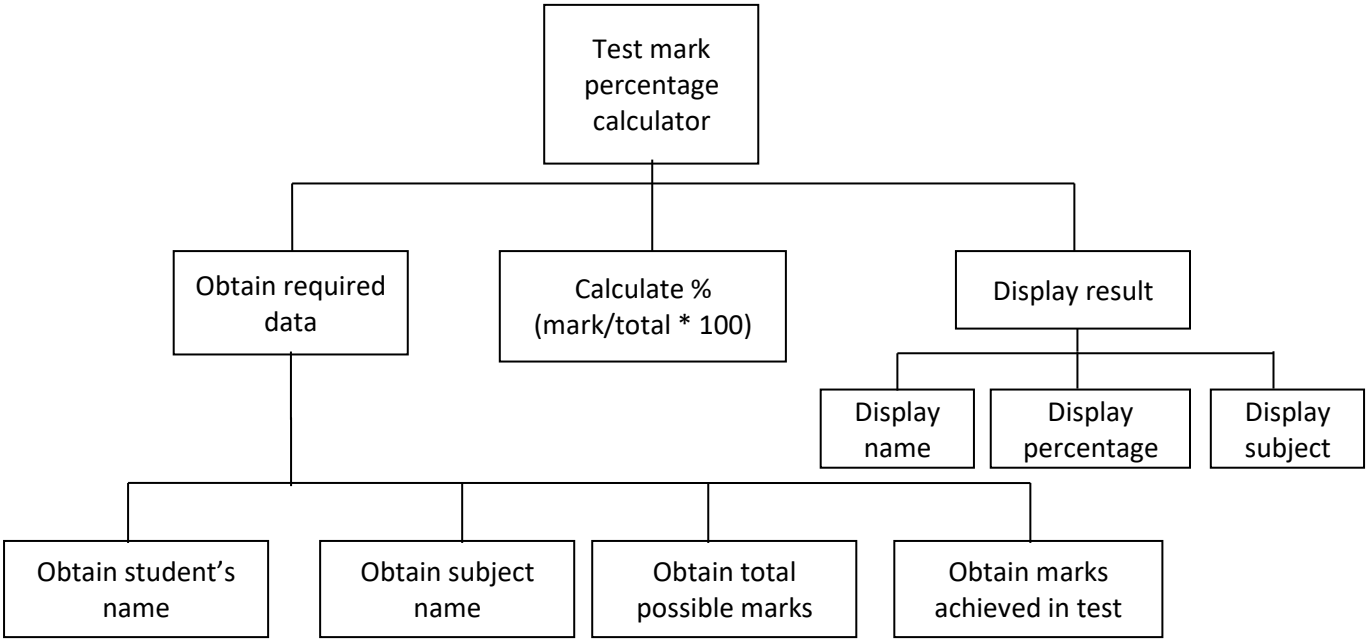
3. Design and write a program to prompt the user for five test scores (out of 25); the program will calculate and display the average score and its percentage. A typical screen showing the program execution could look like (italics show user input):

> Please enter score 1 (out of 25): *17*
> Please enter score 2 (out of 25): *23*
> Please enter score 3 (out of 25): *5*
> Please enter score 4 (out of 25): *12*
> Please enter score 5 (out of 25): *21*
>
> The average score is 15.6 (which is 62.4%).

4. Design and write the program to:
   - input the names of three people
   - request the input of how much each person has spent on Amazon last week
   - display the average amount spent.

   eg interaction (user input in italics):

> Please enter the name of person 1: *Hermoust*
> Please enter the name of person 2: *Jergit*
> Please enter the name of person 3: *Lermin*
>
> Thank you, now enter the amount spent on Amazon last week for:
>
> Hermoust: £*23.17*
> Jergit: £*42.99*
> Lermin: £*23.31*
>
> Note: '£' is NOT in italics!
>
> The average amount spent on Amazon by Hermoust, Jergit and Lermin was £29.82




```
PROGRAM CalcTest;
  USES
    CRT, SYSUTILS;

  VAR
    Name, Subject : STRING;
    Mark, MaxMark : LONGINT;
    Percentage: REAL;

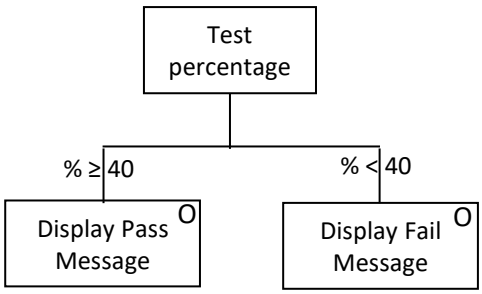
  PROCEDURE ObtainData;
    BEGIN
      WRITE('Please enter your name, then press <return>: ');
      READLN(Name);
      WRITELN; {leave a blank line}
      WRITE('Thank you ',Name,'. Which subject did you take the test in?');
      READLN(Subject);
      WRITELN;
      WRITE('What was the maximum mark available in the ',Subject, ' test?');
      READLN(MaxMark);
      WRITE('What mark did you achieve ',Name, '? ');
      READLN(Mark);
    END;

  PROCEDURE CalcAndShowPercent;
    BEGIN
      Percentage := Mark/MaxMark*100;
      WRITELN(Name,', you gained ',Percentage:4:2,'% in the ',subject,' test');
    END;


  BEGIN
    ObtainData;
    CalcAndShowPercent;

    READLN
  END.
```

We could adapt this to see if the student has passed the test, eg 40% or more.




```
PROGRAM CalcTest;
  USES
    CRT, SYSUTILS;

  VAR
    FirstName : STRING;
    Mark, MaxMark : LONGINT;
    Percentage: REAL;

  PROCEDURE ObtainData;
    BEGIN
      WRITE('Please enter your name, then press <return>: ');
      READLN(FirstName);
      WRITELN; {leave a blank line}
      WRITE('Thank you ',FirstName,' now enter your mark: ');
      READLN(Mark);
      WRITELN;
      WRITE('Please enter the maximum mark: ');
      READLN(Maxmark);
    END;

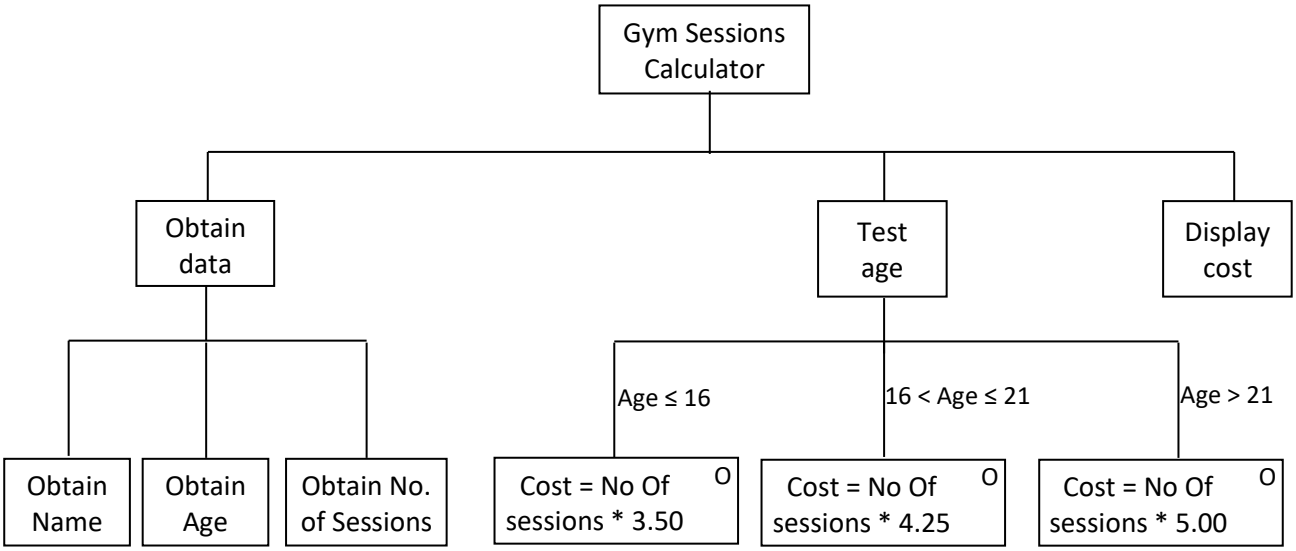
  PROCEDURE PassMessage;
    BEGIN
      WRITELN('Well done ',Firstname,', you have passed');
      WRITELN;
      WRITELN('You achieved ',Percentage:4:2,'%');
      WRITELN;
      WRITELN('Please press <enter>');
      READLN;
    END;

  PROCEDURE FailMessage;
    BEGIN
      WRITELN('Bad luck ',Firstname,', you have failed');
      WRITELN;
      WRITELN('You achieved ',Percentage:4:2,'%');
      WRITELN;
      WRITELN('Please press <enter>');
      READLN;
    END;

  PROCEDURE CalcAndShowPercent;
    BEGIN
      Percentage := Mark/MaxMark*100;
      IF Percentage >= 40 THEN
        PassMessage
      ELSE
        FailMessage;
    END;


  BEGIN
    ObtainData;
    CalcAndShowPercent;
  END.
```

A gym charges different rates for a session of exercise depending on the age of the person. A child's membership cost (16 or under) is £3.50 per session; an adult's membership cost (over 21) is £5 per session; and a youth's membership cost (between child and adult) is £4.25 a session.




```
PROGRAM GymCharge;
  USES
    CRT, SYSUTILS;
  VAR
    Name          : STRING;
    Sessions, Age : LONGINT;
    Cost          : REAL;

  PROCEDURE ObtainData;
    BEGIN
      WRITE('Please enter your name: ');
      READLN(Name);
      WRITELN;
      WRITE('How many gym sessions do you want to book? ');
      READLN(Sessions);
      WRITELN;
      WRITE('How old are you ',Name,'?');
      READLN(Age);
    END;

  PROCEDURE ChildMember;
    BEGIN
      Cost := Sessions * 3.5;
      WRITELN; WRITELN; WRITELN;
      WRITELN(Name,', the cost of your gym membership as a child member is
               £',Cost:4:2);
      WRITELN('Thank you for your custom.');
    END;

  PROCEDURE YouthMember;
    BEGIN
      Cost := Sessions * 4.25;
      WRITELN; WRITELN; WRITELN;
      WRITELN(Name,', the cost of your gym membership as a youth member is
               £',Cost:4:2);
      WRITELN('Thank you for your custom.');
    END;
```

```
PROCEDURE AdultMember;
  BEGIN
    Cost := Sessions * 5;
    WRITELN; WRITELN; WRITELN;
    WRITELN(Name,', the cost of your gym membership as an adult member is
             £',Cost:4:2);
    WRITELN('Thank you for your custom.');
  END;

PROCEDURE CalcCost;
  BEGIN
    IF age <= 16 THEN
      ChildMember;

    IF (age>16) AND (age<=21) THEN
      YouthMember;

    IF age > 21 THEN
      AdultMember;
  END;


BEGIN
  ObtainData;
  CalcCost;
  READLN;
END.
```

or

```
IF age <= 16 THEN
  ChildMember
ELSE
  IF (age>16) AND (age<=21) THEN
    YouthMember
  ELSE
    AdultMember;
```

or

Multiple IF test on age:

```
CASE age OF
  0..16: ChildMember;
  17..21: YouthMember;
  22..100: AdultMember;
END;
```

Notice the order of the procedures. A procedure (or main program) can only call another procedure that has already been declared. (Otherwise it would be a forward reference; which can be declared if absolutely necessary.)

Selection statements in Pascal:

- `IF … THEN`
- `IF … THEN … ELSE`
- `CASE`

Boolean tests can be single	eg `IF age>17 THEN ...`
or combined using `OR` and/or `AND` where each test must be in brackets (and a complete test)	eg `IF(age>17) AND (height>=1.6) THEN ...` `IF(age>17) OR (height>=1.6) THEN ...` Whats's the difference between the two above?? `IF (age>17) AND (age<=25) THEN ... //`✓✓ `IF age>17 AND age<=25 THEN ... //`**XX** `IF age>17 AND <=25 THEN ... //`**XX** `IF 17<age<=25 THEN ... //`**XX**

If an integer or character variable needs multiple tests, `CASE` can be used.

Valid examples:

1
```
IF percentage>=40 THEN
  BEGIN
    WRITELN('Well done');
    WRITELN;
    WRITELN('You have passed');
  END;
```

2
```
IF percentage>=40 THEN
  BEGIN
    WRITELN('Well done');
    WRITELN;
    WRITELN('You have passed');
  END
ELSE
  BEGIN
    WRITELN('Bad Luck');
    WRITELN('You have failed');
    WRITELN('Better luck next time');
  END;
```
There is **never** a `;` before an `ELSE`

3
```
IF percentage>=40 THEN
  Passmessage
ELSE
  FailMessage;
```
Where `PassMessage` and `FailMessage` are previously declared procedures written by the programmer

4
```
READLN(Password);
IF Password = 'Reserved' THEN
  CorrectPassword
ELSE
  IncorrectPassword;
```
Where `CorrectPassword` and `IncorrectPassword` are previously declared procedures by the programmer and `Password` is a variable of the data type `STRING`

5
```
READLN(MenuChoice);
CASE MenuChoice OF
  'A','a':AddNewData;
  'B','b':SearchData;
  'C','c':ModifyData;
  'D','d':Deletedata;
END;
```
Or
```
MenuChoice:=READKEY;
WRITELN(MenuChoice);
```

Selection questions

1. Write a program to identify whether a person can go on a theme park ride.
    a. A person has to be over 13, taller than 1.5m and weigh more than 50kg.
    b. The rules are changed so that a person has to weigh more than 50kg and be taller than 1.5m or over 13.

2. Write a program to enter two surnames state whether they match (as case-sensitive)
    a. Adapt this to be non-case-sensitive.

3. Write a program to enter a master personal identification number (PIN) [consider data type]. Then accept a second PIN. If it matches, give a suitable message; if it does not match, then allow another chance to enter a PIN and test again

4. Write a program to generate a random integer between 1 and 20. This is done by using `RANDOMIZE` once, then `Number := RANDOM(20)+1;`
   eg:

```
     RANDOMIZE;

:

PROCEDURE GenerateRandomNumber;
  BEGIN
    Number := RANDOM(20)+1;
  END;
```

   Allow the user to enter a guess and give a relevant messages for:
    - Equal
    - Whether it was lower or higher
    - If it was within two of the number

5. Write a program to enter someone's name and age and identify what type of membership is available at a gym. People under 12 cannot join; 12-14 year olds can have a junior membership with a parent's signature; 15 year olds can be junior members; 16 and 17 year olds can have youth membership; 18-21 year olds can have young adult membership; others can have adult membership (but if someone has a student card then there is a student membership category).
    a. Use `IF` statements
    b. Use `CASE` statement

Selection – Additional Questions

1. Design the program to calculate the percentage gained in a test and to display the final grade. The inputs will be: Student's name, Subject, Mark achieved, Total marks possible. The output will be a relevant message along with the grade achieved as follows: Distinction if more than 70%; Merit if more than 55%; Pass if more than 40%; Fail otherwise.
2. Code the program for the above design.
3. Adapt the program to allow the user to enter the grade boundary percentages for each grade.
4. Write the program to calculate a percentage for a test and award the grades A* for greater than or equal 90%; A ≥ 80%; B ≥ 70%; etc E ≥40%; U otherwise.
5. Design the program to enter two numeric values and then select from a menu whether to A) Add them; S) Subtract them; M) Multiply them; D) Divide them; error message if none of these chosen.
6. Code the program for the above design. [Add a test that if either of the input values are not numbers, then do not continue with the program. You need to use `VAL(data1,data2,ErrCode);`]

Iteration/Repetition

There are three iteration/repetition constructs:

- `REPEAT … UNTIL`
- `WHILE … DO`
- `FOR … DO`

The section of code to be repeated is enclosed within the statement.

REPEAT … UNTIL

The code between the `REPEAT` and the `UNTIL` is executed at least once. The `UNTIL` contains a Boolean test; and the code continues to be executed until the Boolean test is TRUE.

Examples:

```
PROGRAM CalcTest;
  USES
    CRT, SYSUTILS;

  VAR
    FirstName : STRING;
    Mark, MaxMark : LONGINT;
    Percentage: REAL;

  PROCEDURE ObtainData;
    BEGIN
      WRITE('Please enter your name, then press <return>: ');
      READLN(FirstName);
      WRITELN; {leave a blank line}
      WRITE('Thank you ',FirstName,' now enter your mark: ');
      READLN(Mark);
      WRITELN;
      WRITE('Please enter the maximum mark: ');
      READLN(Maxmark);
    END;

  PROCEDURE CalcPercent;
    BEGIN
      Percentage := Mark/MaxMark*100;
    END;

  PROCEDURE DisplayResult;
    BEGIN
      WRITELN(FirstName,', you achieved ',Percentage:4:2,'% in your test');
      WRITELN;
      WRITELN('Please press <return>');
      READLN;
    END;


  BEGIN
    ObtainData;
    CalcPercent;
    DisplayResult;
  END.
```

This program can be executed repeatedly until the user does not wish to continue, by changing the main program to:

```
BEGIN
  REPEAT
    CLRSCR;
    ObtainData;
    CalcPercent;
    DisplayResult;

    WRITE('Do you wish to calculate another percentage (Y/N): ');
    Again := UPCASE(READKEY);
  UNTIL Again = 'N';
END.
```

`Again` needs to be declared as `CHAR`.

Input data can also be validated using a REPEAT…UNTIL:

```
REPEAT
  WRITE('Please enter a percentage: ');
  READLN(percent);
  IF (percent < 0) OR (percent > 100) THEN
    BEGIN
      WRITELN('Error in value – it must be between 0 and 100');
      WRITELN('You need to re-enter. Press any key to continue');
      READKEY;
    END;
UNTIL (percent >= 0) AND (percent <= 100);
```

Or:

```
REPEAT
  WRITE('Please enter Maximum mark in test: ');
  READLN(MaxMark);
  WRITE('Please enter mark achieved in test: ');
  READLN(Mark);

  IF (Mark < 0) OR (Mark > MaxMark) THEN
    BEGIN
      WRITELN('Error in value – it must be between 0 and ',MaxMark);
      WRITELN('You need to re-enter. Press any key to continue');
      READKEY;
    END;
UNTIL (Mark >= 0) AND (Mark <= MaxMark);
```

A Repeat…Until loop continues until a Boolean condition is TRUE.
The contents of the loop will be carried out once.

More examples:

```
PROCEDURE MainMenu;
  BEGIN
    REPEAT
      CLRSCR;
      WRITELN('Welcome to the Update Program');
      WRITELN;
      WRITELN('S: Search for a record');
      WRITELN('U: Update/Amend a record');
      WRITELN('A: Add a new record');
      WRITELN('D: Delete a record');
      WRITELN;
      WRITELN('X: eXit');

      WRITELN; WRITELN;

      WRITE('Enter choice: ');
      MainMenuChoice := UPCASE(READKEY);
      WRITELN(MainMenuChoice);

      CASE MainMenuChoice OF
        'S': SearchRoutine;
        'U': AmendRoutine;
        'A': AddRoutine;
        'D': DeleteRoutine;
        'X': {Do nothing, loop will end};
      ELSE
        BEGIN
          WRITELN('Error – incorrect menu choice made');
          WRITE('Press any key to continue to see the menu again');
          READKEY;
        END;
      END; {of CASE}
    UNTIL MainMenuChoice = 'X';
  END;
```

A multiple IF or CASE statement

Or to validate a number input using a type check:

```
REPEAT
  WRITE('Please enter the number: ');
  READLN(NumberStr); {a STRING data type}
  VAL(NumberStr,Number,ErrCode);
           {Number can be REAL or LONGINT, ErrCode must be LONGINT}
  IF ErrCode <> 0 THEN
    BEGIN
      WRITELN('Error – you did not enter a valid number')
      WRITELN('Press any key to continue, and then re-enter');
      READKEY;
    END;
UNTIL ErrCode = 0;
```

FOR … DO

A FOR…DO loop is executed a set number of times. It requires an integer variable to control ("count") how many iterations are carried out. Examples:

```
FOR Times := 1 TO 5 DO
  WRITELN('Hello');
```

```
FOR Count := 1 TO 6 DO
  WRITELN('Count: ',Count);
```

```
FOR Loop := 1 TO 4 DO
  WRITELN(Loop, ' x ',8, ' = ',Loop*8);
```

```
FOR ToDo := 1 TO 7 DO
  WRITELN('Hello');
  WRITELN:
  WRITELN(ToDo);
```

```
FOR ToDo := 1 TO 7 DO
  BEGIN
    WRITELN('Hello');
    WRITELN:
    WRITELN(ToDo);
  END;
```

```
FOR Times := 1 TO 3 DO
  BEGIN
    WRITE('Enter first Code: ');
    READLN(Code1);
    Code1 := LOWERCASE(Code1);
    WRITE('Enter second Code: ');
    READLN(Code2);
    Code2 := LOWERCASE(Code2);

    IF Code1 = Code 2 THEN
      BEGIN
        WRITELN('Codes match - press any key');
        READKEY;
      END
    ELSE
      BEGIN
        WRITE('Codes do not match – re-enter second code: ');
        READLN(Code2);
        Code2 := LOWERCASE(Code2);
        IF Code1 = Code 2 THEN
          BEGIN
            WRITELN('Codes match - press any key');
            READKEY;
          END
        ELSE
          BEGIN
            WRITELN('Codes do not match You have had two attempts.');
            WRITELN('Program will now close – press any key');
            READKEY;
          END;
      END;
  END;
```

What does LOWERCASE do?

- UPPERCASE
- UPCASE

WHILE…DO

The `WHILE..DO` loop contains a Boolean test, and if `TRUE` the loop is entered.

Example:

```
READLN(Number);
WHILE Number>0 DO
  BEGIN
    WRITELN('The value is positive');
    READLN(Number);        What happens if this line is left out?
  END;
```

REPEAT…UNTIL

The `REPEAT..UNTIL` loop contains a Boolean test, and if `TRUE`, the loop is exited.

Example:

```
READLN(Number);
REPEAT
    WRITELN('The value is positive');
    READLN(Number);        What happens if this line is left out?
  UNTIL Number<0;
```

- Are the two loops above the same?
- What is the difference between a `REPEAT…UNTIL` loop and a `WHILE…DO` loop?
- Can a Repeat loop be replaced by a While loop? (and/or vice-versa)
- When is a Repeat loop better used than a While loop (if ever)?
- When is a While loop better used than a Repeat loop (if ever)?

Vehicle Hire Program
Problem and Design

A program is required to calculate vehicle hire costs.

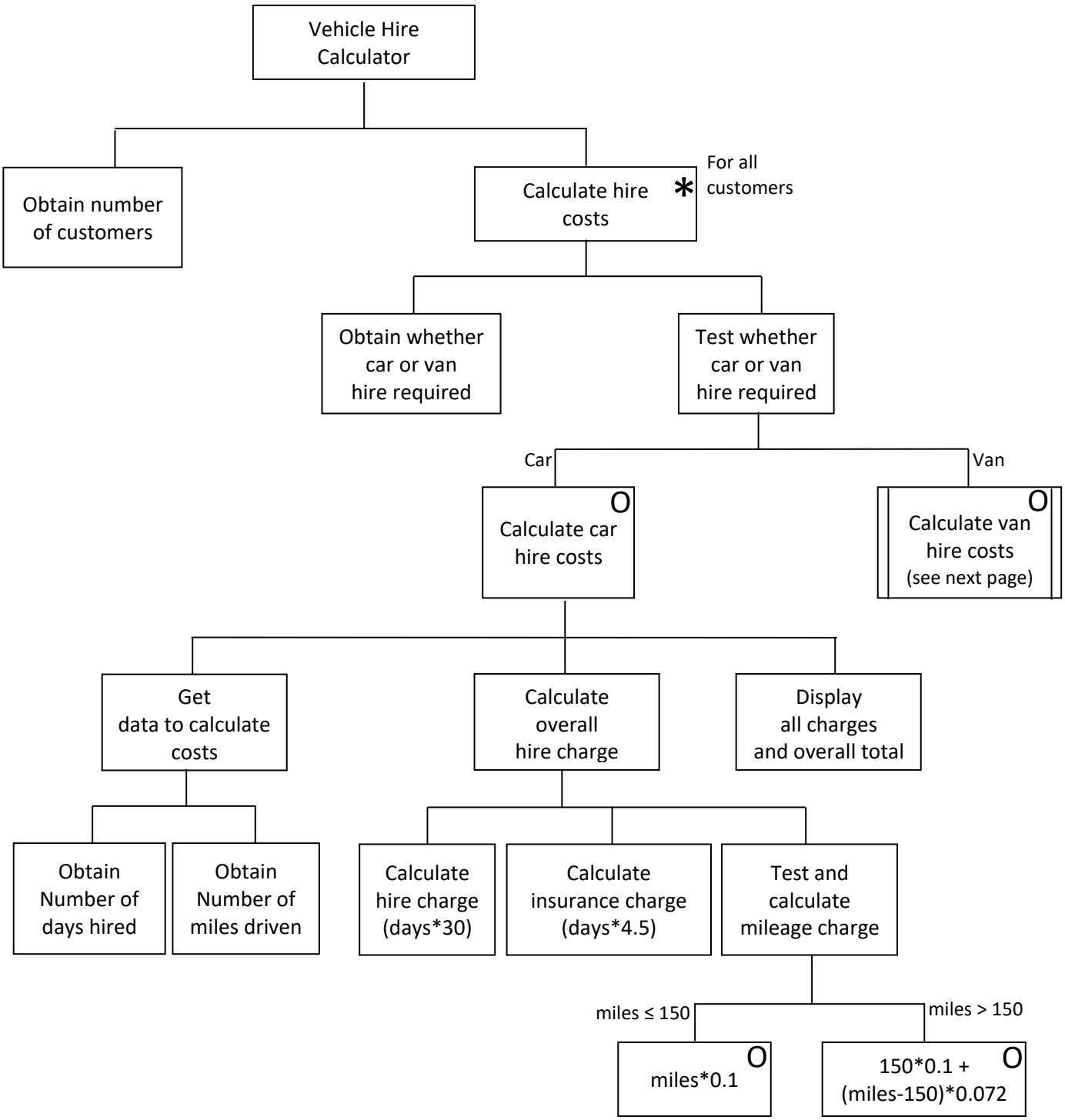
The charges to hire a car are as follows:

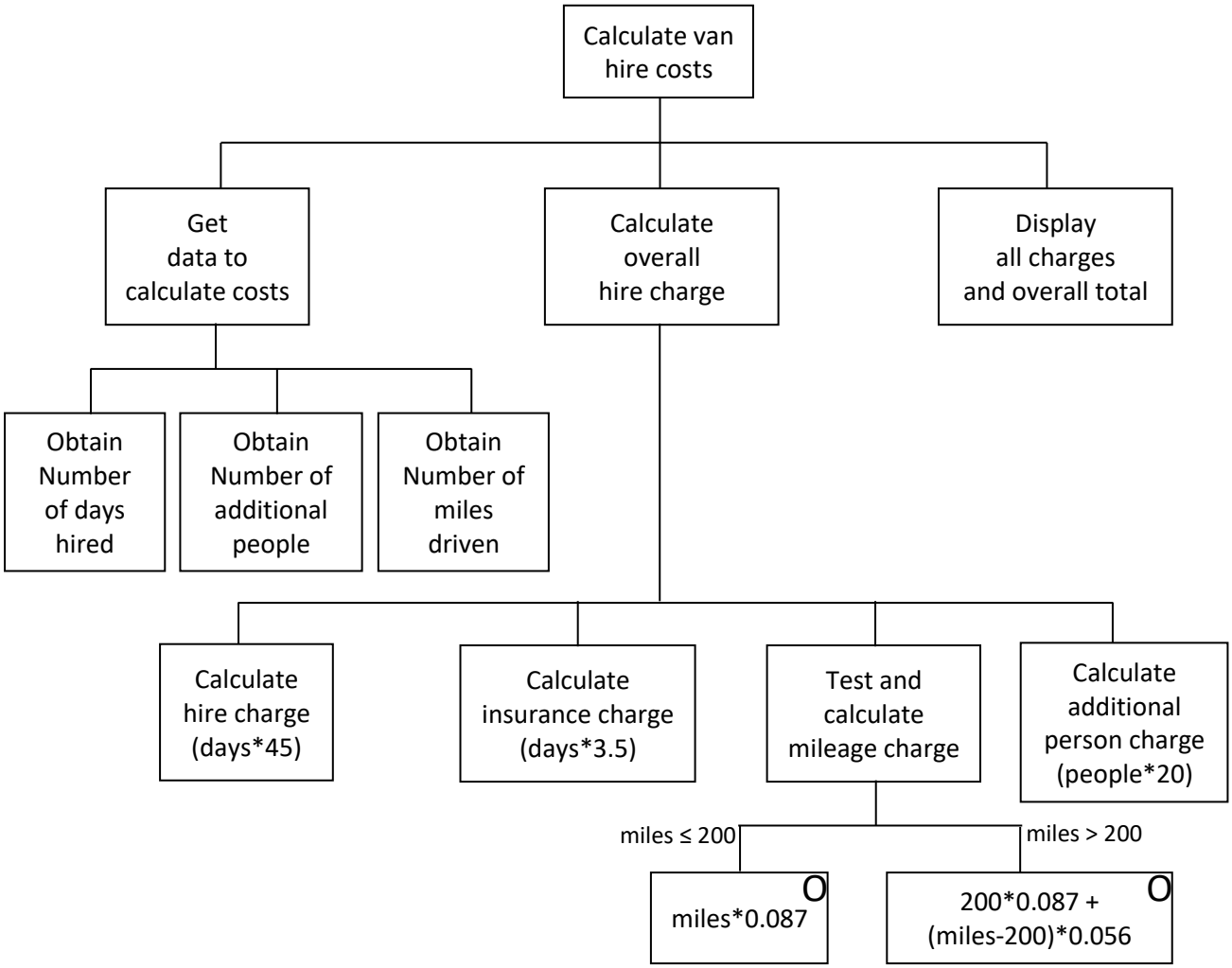
Car hire charge for each day, or part of a day	£30	
Insurance charge per day		£4.50
Each mile driven up to 150 miles		10p
Every mile over 150		7.2p

The charges to hire a van are as follows:

Van hire charge for each day, or part of a day	£45	
Insurance charge per day		£3.50
Each person other than the driver		£20 single payment each
Each mile driven up to 200 miles		8.7p
Every mile over 200		5.6p

Design and write the program to enter all the relevant data and display the charges and the overall cost for a number of different customers.


Vehicle Hire Calculator
Obtain number of customers
Calculate hire costs
*
For all customers
Obtain whether car or van hire required
Test whether car or van hire required
Car
Van
O
Calculate car hire costs
O
Calculate van hire costs
(see next page)
Get data to calculate costs
Calculate overall hire charge
Display all charges and overall total
Obtain Number of days hired
Obtain Number of miles driven
Calculate hire charge (days*30)
Calculate insurance charge (days*4.5)
Test and calculate mileage charge
miles ≤ 150
miles > 150
O
miles*0.1
O
150*0.1 + (miles-150)*0.072




```
PROGRAM CalculateVehicleHireCosts;
  USES CRT, SYSUTILS;

  VAR
    NoOfCarDays, NoOfCarMiles : LONGINT;
    NoOfVanDays, NoOfVanMiles, NoOfExtraVanPeople : LONGINT;
    CarHireCharge, CarInsuranceCharge, CarMilesCharge,
    CarTotal : REAL;
    VanHireCharge, VanInsuranceCharge, VanMilesCharge,
    VanPeopleCharge, VanTotal : REAL;
    NoOfCustomers, Times : LONGINT;
    CarVanChoice : CHAR;

  PROCEDURE GetCustomerNo;
    BEGIN
      WRITE('Enter number of customers: ');
      READLN(NoOfCustomers);
      WRITELN; WRITELN;
    END;
```

```
PROCEDURE CarOrVan;
  BEGIN
    WRITE('Enter C for car hire, or V for van hire: ');
    CarVanChoice := UPCASE(READKEY);
    WRITELN(CarVanChoice);
    WRITELN; WRITELN;
  END;

PROCEDURE ObtainCarData;
  BEGIN
    WRITE('Enter number of days car is hired for: ');
    READLN(NoOfCarDays);
    WRITE('Enter number of miles driven: ');
    READLN(NoOfCarMiles);
  END;

PROCEDURE CalculateCarCharges;
  BEGIN
    CarHireCharge:=NoOfCarDays*30;
    CarInsuranceCharge:=NoOfCarDays*4.5;
    IF NoOfCarMiles <= 150 THEN
      CarMilesCharge := NoOfCarMiles*0.1
    ELSE
      CarMilesCharge := (150*0.1)+((NoOfCarMiles-150)*0.072);

    CarTotal:=CarHireCharge + CarInsuranceCharge + CarMilesCharge;
  END;

PROCEDURE DisplayCarInvoice;
  BEGIN
    CLRSCR;
    WRITELN('SBE Vehicle Hire Centre':30);
    WRITELN;WRITELN;
    WRITELN('Car Hire');
    WRITELN;
    WRITELN('Number of days hired for: ':40,NoOfCarDays);
    WRITELN('Number of miles driven: ':40,NoOfCarMiles);
    WRITELN;
    WRITELN('Car Hire Charge: £':40,CarHireCharge:6:2);
    WRITELN('Insurance charge: £':40,CarInsuranceCharge:6:2);
    WRITELN('Mileage charge: £':40,CarMilesCharge:6:2);
    WRITELN;
    WRITELN('Total to pay: £':40,CarTotal:6:2);
  END;

PROCEDURE CarHire;
  BEGIN
    ObtainCarData;
    CalculateCarCharges;
    DisplayCarInvoice;
  END;
```

```
PROCEDURE ObtainVanData;
  BEGIN
    WRITE('Enter number of days van is hired for: ');
    READLN(NoOfVanDays);
    WRITE('Enter number of additional people other than driver: ');
    READLN(NoOfExtraVanPeople);
    WRITE('Enter number of miles driven: ');
    READLN(NoOfVanMiles);
  END;

PROCEDURE CalculateVanCharges;
  BEGIN
    VanHireCharge:=NoOfVanDays*45;
    VanInsuranceCharge:=NoOfVanDays*3.5;
    IF NoOfVanMiles <= 200 THEN
      VanMilesCharge := NoOfVanMiles*0.087
    ELSE
      VanMilesCharge := (200*0.087) + ((NoOfVanMiles-200)*0.056);

    VanPeopleCharge:=NoOfExtraVanPeople*20;
    VanTotal := VanHireCharge + VanInsuranceCharge +
                  VanMilesCharge+VanPeopleCharge;
  END;

PROCEDURE DisplayVanInvoice;
  BEGIN
    CLRSCR;
    WRITELN('SBE Vehicle Hire Centre':30);
    WRITELN;WRITELN;
    WRITELN('Van Hire');
    WRITELN;
    WRITELN('Number of days hired for: ':40,NoOfVanDays);
    WRITELN('Number of miles driven: ':40,NoOfVanMiles);
    WRITELN;
    WRITELN('Van Hire Charge: £':40,VanHireCharge:6:2);
    WRITELN('Passenger charge: £':40,VanPeopleCharge:6:2);
    WRITELN('Insurance charge: £':40,VanInsuranceCharge:6:2);
    WRITELN('Mileage charge: £':40,VanMilesCharge:6:2);
    WRITELN;
    WRITELN('Total to pay: £':40,VanTotal:6:2);
  END;

PROCEDURE VanHire;
  BEGIN
    ObtainVanData;
    CalculateVanCharges;
    DisplayVanInvoice;
  END;

PROCEDURE PressAnyKey;
  BEGIN
    WRITELN('Press any key to continue');
    READKEY;
  END;
```

```
PROCEDURE CalculateHireCosts;
  BEGIN
    GetCustomerNo;
    FOR Times := 1 TO NoOfCustomers DO
      BEGIN
        CLRSCR;
        REPEAT
         CarOrVan;
         IF CarVanChoice = 'C' THEN
           CarHire
         ELSE
           IF CarVanChoice = 'V' THEN
             VanHire
           ELSE
             WRITELN('Error - enter C or V');
        UNTIL (CarVanChoice='C') OR (CarVanChoice='V'); //Validation

        PressAnyKey;
      END;
  END;


BEGIN
  CalculateHireCosts;
END.
```

Iteration worksheet

<table>
<tr><td>1</td><td>Design, code and test a program to extend the Guess a Random Number between 1 and 20 program to allow:
a) As many guesses as required until the correct value is guessed (with nice messages about higher, lower, with 2 etc)
b) A maximum of five guesses (with relevant messages, especially if not found)</td></tr>
<tr><td>2</td><td>Design, write and test a program to find the sum of the first 10 natural numbers (1, 2, 3,4, etc). The expected output will be similar to:
The sum of the first 10 natural numbers is :
1+2+3+4+5+6+7+8+9+10 = 55
Write the program with:
a) A <code>FOR…DO</code> loop
b) A <code>REPEAT…UNTIL</code> loop
c) A <code>WHILE…DO</code> loop</td></tr>
<tr><td>3</td><td>Adapt the programs in question 2 to allow the user to choose and enter the largest value to sum up to. For example if 13 is entered the output will be similar to:
The sum of the first 13 natural numbers is :
1+2+3+4+5+6+7+8+9+10+11+12+13 = 91
(It would be good to validate the entry. For example ensure that it’s an integer; greater than 0; the entered value is not too large –</td></tr>
<tr><td>4</td><td>A light plane is flying from London to Paris. After all the cargo has been loaded there is room for up to six passengers with a combined weight of not more than 500 kg. Design, write and test a program to accept individual passengers weights up to the limit and then to display the total allowed weight and number of passengers. Example input data:

Passenger weight
1 60 kg
2 110 kg
3 85 kg
4 90 kg
5 90 kg
6 75 kg

and the program would display, eg:

5 passengers allowed, 435 kg

(The last passenger not being counted as this would put the total weight over the 500 kg limit.)

Passenger weight
1 60.2 kg
2 72.8 kg
3 72 kg
4 51.5 kg
5 81.5 kg
6 107 kg
7 52.1 kg

and the program would display, eg:

6 passengers allowed, 445.1 kg

(The last passenger not being counted as this would put the number of passengers over the maximum allowed. Person 7 may not be entered with a message stating the maximum is reached after the sixth passenger.)

Write the program with:
a) a <code>REPEAT</code> loop
b) a <code>WHILE</code> loop

You could validate the input.</td></tr>
<tr><td>5</td><td>Next page</td></tr>
</table>

Design and write a program to allow a user to input students' names and their results in a test (marked out of 40). The input will end when the user enters "ZZZ" for the name and "-1" for the result. The program will output the overall average of the test (both out of 40 and as a percentage); and the name and result (out of 40 and as a percentage) of the students with the best and worst results.

An example interaction could be (user input in bold and italics):

Welcome to the automated test statistics' calculator:

Please enter the name of student 1 (ZZZ to end): ***Hermoust***
Please enter Hermoust's result (-1 to end): ***25***

Please enter the name of student 2 (ZZZ to end): ***Lermin***
Please enter Lermin's result (-1 to end): ***45***

Please re-enter Lermin's result; it must be between 0 and 40: ***45***
Please re-enter Lermin's result; it must be between 0 and 40: ***35***

Please enter the name of student 3 (ZZZ to end): ↵(only <enter> pressed)

Please re-enter the name of student 3 (ZZZ to end) – you didn't enter anything: ***Jergit***
Please enter Jergit's result (-1 to end): ***$(***

Please re-enter Jergit's result; it must be a valid number: ***39***

Please enter the name of student 4 (ZZZ to end): ↵

Please re-enter the name of student 4 (ZZZ to end) – you didn't enter anything: ***Fimble***
Please enter Fimble's result (-1 to end): ********

Please re-enter Fimble's result; it must be a valid number: ***77***
Please re-enter Fimble's result; it must be between 0 and 40: ***-77***
Please re-enter Fimble's result; it must be between 0 and 40: ***7***

Please enter the name of student 4 (ZZZ to end): ***zzz***
Please enter zzz's result (-1 to end): ***-1***

The results of the test are:
Average score (out of 40) : 26.5
Average percentage in test: 66.25%

- Could also test for no entry for the result, eg:
  Please enter Fimble's result (-1 to end): ********

  Please re-enter Fimble's result; it must be a valid number: ***77***
  Please re-enter Fimble's result; it must be between 0 and 40: ***-77***
  Please re-enter Fimble's result; it must be between 0 and 40: ↵
  Please re-enter Fimble's result; you didn't enter anything. It must be between 0 and 40: ***7***

- Also, could terminate program immediately after "ZZZ" is entered (ie not ask for ZZZ's result to be -1)

Procedures to functions via rogue (sentinel) values and loops (iteration)

```
PROGRAM CalcAverage;
  USES
    CRT, SYSUTILS;
  VAR
    Number, Total, Average : REAL;
    Count : LONGINT;

  PROCEDURE GetDataAndCalc;
    BEGIN
      Total := 0;
      Count := 0;
      REPEAT
        WRITE('Enter value (-1) to end: ');
        READLN(Number);
        Total := Total + Number;
        Count := Count + 1;
      UNTIL Number = -1;
      Average := Total/Count;
      WRITELN('The average of the ',Count,' values is ',Average:5:2);
    END;

  BEGIN
    GetDataAndCalc;

    READLN;
  END.
```

Q: What is wrong with this?
A: It includes the rogue (or sentinel) value in the calculation. Eg entering 2,3,7,-1; should give the answer as 4 as the average (12÷3), instead it will give 2.75 (11÷4).

Q: How can we stop this?
A: Three ways:

1

```
PROCEDURE GetDataAndCalc;
  BEGIN
    Total := 0;
    Count := 0;
    WRITE('Enter value (-1) to end: ');
    READLN(Number);
    REPEAT
      Total := Total + Number;
      Count := Count + 1;
      WRITE('Enter value (-1) to end: ');
      READLN(Number);
    UNTIL Number = -1;
    Average := Total/Count;
    WRITELN('The average of the ',Count,' values is ',Average:5:2);
  END;
```

2

```
PROCEDURE GetDataAndCalc;
  BEGIN
    Total := 0;
    Count := 0;
    REPEAT
      WRITE('Enter value (-1) to end: ');
      READLN(Number);

        IF Number <> -1 THEN
          BEGIN
            Total := Total + Number;
            Count := Count + 1;
          END;

    UNTIL Number = -1;
    Average := Total/Count;
    WRITELN('The average of the ',Count,' values is ',Average:5:2);
  END;
```

3

```
PROCEDURE GetDataAndCalc;
  BEGIN
    Total := 0;
    Count := 0;
    REPEAT
      WRITE('Enter value (-1) to end: ');
      READLN(Number);
      Total := Total + Number;
      Count := Count + 1;
    UNTIL Number = -1;

    Total := Total + 1;
    Count := Count -1;

    Average := Total/Count;
    WRITELN('The average of the ',Count,' values is ',Average:5:2);
  END;
```

Q: Is everything now perfect?

A: No. What if the user were to enter -1 as the first value??

Since we are returning a single result, we could (should) use a function:

```
PROGRAM CalcAverage;
  USES
    CRT, SYSUTILS;
  VAR
    Number, Total, Average : REAL;
    Count : LONGINT;

  PROCEDURE GetDataAndCalc;
    BEGIN
      Total := 0;
      Count := 0;
      REPEAT
        WRITE('Enter value (-1) to end: ');
        READLN(Number);

          IF Number <> -1 THEN
            BEGIN
              Total := Total + Number;
              Count := Count + 1;
            END;

      UNTIL Number = -1;
      Average := Total/Count;
      WRITELN('The average of the ',Count,' values is ',Average:5:2);
    END;

  BEGIN
    GetDataAndCalc;
    READLN;
  END.
```

```
PROGRAM CalcAverage;
  USES
    CRT, SYSUTILS;
  VAR
    Number, Total: REAL;
    Count : LONGINT;

  FUNCTION Average:REAL;
    BEGIN
      Total := 0;
      Count := 0;
      REPEAT
        WRITE('Enter value (-1) to end: ');
        READLN(Number);

          IF Number <> -1 THEN
            BEGIN
              Total := Total + Number;
              Count := Count + 1;
            END;

      UNTIL Number = -1;
      Average := Total/Count;
    END;

  BEGIN
    WRITELN('The average of the ',Count,' values is ',Average:5:2);
    READLN;
  END.
```

What if the user entered -1 as the first value??

Q: What if we want to do more processing on the numbers, and not just calculate the mean (average), eg how many values are above/below the mean etc

A: We need to store the values as they are entered, rather than trying to use one store/variable. This can be done by using an array.

Arrays

- One-dimensional

Number	12	27	17	32	----------	29
	[0]	[1]	[2]	[3]	---	[N]

eg 17 is in Number[3]

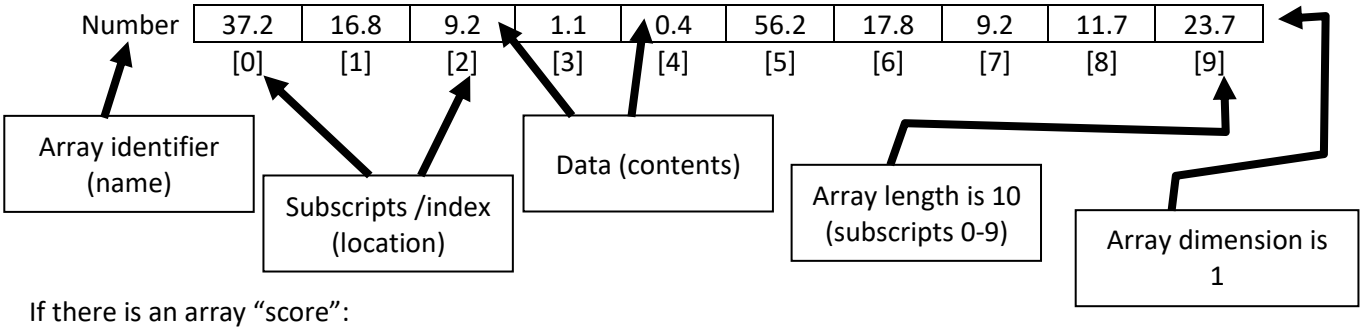
eg
```
TYPE
 NumArray = ARRAY[0..9] OF REAL; {OF LONGINT etc}
VAR
  Number : NumArray;
```

An array is a data structure that is a collection/organisation of data all of the same type.

The length of the array determines the number of data elements within it.

An array can be a single dimension or more.

A logical representation of a single dimension (1D) array is:



If there is an array "score":

Score	37.2	16.8	9.2	1.1	0.4	56.2	17.8	9.2	11.7	23.7
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

How did the data get into the array?

Eg

```
for pos = 0 to 9 do
  print("enter value")
  input(score[pos])
endfor
```

```
sub=0
repeat
  print("enter value")
  input(score[sub])
  sub=sub+1
until sub == 10
```

```
pos=0
while pos ≠ 10 do {≠ or !=}
  print("enter value")
  input(score[pos])
  pos=pos+1
endwhile
```

We could calculate the average

```
sum=0
for sub = 0 to 9 do
  sum = sum + score[sub]
endfor
average = sum/10
print("mean value is: ", average)
```

Can this be done with a repeat or while loop?

```
PROGRAM ValsAboveAverage;
  USES
    CRT, SYSUTILS;
  TYPE
    NumArray = ARRAY[0..9] OF REAL; {OF LONGINT etc}
   VAR
    Number : NumArray;
    Mean : REAL;
    HighCount : LONGINT;

  PROCEDURE GetData;
    VAR
      Pos : LONGINT;
    BEGIN
      FOR pos:= 0 TO 9 DO
        BEGIN
          WRITE('Enter value ',pos, ': ');
          READLN(Number[pos]);
        END;
    END;

  PROCEDURE CalcMean;
    VAR
      Pos : LONGINT;
      Total : REAL;
    BEGIN
      Total := 0;
      FOR pos:= 0 TO 9 DO
         total := total + Number[pos];

      Mean := Total/10;
    END;

  PROCEDURE CountHigher;
    VAR
      Pos : LONGINT;
    BEGIN
      HighCount := 0;
      FOR pos:= 0 TO 9 DO
        IF Number[pos] > Mean THEN
          highcount := highcount + 1;
    END;

  PROCEDURE Display;
    BEGIN
      WRITELN('The mean value is : ',mean:5:2);
      WRITELN('There are ',highcount,' values greater than the mean');
      WRITELN('Press any key to continue');
      READKEY;
    END;

  BEGIN
    GetData;
    CalcMean;
    CountHigher;
    Display;
  END.
```

Why no BEGIN/END? ie

```
FOR pos:= 0 TO 9 DO
   BEGIN
      total := total + Number[pos];
   END;
```

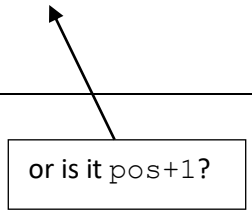
Q: Why are some variables global and others local?
A: Consider where they are needed/required.

Q: What if we want to display the values that are above average (and possibly their position/subscript/index) in the array?
A: We need to combine the display routines within the processing routines, rather than have a separate subprogram:

```
PROCEDURE CalcAndDisplayMean;
    VAR
      Pos : LONGINT;
      Total : REAL;
    BEGIN
      Total := 0;
      FOR pos:= 0 TO 9 DO
         total := total + Number[pos];

      Mean := Total/10;
      WRITELN('The mean value is : ',mean:5:2);
    END;
```

```
PROCEDURE CountandDisplayHigherValues;
    VAR
      Pos : LONGINT;
    BEGIN
      HighCount := 0;
      FOR pos:= 0 TO 9 DO
         IF Number[pos] > Mean THEN
           BEGIN
             highcount := highcount + 1;
             WRITELN('Value ',Number[pos]:5:2,' is larger than the mean');
             WRITELN('which is in position ',pos,' in the list');
             WRITELN;
           END;
    END;
```



Data Types and Data Structures

There are four primitive data types:

- integer
- real (float)
- character
- Boolean

A data structure is a method of organising a collection of data items so that a progam can organise and process them within a single structure. For example the array data structure allows data items of the same data type to be stored and manipulated using a single identifier (and the use of subscripts/indexes).

String data is often considered a primitive data type since it is used so frequently, however it is actually an array of characters. So the string 'Hermoust' stored in Name (declared as `STRING`) is actually stored as:

Name	'H'	'e'	'r'	'm'	'o'	'u'	's'	't'				
	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	…	…	…	…

The two most common data structures available in programming languages are arrays and records.

Arrays allow data of the same data type to be collected/grouped/stored together using one identifier and can be multi-dimensional; one-dimensional array, two-dimensional array, three-dimensional array etc.

Records allow data of different data types to be collected/grouped/stored together using one identifier. Each item of data in a record is called a field. These records can then be written (output) to external storage in a file.

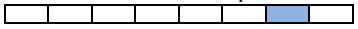
Example Pascal `TYPE`/`VAR` statements:

Arrays

```
TYPE
  MarkArray = ARRAY[0..29] OF LONGINT;

VAR
  Mark : MarkArray;
```

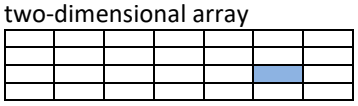
one-dimensional array



```
TYPE
  MonthArray = ARRAY[1..12] OF STRING[3];

VAR
  MonthName : MonthArray;
```

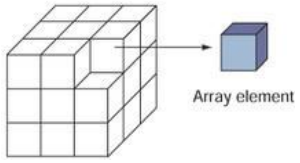
two-dimensional array



```
TYPE
  TestResArray = ARRAY[0..29,1..12] OF REAL;

VAR
  TestRes : TestResArray;
```

three-dimensional array




```
TYPE
  SalesArray = ARRAY[0..10,1..12,0..5] OF REAL;

VAR
  Sales : SalesArray;
```

Record

```
TYPE
  EmployeeRec = RECORD
                  Surname    : STRING;
                  Forename   : STRING;
                  Salary     : REAL;
                  YrsService : LONGINT;
                  InPension  : BOOLEAN;
                END;
VAR
  Employee : EmployeeRec;
```

This declares one record with five fields.

(Could limit a field size, eg `Surname` could be `STRING[35]`)

A data structure can encompass other data structures, for example a record can have an array as the data structure required for one of the fields, or a field may require a record to store its data, eg:

```
TYPE
  DateRec = RECORD
              Day,
              Month,
              Year  : LONGINT;  // or Day,Month,Year : LONGINT;
            END;

  TestResultsArray = ARRAY[1..6] OF LONGINT;

  StudentRec = RECORD
                 Surname    : STRING[35];
                 Forename   : STRING[35];
                 TutorGrp   : STRING[5];
                 DoB        : DateRec;   //or use TDATETIME
                 TestResults : TestResultsArray;
               END;
VAR
  Student : StudentRec;
```

Note: a data type needs to be declared before it is used by another data structure. This is the same as procedures having to be declared before they are called. This is called forward declaration.

eg input of a record by the user:

```
WRITE('Enter student surname: ');
READLN(Student.Surname);
WRITE('Enter student forename: ');
READLN(Student.Forename);
WRITE('Enter ',student.forename,' ',student.surname,'''s tutor group: ');
READLN(Student.TutorGrp);
WRITELN('Enter date of birth, pressing return after each section');
READLN(Student.DoB.Day);
READLN(Student.DoB.Month);
READLN(Student.DoB.Year);
WRITELN('Enter test results for each half term');
FOR times := 1 TO 6 DO
  BEGIN
    WRITE('Enter score for term ',times,': ');
    READLN(Student.TestResults[times]);
  END;
```

Note: we have to use the record identifier (`Student`) to specify which record variable is being used, this can be shortened by the `WITH` statement many fields are being used:

```
WITH Student DO
  BEGIN
    WRITE('Enter student surname: ');
    READLN(Surname);                    //no need for student.surname
    WRITE('Enter student forename: ');
    READLN(Forename);                   //no need for student.forename etc
    WRITE('Enter ',forename, ' ',surname,'''s tutor group: ');
    READLN(TutorGrp);
      :
    : etc
  END;
```

We can also have an array of records, eg:

```
TYPE
  EmployeeRec = RECORD
                  Surname   : STRING;
                  Forename  : STRING;
                  Salary    : REAL;
                  YrsService : LONGINT;
                  InPension  : BOOLEAN;
                END;

  EmployeeArray = ARRAY[0..19] OF EmployeeRec;

VAR
  Employee : EmployeeArray;
```

To input data, we could use:

```
FOR person := 0 TO NoOfEmployees DO
  BEGIN
    WRITE('Enter Forename: ');
    READLN(Employee[person].Forename);
    WRITE('Enter Surname: ');
    READLN(Employee[person].Surname);
    :
    : etc
  END;
```

or

```
FOR person := 0 TO NoOfEmployees DO
  BEGIN
    WITH Employee[Person] DO
      BEGIN
        WRITE('Enter Forename: ');
        READLN(Forename);
        WRITE('Enter Surname: ');
        READLN(Surname);
        :
        : etc
      END;
  END;
```

If we want to store records in a file on external storage, we need to create a link between the program and the file location on the storage (eg disc, memory stick etc), eg:

```
TYPE
  EmployeeRec = RECORD
                  Surname   : STRING;
                  Forename  : STRING;
                  Salary    : REAL;
                  YrsService : LONGINT;
                  InPension  : BOOLEAN;
                END;

  EmployeeFileType = FILE OF EmployeeRec;

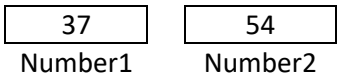
VAR
  Employee     : EmployeeRec;
  EmployeeFile : EmployeeFileType;
```

We need to link the file variable identifier (`EmployeeFile`) with the actual location externally (eg C:, N:, D:, E: etc). We use the `ASSIGN` statement to do this. So if we want to call the file on a memory stick on the E drive 'Employee.dta'), we would use:

```
ASSIGN(EmployeeFile,'E:\Employee,dta');
```

Swapping two values

If we wish to swap the contents of two variables we need to use a third, temporary store/variable. For example consider the variables `Number1` and `Number2` that store 37 and 54 respectively:



If we want to swap them (ie 54 goes to `Number1` and 37 goes to `Number2`), then a common error is to start the process with `Number1 := Number2;`
However, although this correctly puts 54 into `Number1`, it has done so by overwriting the contents of `Number1` and the value 37 is therefore not available any more to place into `Number2`, instead we have both `Number1` and `Number2` storing 54. The solution is to create a copy of `Number1` (with the value 37) in a temporary store before we overwrite its contents, and then place the contents of this temporary store into `Number 2`, so we have:

```
Temp := Number1;
Number1 := Number2;
Number2 := Temp;
```

Linear Search

A method to find/search for a value (eg stored in `SearchVal`) in an array (eg called `Numbers`) is to start at the beginning of the array and compare `SearchVal` with every consecutive item in the array until either an item matches or the end of the array is reached.
Eg for an array:

Numbers	17.1	38.3	67.2	19.0	27.8	42.1	12.3	75.6	45.9	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Pseudocode

```
procedure LinearSearch
  print("Enter value to search for :")
  input(SearchNum)
  pos=-1
  repeat
    pos=pos+1
  until (Number[pos] == SearchNum) OR (pos==9)

  if Number[pos] == SearchNum THEN
    print("Number is in list, at position " ,pos)
  else
    print("Number is not in list")
  endif
endprocedure LinearSearch
```

With a `WHILE` loop it is:

```
procedure LinearSearch
  print("Enter value to search for :")
  input(SearchNum)
  pos=0
  WHILE (Number[pos]) != SearchNum) AND (pos != 9) DO
    pos=pos+1

  if Number[pos] == SearchNum THEN
    print("Number is in list, at position " ,pos)
  else
    print("Number is not in list")
  endif
endprocedure LinearSearch
```

< is better

From OCR textbook

```
pointer=0
WHILE pointer < LengthOfList AND list[pointer] != searchedFor
  Add one to pointer
ENDWHILE
IF pointer >= LengthOfList THEN         //Incorrect see below
  PRINT("Item is not in list")
ELSE
  PRINT("Item is at location "+pointer)
ENDIF
```

This is incorrect: instead of `IF pointer >= LengthOfList THEN`
it should be: `IF list[pointer] != searchedFor THEN`

Why is it incorrect?

A Linear search will work on any array (unsorted or sorted/unordered or ordered) and will stop at the first occurrence of the value in `SearchVal` being found. This is ideal for data that is unique (ie non-repeating data).

Pascal code for a Linear Search:

```
PROGRAM LinearSearch; {for sorted or unsorted data}
  USES
    CRT,SYSUTILS;

  TYPE
    NumArray = ARRAY[0..9] OF REAL;

  VAR
    Numbers : NumArray;
    Sub : LONGINT;
    SearchVal : REAL;

  PROCEDURE InitialiseData;
    BEGIN
      Numbers[0]:=17.1; Numbers[1]:=38.3; Numbers[2]:=67.2; Numbers[3]:=19.0;
      Numbers[4]:=27.8; Numbers[5]:=42.1; Numbers[6]:=12.3; Numbers[7]:=75.6;
      Numbers[8]:=45.9; Numbers[9]:=50.5;
    END;

  PROCEDURE ShowNums;
    BEGIN
      FOR Sub:=0 TO 9 DO
        WRITE(Numbers[Sub]:4:1,'   ');
      WRITELN;
    END;

  PROCEDURE Search;
    BEGIN
      WRITE('Enter value to search for :');
      READLN(SearchVal);

      Sub:=-1;
      REPEAT
        Sub:=Sub+1;
      UNTIL (Numbers[Sub] = SearchVal) OR (Sub=9);

      IF Numbers[Sub] = SearchVal THEN
        WRITELN('Number is in list, at position ',Sub+1,' (subscript ',sub,')')
      ELSE
        WRITELN('Number is not in the list');
    END;


  BEGIN
    InitialiseData;
    WRITELN('Initial data');
    ShowNums;

    Search;

    READKEY;
  END.
```

Why initialise Sub to -1?

or

```
Sub:=0;
WHILE (Numbers[Sub] <> SearchVal) AND (Sub<9) DO
   Sub:=Sub+1;
```

We can use the idea of swapping values (as shown earlier) in sorting data in an array, eg for an array Numbers:

Numbers	17.1	38.3	67.2	19.0	27.8	42.1	12.3	75.6	45.9	50.5
	[0]	[1]	[2]	[3	[4]	[5]	[6]	[7]	[9]	[9]

Array manipulation/consideration usually uses a loop/iteration; eg to display each value in turn we can use:

```
FOR sub := 0 TO 9 DO
  WRITELN(Numbers[sub]);
```

```
sub := 0;
REPEAT
  WRITELN(Numbers[sub]};
  sub := sub+1;
UNTIL sub > 9;
```

```
sub := 0;
WHILE sub <= 9 DO
  BEGIN
    WRITELN(Numbers[sub]};
    sub := sub+1;
  END;
```

If we compare each value in turn with the adjacent/next value and swap them if the first value is greater than the second one (if we want to sort in ascending order), for the above array we would get:

- Test value at subscript [0] with value at subscript [1], ie 17.1 with 38.3, these are in order so leave
- Now test values at [1] and [2], ie test 38.3 with 67.2, these are in order so leave
- Now test values at [2] and [3], ie test 67.2 with 19.0, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	67.2	27.8	42.1	12.3	75.6	45.9	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Continuing the comparisons: now test values at [3] and [4], ie test 67.2 with 27.8, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	27.8	67.2	42.1	12.3	75.6	45.9	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Now test values at [4] and [5], ie test 67.2 with 42.1, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	27.8	42.1	67.2	12.3	75.6	45.9	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Now test values at [5] and [6], ie test 67.2 with 12.3, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	27.8	42.1	12.3	67.2	75.6	45.9	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Now test values at [6] and [7], ie test 67.2 with 75.6, these are in order so leave
- Now test values at [7] and [8], ie test 75.6 with 45.9, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	27.8	42.1	12.3	67.2	45.9	75.6	50.5
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

- Continuing the comparisons: now test values at [8] and [9], ie test 75.6 with 50.5, these are not in order so swap; the array now looks like:

Numbers	17.1	38.3	19.0	27.8	42.1	12.3	67.2	45.9	50.5	75.6
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

We have now considered each pair of values in the array to the end (one “pass”) so the algorithm stops.

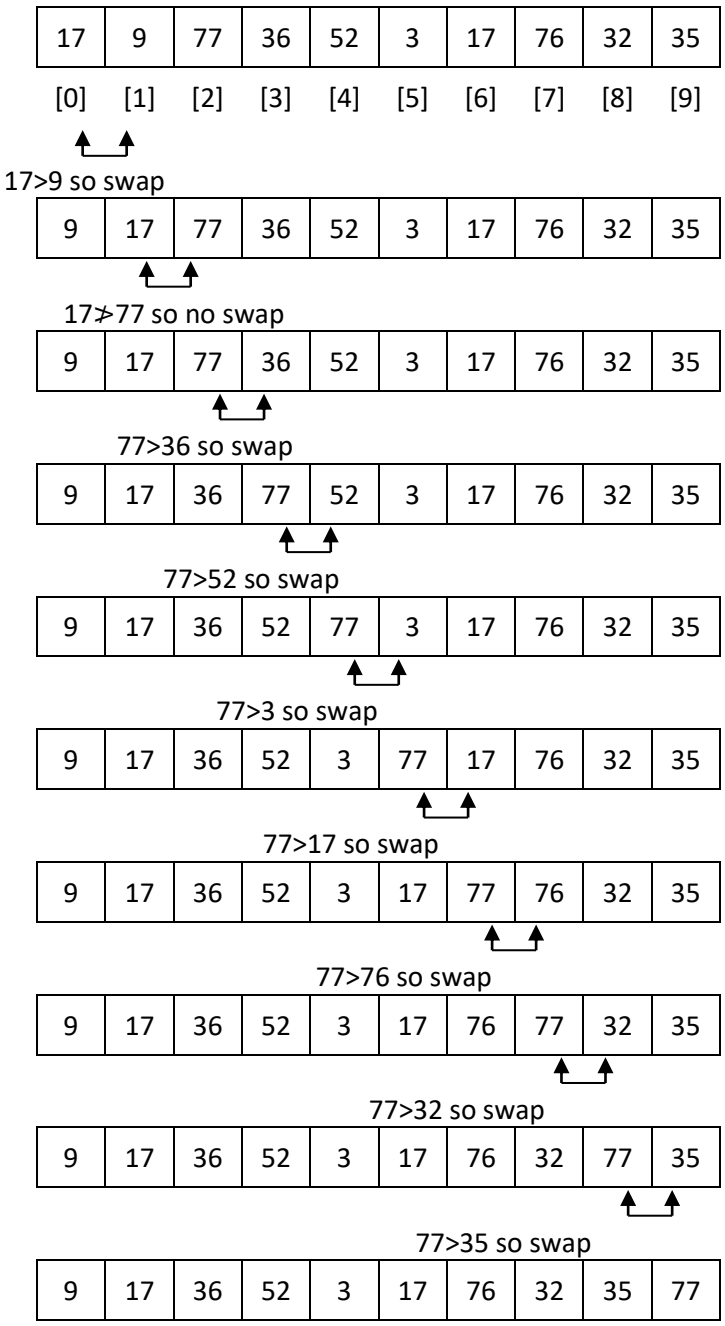
The result of this pass is that the largest value has “bubbled” up to the end of the array. This is the basis of the Bubble Sort. The passes are repeated until the data are fully sorted. (Another pass would result in the next largest value being in the second location from the end; another pass would place the third largest value in the correct location, and so on until sorted.)

Bubble Sort (for N values)

- Start from first position of the data
- Compare adjacent values
  - if not in the correct order (ie first value > second value) then swap **
- Continue comparing until the end of the data
  - This will result in the largest value "bubbling" to the end
- Repeat from the beginning (reducing end comparison by one position each time)
  - Until no swaps have occurred in a pass OR N-1 passes have been made

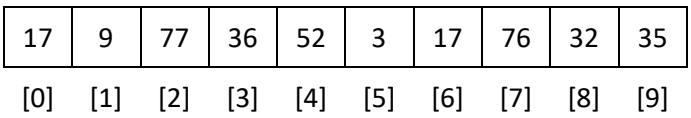
(** for descending order use if first value < second value then swap)

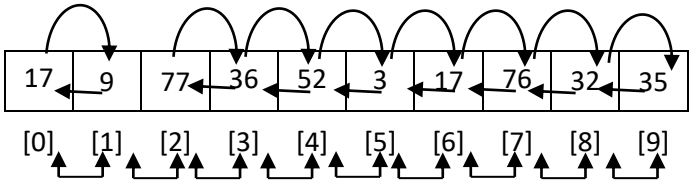
eg First pass for:



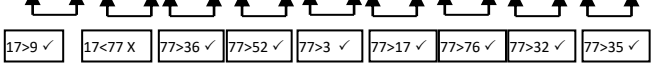
End of first pass. The largest value (77) has "bubbled" to the end.

Bubble Sort

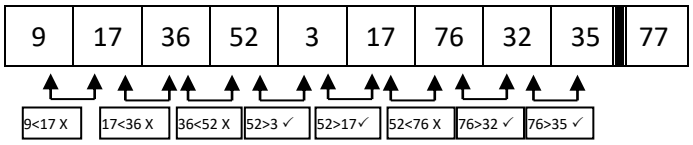




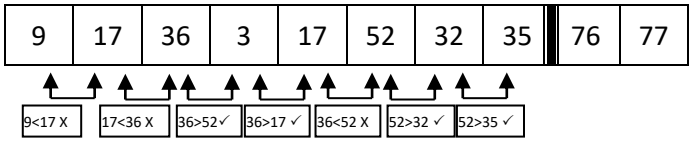
First pass:



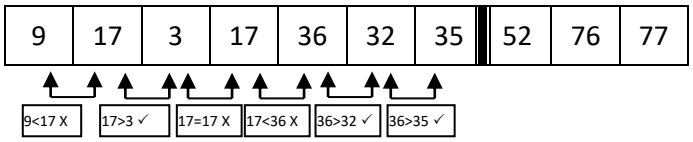
✓ = Swap
X = No swap



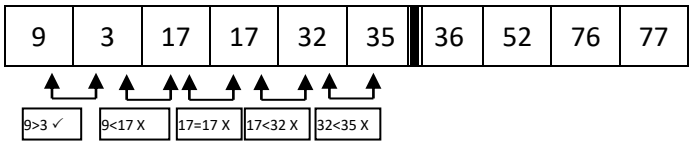
Second pass:



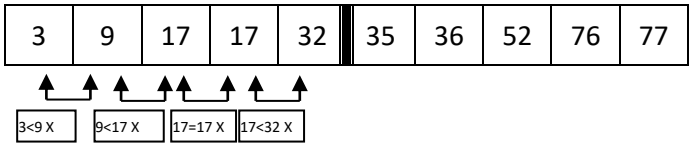
Third pass:



Fourth pass:

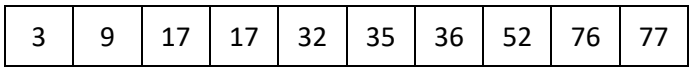


Fifth pass:



Sixth pass:

No swaps required during pass, so stop



Bubble Sort

The code to carry out the pass of the Bubble Sort shown is:

```
PROCEDURE OnePass;
  BEGIN
    FOR Sub := 0 TO 8 DO
      BEGIN
        IF Numbers[Sub] > Numbers[Sub+1] THEN
          Swap;
      END;
  END;
```

Where the procedure `Swap` exchanges/swaps the two values in `Numbers[Sub]` and `Numbers[Sub+1];` using the example of swapping earlier, the code for `Swap` would be (which would have to be declared before `PROCEDURE OnePass` is:

```
PROCEDURE Swap;
  VAR
    Temp : REAL;
  BEGIN
    Temp:=Numbers[Sub];
    Numbers[Sub]:=Numbers[Sub+1];
    Numbers[Sub+1]:=Temp;
  END;
```

If OnePass was carried out 9 times then the array would be sorted (if the 9 largest values are in the correct position, then the tenth (ie the smallest) will also be correct).

So a simple Bubble Sort would be:

```
PROCEDURE Sort;
  VAR
    Times : LONGINT;
  BEGIN
    FOR Times:=0 TO 8 DO
      BEGIN
        OnePass;
      END;
  END;
```

However, this is not very efficient in two ways:

1. In the procedure `OnePass`, the comparisons always go up to the end of the array; this is not necessary. After the first pass, we do not need to consider the last item since it is in the correct position (ie the largest is at the end), so the `FOR` loop only needs to go up to 8; then on the next pass up to 7; then 6 and so on. We can change this by making the `FOR` loop end using a variable (eg `EndPass`) which is decremented (reduced by one) each time `OnePass` is called.
   This changes `OnePass` and `Sort` to:

```
PROCEDURE OnePass;
  BEGIN
    FOR Sub:=0 TO EndPass DO
      BEGIN
        IF Numbers[Sub] > Numbers[Sub+1] THEN
          Swap;
      END;
  END;
```

```
PROCEDURE Sort;
  VAR
    Times : LONGINT;
  BEGIN
    EndPass := 8;
    FOR Times := 0 TO Endpass DO
      BEGIN
        OnePass;
        EndPass := EndPass-1;
      END;
  END;
```

2. The current procedure `Sort` carries out `OnePass` nine times regardless, even if the array becomes sorted before carrying out the next iteration. It is easy to test if the array becomes sorted part way through the sort; if a pass is carried out and no swaps are required, then the array must be sorted. If we use a Boolean variable (eg `Swapped`) that before each iteration in `Sort` is initialised to `FALSE`, and then set to `TRUE` if a swap is carried out (ie set `Swapped` to `TRUE` within the `Swap` procedure), we can continue iterating until `Swapped` remains `FALSE`, ie no swaps (or `NOT Swapped`).
   This changes `Swap`, `OnePass` and `Sort` to:

```
PROCEDURE Swap;
  VAR
    Temp : REAL;
  BEGIN
    Temp := Numbers[Sub];
    Numbers[Sub] := Numbers[Sub+ ];
    Numbers[Sub + 1] := Temp;
    Swapped := TRUE;
  END;
```

```
PROCEDURE OnePass;
  BEGIN
    Swapped := FALSE;
    FOR Sub:=0 TO EndPass DO
      BEGIN
        IF Numbers[Sub] > Numbers[Sub+1] THEN
          Swap;
      END;
  END;
```

```
PROCEDURE Sort;
  VAR
    Times : LONGINT;
  BEGIN
    EndPass := 8;
    REPEAT
      OnePass;
      EndPass := EndPass-1;
    UNTIL (NOT Swapped) OR (EndPass < 0);
  END;
```

Note: `NOT Swapped` is the same as `Swapped = FALSE`.
Also the test for `EndPass = 0` is added for the occasion when the smallest value in the array is initially in the last location and the number of iterations is a maximum.

The code can be written as one procedure (assuming all variables suitable declared):

Pascal	Pseudocode algorithm

`PROCEDURE BubbleSort;`
`  BEGIN`
`    EndPass:=8; //set to array length -2`
`    REPEAT`
`      Swapped:=FALSE;`
`      FOR Sub := 0 TO EndPass DO`
`        BEGIN`
`          IF Numbers[Sub]>Numbers[Sub+1] THEN`
`            BEGIN`
`              Temp := Numbers[Sub];`
`              Numbers[Sub] := Numbers[Sub+ ];`
`              Numbers[Sub + 1] := Temp;`
`              Swapped := TRUE;`
`            END;`
`        END;`
`      EndPass := EndPass-1;`
`    UNTIL (NOT Swapped) OR (EndPass < 0);`
`  END;`
	`procedure BubbleSort;`
	`  endPass = array length -2`
	`  repeat   //or do`
	`    swapped = FALSE;`
	`    for i = 0 to endPass`
	`      if numbers[i]>numbers[i+1] then`
	`        temp = numbers[i]`
	`        numbers[i] = numbers[i+1]`
	`        numbers[i+1] = temp`
	`        swapped = TRUE`
	`      endif`
	`    next i`
	`    endPass = endPass-1`
	`  until (NOT Swapped) OR (endPass<0)`
	`endprocedure;`

Some arrays start at subscript [1] rather than [0]. Check any question carefully The procedure would then change to:

```
PROCEDURE BubbleSort;
  BEGIN
    EndPass:=ArrayLength-1; //eg 9 if array [1] to [10]
    REPEAT
      Swapped := FALSE;
      FOR Sub:=1 TO EndPass DO
        BEGIN
          IF Numbers[Sub] > Numbers[Sub+1] THEN
            BEGIN
              Temp:=Numbers[Sub];
              Numbers[Sub]:=Numbers[Sub+1];
              Numbers[Sub+1]:=Temp;
              Swapped:=TRUE;
            END;
         END;
      EndPass:=EndPass-1;
    UNTIL (NOT Swapped) OR (EndPass=0);
  END;
```

The routine can also be written using a `WHILE` loop: (assuming first index/subscript is 0)

Pascal	Pseudocode algorithm

`PROCEDURE BubbleSort;`
`  BEGIN`
`    EndPass := ArrayLength -2;`
`    Swapped := TRUE;`
`           //to enter loop on first iteration`
`    WHILE (Swapped) AND (EndPass>=0) DO`
`      BEGIN`
`        Swapped:=FALSE;`
`        FOR Sub:=0 TO EndPass DO`
`          BEGIN`
`            IF Numbers[Sub]>Numbers[Sub+1] THEN`
`              BEGIN`
`                Temp:=Numbers[Sub];`
`                Numbers[Sub]:=Numbers[Sub+1];`
`                Numbers[Sub+1]:=Temp;`
`                Swapped:=TRUE;`
`              END;`
`          END;`
`        EndPass:=EndPass-1;`
`      END;`
`  END;`
	`procedure BubbleSort;`
	`  endPass = array length -2`
	`  swapped = TRUE`
	
	`  while swapped AND endPass>=0`
	`    swapped = FALSE`
	`    for i = 0 to endPass`
	`      if numbers[i]>numbers[i+1] then`
	`        temp = numbers[i]`
	`        numbers[i] = numbers[i+1]`
	`        numbers[i+1] = temp`
	`        swapped = TRUE`
	`      endif`
	`    next i`
	`    endPass = endPass-1`
	`  endwhile`
	`endprocedure;`

Bubble Sort algorithm from OCR textbook

```
Set SwapMade to TRUE
WHILE SwapMade is TRUE
  Set SwapMade to FALSE
  Start at position 0
  FOR position=0 TO Listlength -2 //ie the last but one position
    Compare the item at the position you are at with the one ahead of it
    IF they are out of order THEN
      Swap items and Set SwapMade to TRUE
    ENDIF
  NEXT position
ENDWHILE
```

Is this efficient? (No the end of the `FOR` loop is not decremented on each pass.)

Binary Search

A Binary Search is a more efficient search than a Linear Search and for most sets of data will find a value in a faster time; but it ONLY works on sorted/ordered data. The method is to locate the middle element and test it with the value being searched for (eg `SearchVal`). If it is not found, search the lower half if `SearchVal` is less than the middle item, or upper half otherwise. Repeat until found or not present

- The middle value of the set is checked against the search value.
- If it is not equal to it, if the search value is less than the middle value then the second half of the list is discarded, otherwise the first half of the list is discarded.
- This process is continued, halving the size of the list, until the search value is found or not present

A binary search can only be carried out on **SORTED/ORDERED** data. For example the array `Pet`

Pet	bat	cat	dog	elk	gnat	mouse	owl	rat	unau
	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]

Search item: mouse; initially StartSub = 0 and EndSub = 8

- Calculate subscript of middle item;
  - MidSub = (StartSub+EndSub) DIV 2
  - MidSub = (8+0) DIV 2 = 4
- Test SearchItem with `Pet`[MidSub]
  - mouse > gnat (in `Pet`[4])
  - therefore, item is not in first half of the array

Pet	~~bat~~	~~cat~~	~~dog~~	~~elk~~	gnat	mouse	owl	rat	unau
	~~[0]~~	~~[1]~~	~~[2]~~	~~[3]~~	[4]	[5]	[6]	[7]	[8]

- Change StartSub to MidSub **+1** (it becomes 5)

- Calculate subscript of middle item in remaining subarray
  - MidSub = (5+8) DIV 2, = 13 DIV 2, so MidSub = 6
- Test SearchItem with `Pet`[MidSub]
  - mouse < owl (in `Pet`[6])
  - therefore, item is not in second half of the subarray

Pet	~~bat~~	~~cat~~	~~dog~~	~~elk~~	~~gnat~~	mouse	~~owl~~	~~rat~~	~~unau~~
	~~[0]~~	~~[1]~~	~~[2]~~	~~[3]~~	~~[4]~~	[5]	~~[6]~~	~~[7]~~	~~[8]~~

- Change EndSub to MidSub **-1** (it becomes 5)

- Calculate subscript of middle item in remaining subarray
  - MidSub = (5+5) DIV 2, = 10 DIV 2, so MidSub = 5
- Test SearchItem with `Pet`[MidSub]
  - mouse = mouse (in `Pet`[5])
  - therefore, item is found at subscript 5

1. What if newt was searched for?

2. A binary search is carried out on the following list of names:
   List the comparisons that will be examined when searching for Mia.

1	Ava
2	Chloe
3	Ella
4	Evie
5	Grace
6	Imogen
7	Isabella
8	Jessica
9	Lily
10	Mia
11	Ruby
12	Scarlett

Pseudocode for Binary Search

```
procedure BinarySearch
  print('Enter value to search for :')
  input(SearchVal)
  Start=0
  Finish=9
  Found=FALSE

  repeat
    Mid = (Start + Finish) div 2 {DIV is integer division}
    if SearchVal == List[Mid] THEN
      Found=TRUE;
      print("Number is in list, at position ",Mid)
    else
      if SearchVal < List[Mid] then
        Finish = Mid-1
      else
        Start = Mid+1
      endif
    endif
  until (Found) OR (Start > Finish)

  if not(Found) THEN
    print('Number is not in list')
  endif
endprocedure BinarySearch
```

From OCR textbook

```
LowerBound=0
UpperBound=LengthOfList-1
Found=FALSE
WHILE Found=FALSE AND LowerBound != UpperBound
  MidPoint = ROUND((LowerBound+UpperBound)/2)
  IF List[MidPoint] == SearchedFor THEN
    Found=TRUE
  ELSEIF List[MidPoint] < SearchedFor THEN
    LowerBound = MidPoint+1
  ELSE
    UpperBound = MidPoint-1
  ENDIF
ENDWHILE

IF Found == TRUE THEN
  PRINT("Item found at "+MidPoint)
ELSE
  PRINT("Item not in list")
ENDIF
```

Pascal code for a Binary Search

```
PROGRAM BinarySearch {only for SORTED/ORDERED data};
  USES
    CRT,SYSUTILS;
  TYPE
    NumArray = ARRAY[0..9] OF REAL;

  VAR
    Numbers : NumArray;
    StartSub, EndSub, MidSub, Sub : LONGINT;
    SearchVal : REAL;
    Found : BOOLEAN;

  PROCEDURE InitialiseData;
    BEGIN
      Numbers[0]:=12.3; Numbers[1]:=17.1; Numbers[2]:=19.0; Numbers[3]:=27.8;
      Numbers[4]:=38.3; Numbers[5]:=42.1; Numbers[6]:=45.9; Numbers[7]:=50.5;
      Numbers[8]:=67.2; Numbers[9]:=75.6;
    END;

  PROCEDURE ShowNums;
    BEGIN
      FOR Sub:=0 TO 9 DO
        WRITE(Numbers[Sub]:4:1,'   ');
      WRITELN;
    END;

  PROCEDURE Search;
    BEGIN
      WRITE('Enter value to search for :');
      READLN(SearchVal);
      StartSub:=0;
      EndSub:=9;
      Found:=FALSE;

      REPEAT
        MidSub := (StartSub + EndSub) DIV 2;

        IF Numbers[MidSub] = SearchVal THEN
          Found:=TRUE
        ELSE
          IF Numbers[MidSub] < SearchVal THEN
            StartSub := MidSub+1
          ELSE
            EndSub := MidSub-1;
      UNTIL (Found) OR (StartSub > EndSub);

      IF Found THEN
        WRITELN('Number found at position ',MidSub+1,' (subscript ',MidSub,')')
      ELSE
        WRITELN('Number is not in list');
    END;

  BEGIN
    InitialiseData;
    WRITELN('Initial data');
    ShowNums;

    Search;

    READKEY;
  END.
```

OR:

```
WHILE (NOT Found) AND (StartSub<=EndSub) DO
  BEGIN
```

```
END;
```

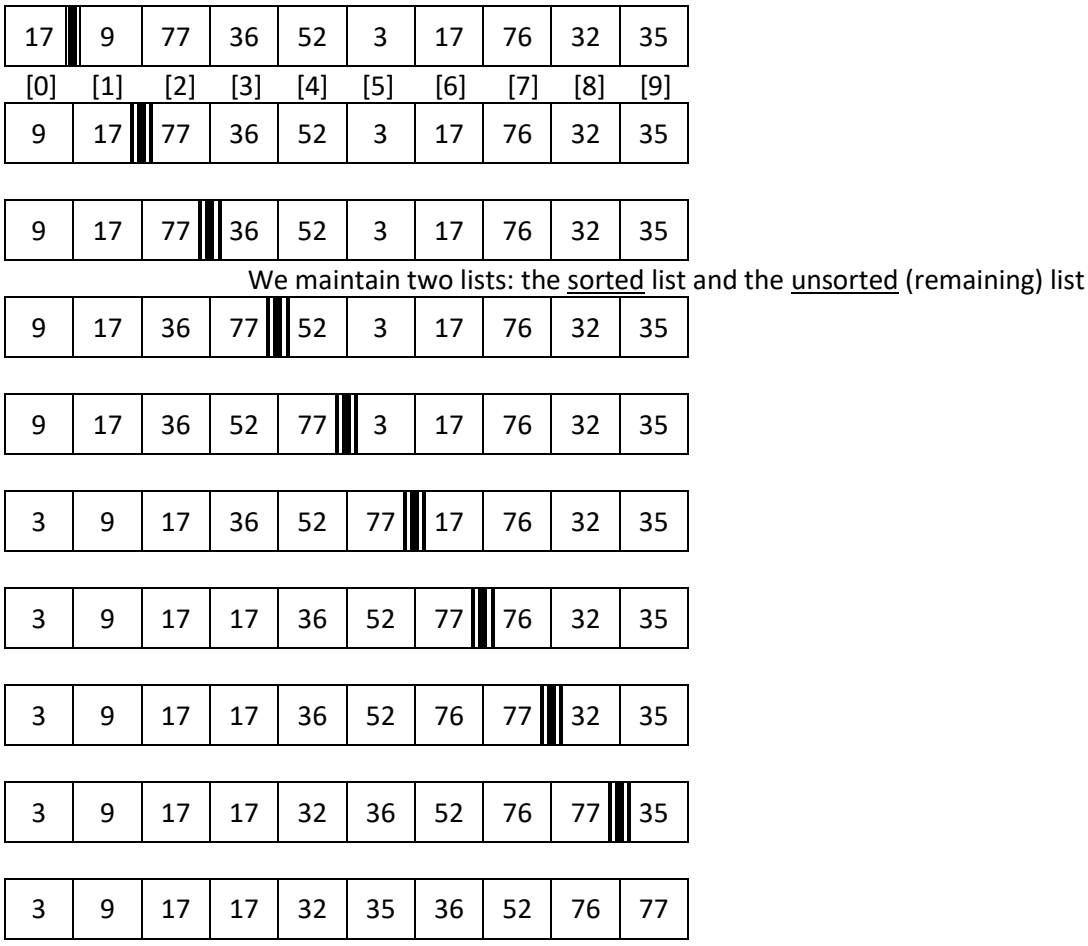
Insertion Sort

Divide the array into two lists; the sorted list and the unsorted list. Initially the first value in the array is the sorted list and the remaining values are in the unsorted list.
Take the first value in the unsorted list and insert it into the correct position in the sorted list (by swapping or shuffling it down to the correct position). The sorted list is now increased by one. Continue inserting the first value from the unsorted list into the sorted list until the unsorted list is empty.

Swapping method for N values

1. Compare second value with first value
    - If Second value < first value then swap
2. Compare third value with previous values
    - Repeat comparison with each adjacent previous value
        - If value < previous value swap them
    - Until value ≮ previous value OR value is now in first position
3. Repeat from point 2, increasing each position by one until compared N^{th} value

At the end of each pass, the next value to be considered has been inserted into its correct position, creating a sorted sub-list each time.



(Always takes N-1 iterations/passes.)

Insertion Sort algorithm from OCR textbook to swap the values

```
Make the first item the sorted list, the remaining items are the unsorted
list

WHILE there are items in the unsorted list
  Take the first item of the unsorted list
  WHILE there is an item to the left of it which is smaller than itself
    Swap with that item
  ENDWHILE

  The sorted list is now one item bigger
ENDWHILE
```

Insertion Sort algorithm from OCR exam to shuffle the value

```
procedure sortit(dataArray, lastIndex)
  for x = 1 to lastIndex
    currentData = dataArray[x]
    position = x
    while (position > 0 AND dataArray[position-1] > currentData)
      dataArray[position] = dataArray[position-1]
      position = position - 1
    endwhile
    dataArray[position] = currentData
  next x
endprocedure
```

Pascal procedures:

```
PROCEDURE InsertionSort;//by swapping
  BEGIN
    FOR FirstUnsort:=1 TO Listlength-1 DO //unsorted list starts at 2nd position
      BEGIN
        Lookpos := FirstUnsort;
        WHILE (List[lookpos] < List[lookpos-1]) AND (lookpos>0) DO
          BEGIN
            Temp:=List[lookpos];
            List[lookpos]:=List[lookpos-1];
            List[lookpos-1]:=Temp;

            lookpos:=lookpos-1;
          END;
      END;
  END;
```

or

```
FirstUnsort:=1; {Second position in list}
WHILE FirstUnsort < Listlength DO
  BEGIN
```

```
    FirstUnsort := FirstUnsort+1;
  END;
```

or

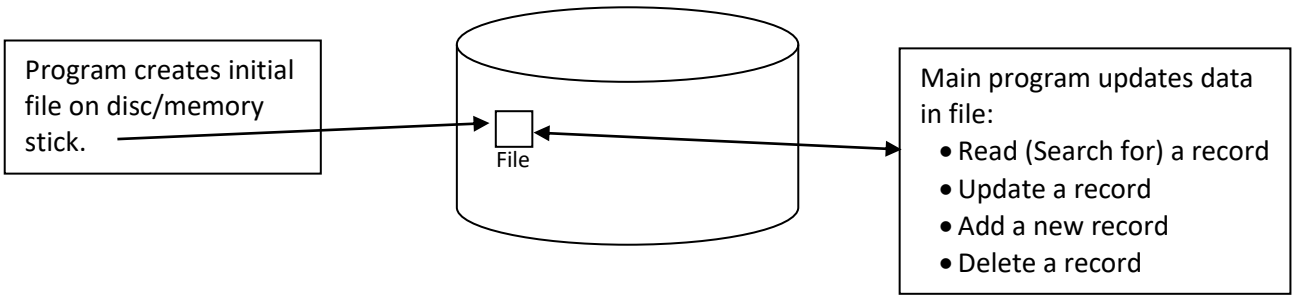
```
PROCEDURE InsertionSort;//by shuffling
  BEGIN
    FOR FirstUnsort:=1 TO Listlength-1 DO
      BEGIN
        CurrentData:=List[FirstUnsort];
        Position:=FirstUnsort;
        WHILE (Position>0) AND (List[Position-1]>currentData) DO
          BEGIN
            List[Position]:=List[Position-1];
            Position:=Position-1;
          END;
        List[Position] = CurrentData;
      END;
  END;
```

Files and Records

You need to consider the structure of the records for the file required. For example a stock file for supermarket products may require:

Field Name	Data Type	Field Length	Eg, justification
ProductID (Barcode)	String/Text	6 bytes	Key field, unique identifier, eg “12345X”
Description	String/Text	20 bytes	Brief description to print on receipts, eg “Beans 120g”
Price	Real	4 bytes	To calculate overall total to pay, eg “0.32”
NoInStock	Integer	2 bytes	Used in stock control testing, eg “315”
ReorderLevel	Integer	2 bytes	Used to test whether new stock needs ordering eg “75”
SupplierCode	String/Text	4 bytes	Used in sending re-order letters, eg “AB12”

An initial file needs to be created in a separate program. This program needs to be executed once, then the file can be maintained in the main program.



For example:

- A record would be read (input from the file) when a product id is input (eg barcode being scanned) and the corresponding record is searched for and read (input) to the program to retrieve the price to add to the current running total of amount to pay by the customer. The details of the product (stored in the record’s fields) will also be output in two locations, printed on the receipt and displayed on a small monitor (why?).
- The record would also be updated (output to the file, overwriting the existing record) when the data in the field containing the number in stock has been changed (eg reduced by one). A record would also be updated if a delivery of that product has arrived and the number of stock will be increased by the number of items delivered. It will also be updated if there is a change in the product’s price.
- A new record would be added if a brand new product not sold by the supermarket before is going to be stocked.
- A product record will be deleted if the supermarket decides that it is not going to stock a particular item again (ie discontinued).

Program to create initial file:

```
PROGRAM CreateInitialFile;
  USES
    SYSUTILS, CRT;
  TYPE
    StockRecord = RECORD
                  Barcode          : STRING[6];
                  Description      : STRING[20];
                  Price      : REAL;
                  NoInStock  : LONGINT;
                END;
    StockFileType = FILE OF StockRecord;

  VAR
    Stock : StockRecord;
    StockFile : StockFileType;


  PROCEDURE InitialFile;
    BEGIN
      ASSIGN(StockFile, 'E:\StockItems.dta');
      REWRITE(StockFile);
      WITH Stock DO
        BEGIN
          Barcode := '12345X';
          Description := 'Beans 120g';
          Price := 0.32;
          NoInStock := 315;
          WRITE(StockFile, Stock);

          Barcode := '673289';
          Description := 'Fish Paste 45g';
          Price := 1.32;
          NoInStock := 89;
          WRITE(StockFile, Stock);

          Barcode := '773344';
          Description := 'Stuffed Anchovies in brine';  {!}
          Price := 0.99;
          NoInStock := 888;
          WRITE(StockFile, Stock);

          :
          etc
          :

        END;
      CLOSE(StockFile);
    END;

  BEGIN
    InitialFile;
  END.
```

Note:

`REWRITE` is the Pascal command to create a new (empty) file.

The previous program has now created a file on backing store (drive E) called `StockItems.dta` .
It can be thought of as:

ID	'12345X'	'673289'	'773344'	'876349';	etc	'99347X'
Desc	'Beans 120g'	'Fish Paste 45g'	'Stuffed Anchovies in'	'Chicken Pot Noodle'		'Beans 240g'
Price	0.32	1.32	0.99	1.04;		0.57
NoInStock	315	89	888	132;		399
	record 0	record 1	record 2	record 3		record N

There is a file pointer associated with each file, and the record where the file pointer is located is where reading or writing occurs. For example if the file pointer is 2, it is positioned at the start of record 2 (Stuffed anchovies), if it is 0 (zero) it is at the start of the file. After every read or write of a record the file pointer automatically moves to the end of that record, ie the start of the next record.

Reserved Pascal files tatements:

`RESET(filename)`	procedure	Moves the file pointer to the start of the file. Is also used to open the file ready for reading and writing records.
`FILESIZE(filename)`	function	Returns the number of records in the file.
`FILEPOS(filename)`	function	Returns the current position of the file pointer, ie the record number it is at the start of.
`SEEK(filename,record number)`	procedure	Moves the file pointer to start of the specified record.
`READ(filename, record variable)`	procedure	Reads the record at the location of the file pointer into the specified variable. (The file pointer moves to the end of the record/start of the next record.)
`WRITE(filename, record variable)`	procedure	Writes the record stored in the specified variable to the file at the location of the file pointer. If the file pointer is at the end of the file, then the record is appended to the file; if the file pointer is at the start of an existing record, it is overwritten with the new data. (The file pointer moves to the end of the record/start of the next record.)
`CLOSE(filename)`	procedure	Flushes the read/write buffer and closes the file; records can longer be read/written (unless `RESET` is then used)
`ASSIGN(filename,external location)`	procedure	Links the file variable in the program with the name and location of the external file on backing store. (Does not open the file – `RESET` or `REWRITE` needs to be used as well.) Only needs to (should) be used once (eg in an initialisation procedure).
`EOF(filename)`	function	Returns `TRUE` if the file pointer is at the end of the file (ie at the end of the last record), `FALSE` otherwise
`REWRITE(filename)`	procedure	Opens the file ready for records to be written to it. It creates an empty file.

Parts of the program to keep the file up-to-date/to maintain the file:

```
PROGRAM MaintainFile;
  USES
    SYSUTILS, CRT;

  TYPE
    StockRecord = RECORD
                  Barcode    : STRING[6];
                  Description : STRING[20];
                  Price      : REAL;
                  NoInStock  : LONGINT;
                END;
    StockFileType = FILE OF StockRecord;

  VAR
    Stock : StockRecord;
    StockFile : StockFileType;
```

The main program initialisation section will be similar to the create files program since it will be accessing the same files. (In reality the user-defined identifiers could be different, but the data types of the fields must be the same and in the same order.)

```
PROCEDURE DisplayFile;
  BEGIN
    RESET(StockFile);

    WHILE NOT EOF(StockFile) DO
      BEGIN
        READ(StockFile, Stock);
        WITH Stock DO
          BEGIN
            WRITELN('Code: ',Barcode);
            WRITELN('Item: ',Description);
            WRITELN('Cost: £',Price:4:2);
            WRITELN('Number in stock: ',NoInStock);
          END;
      END;
  END;
```

The above procedure reads each record in turn and outputs the contents of each field in each record. `RESET` moves the file pointer to the start of the file. A loop is then used to read each record in turn until the end of the file (`EOF`) is reached. (After each `READ`, the file pointer moves on to the next record.)

```
PROCEDURE SearchFile;
  VAR
    SearchCode : STRING[6];
    Found : BOOLEAN;

  BEGIN
    RESET(StockFile);
    WRITELN('Enter barcode to search for');
    READLN(SearchCode);
    SearchCode := UPPERCASE(SearchCode);

    found := FALSE

    REPEAT
      READ(StockFile, Stock);
      IF Stock.Barcode = SearchCode THEN
        BEGIN
          found := TRUE;
          WITH Stock DO
            BEGIN
              WRITELN('Code: ',Barcode);
              WRITELN('Item: ',Description);
              WRITELN('Cost: £',Price:4:2);
              WRITELN('Number in stock: ',NoInStock);
            END;
        END;
    UNTIL (found) OR (EOF(StockFile));

      IF NOT(found) THEN
        WRITELN('Error – Entered barcode not found');
    END;
```

If there may be more than one record meeting the search criterion, then remove the condition for `found`, leaving just the `EOF` test

The above procedure has an input of the ID (barcode) of the record to be found, for example in a supermarket it would be used when the ID (represented as a barcode) is input (scanned), or the cashier enters the code by a keyboard/keypad if the barcode is unreadable (eg due to dirt).

The file pointer is moved to the start of the file (`RESET`) and each record is read in turn into a record variable until the contents of the id field of the record matches the input search code/id. (The data type of the search code variable should be the same as the field being tested.)

If it is possible that the file may become empty (ie no records) then you should test whether a search is possible by:

```
IF FILESIZE(StockFile) = 0 THEN
      WRITELN('Error – empty file')
ELSE
      BEGIN
```

before the `REPEAT` loop.

```
PROCEDURE AmendRecord;
  VAR
    Choice:CHAR;

  BEGIN
    SearchFile;

    WRITELN('Enter item to change');
    WRITELN('1: Barcode');
    WRITELN('2: Description');
    WRITELN('3: Price');
    WRITELN('4: Number in stock');
    WRITELN;

    choice := UPCASE(READKEY);
    WRITELN(choice);

    CASE choice OF
      '1':BEGIN
            WRITE('Enter new barcode: ');
            READLN(Stock.Barcode);
          END;
      '2':BEGIN
            WRITE('Enter new description: ');
            READLN(Stock.Description);
          END;
      '3':BEGIN
            WRITE('Enter new Price: ');
            READLN(Stock.Price);
          END;
      '4':BEGIN
            WRITE('Enter new stock level: ');
            READLN(Stock.NoInStock);
          END;
    END;

    SEEK(StockFile, FILEPOS(StockFile) - 1);
    WRITE(StockFile,Stock);
  END;
```

See below for explanation

The above procedure changes the data in one of the fields of the record currently in the variable `Stock`. It is carried out by calling the procedure `SearchFile` and then changing the contents of one of the fields. It might be used to change the price of a product, or to reduce the number in stock by one after a product is sold etc.

Once the field's contents has been changed, it needs to be written back to the file. However, after the `READ`, the file pointer is at the <u>end</u> of the record required, so if `WRITE` is used without moving the file pointer it will overwrite the next record in the file. To move the file pointer back to the start of the record, we need to use `SEEK` and move it back by 1 from the current position which is given by `FILEPOS`; the command to move the file pointer back by 1 from its current position is:

```
SEEK(StockFile, FILEPOS(StockFile) - 1);
```

The data in the record variable with the new data has now overwritten the record in the file with the old data.

```
BEGIN {of main program}
  ASSIGN(StockFile,'E:\StockItems.dta');
  RESET(StockFile);

  WRITELN('Welcome to file update program');
  WRITELN;

  REPEAT
    CLRSCR;
    WRITELN('A: Display all stock');
    WRITELN('B: Find a product');
    WRITELN('C: Amend an item');
    WRITELN('D: Add a new product');
    WRITELN('E: Delete a product');
    WRITELN;
    WRITELN('F: Quit');
    WRITELN;
    WRITE('Enter choice -> ');
    MenuChoice := UPCASE(Readkey);
    WRITELN(MeuChoice);

    CASE MenuChoice OF
      'A' : DisplayFile;
      'B' : SearchFile;
      'C' : AmendRecord;
      'D' : AddRecord;
      'E' : DeleteRecord;
      'F' : WRITELN('Goodbye');
    ELSE
      WRITELN('Error – only enter A-E. Try again. ');
    END;

  UNTIL MenuChoice = 'F';

  CLOSE(StockFile);
END.
```

The above is the main program for the update program. It carries out the linking of the file variable with the actual location of the file on backing store (`ASSIGN`) and opens it for use (`RESET`). It then uses a menu to allow the user to selct which routines s/he wants to use in a loop until exit is chosen.

In reality it is likely that there will be many different files being used b the program and these will be initialised in a separate procedure. Also, the menu choices require quite a few programming statements and should be put into a procedure. The main program would then look like:

```
BEGIN
  InitialiseFiles;
  MainMenu;
  CloseFiles;
END.
```

Much neater!

To delete a record, we can mark it for deletion, eg use a rogue value in the key field such as -1 which can then be ignored in routines, eg:

```
PROCEDURE DeleteRecord;
  VAR
   Choice:CHAR;

  BEGIN
   SearchFile;

   WRITELN('Delete item - are you sure? (Y/N) ');
   choice := UPCASE(READKEY);
   WRITELN(choice);

   IF choice = 'Y' THEN
    BEGIN
      Stock.Barcode := '-1';
      SEEK(StockFile, FILEPOS(StockFile) - 1);
      WRITE(StockFile,Stock);
      WRITELN('Item deleted');
    END;
  END;
```

When displaying a file, "deleted" records can be ignored, eg in `PROCEDURE DisplayFile;` we can add:

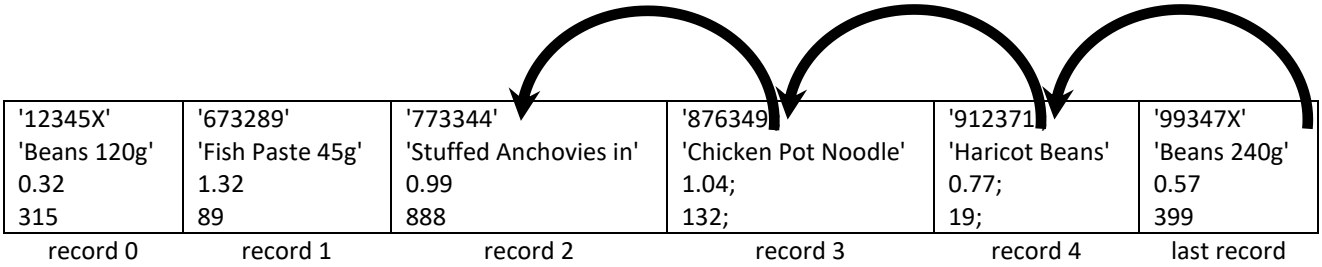
```
 WHILE NOT EOF(StockFile) DO
   BEGIN
    READ(StockFile, Stock);
    WITH Stock DO
     BEGIN
       IF Barcode <> '-1' THEN
        BEGIN
         WRITELN('Code: ',Barcode);
         WRITELN('Item: ',Description);
         WRITELN('Cost: £',Price:4:2);
         WRITELN('Number in stock: ',NoInStock);
        END;
     END;
   END;
```

Permanently deleting a record from a file

A record can be permanently deleted by "shuffling" all the records after it down by one, and then reducing (truncating) the file length by one:

'12345X' 'Beans 120g' 0.32 315	'673289' 'Fish Paste 45g' 1.32 89	'773344' 'Stuffed Anchovies in' 0.99 888	'876349'; 'Chicken Pot Noodle' 1.04; 132;	'912371'; 'Haricot Beans' 0.77; 19;
record 0	record 1	record 2	record 3	record 4

To delete record with id '773344', shuffle each record after it down by one place:



'12345X' 'Beans 120g' 0.32 315	'673289' 'Fish Paste 45g' 1.32 89	'773344' 'Stuffed Anchovies in' 0.99 888	'876349 'Chicken Pot Noodle' 1.04; 132;	'912371 'Haricot Beans' 0.77; 19;	'99347X' 'Beans 240g' 0.57 399
record 0	record 1	record 2	record 3	record 4	last record

which gives

'12345X' 'Beans 120g' 0.32 315	'673289' 'Fish Paste 45g' 1.32 89	'876349'; 'Chicken Pot Noodle' 1.04; 132;	'912371'; 'Haricot Beans' 0.77; 19;	'99347X' 'Beans 240g' 0.57 399	'99347X' 'Beans 240g' 0.57 399
record 0	record 1	record 2	record 3	record 4	last record

then we can truncate/reduce the file length by one:

'12345X' 'Beans 120g' 0.32 315	'673289' 'Fish Paste 45g' 1.32 89	'876349'; 'Chicken Pot Noodle' 1.04; 132;	'912371'; 'Haricot Beans' 0.77; 19;	'99347X' 'Beans 240g' 0.57 399
record 0	record 1	record 2	record 3	last record

```
PROCEDURE DeleteRecord;
VAR
Choice:CHAR;

BEGIN
SearchFile;

WRITELN('Delete item - are you sure? (Y/N) ');
choice := UPCASE(READKEY);
    WRITELN(choice);

IF choice = 'Y' THEN
BEGIN
        WHILE FILEPOS(StockFile) <> FILESIZE(StockFile) DO
  BEGIN
    READ(StockFile,Stock);
    SEEK(StockFile,FILEPOS(StockFile)-2;
    WRITE(StockFile,Stock)
  END;
TRUNCATE(StockFile);
END;
END;
```

Stops at required record (FILEPOS is at the end of the record, ie the stat of the next record).
What if it's not found?

Trace this carefully!
(Why -2?)

We can **add** a new record in the place of a "deleted" record, or at the end of the file if there are no "deleted" records (if the file order does not matter), First use a procedure to get the new record data into a temporary record, eg:

```
PROCEDURE GetData(VAR TempStock:StockRecord);
  BEGIN
    WITH TempStock DO
      BEGIN
        WRITE('Enter Barcode: ');
        READLN(Barcode);
        WRITE('Enter Description: ');
        READLN(Description);
        WRITE('Enter Price: ');
        READLN(Price);
        WRITE('Enter Number in stock: ');
        READLN(NoInStock);
      END;
  END;
```

The add procedure can then call GetData and then search for a record marked for deletion to overwrite (if there aren't any, the record will be added to the end of the file).

```
PROCEDURE AddRecord;
  VAR
    NewStock : StockRecord;

  BEGIN
    GetData(NewStock);
    RESET(StockFile);
    REPEAT
      READ(StockFile, Stock);
    UNTIL (Stock.Barcode = '-1') OR (EOF(StockFile));

    IF Stock.Barcode = '-1' THEN
      SEEK(StockFile, FILEPOS(StockFile)-1);

    WRITE(StockFile,NewStock);
  END;
```

Records are read in turn until one marked for deletion is found, or the end of the file is reached.

If a "deleted" record is found, the file pointer must be moved back to the start of the record to overwrite it. (Remember, after a `READ`, the file pointer will be at the end of the record just read.) If no "deleted" record has been found the file pointer will be at the end of the file and does not need to be moved.

Note – Boolean tests:

REPEAT : : UNTIL found = TRUE;	are the same as	REPEAT : : UNTIL found;
REPEAT : : UNTIL found = FALSE;		REPEAT : : UNTIL NOT(found);
IF continue = TRUE THEN BEGIN : : END;		IF continue THEN BEGIN : : END;

Fields in a record can be of any data type, eg an array:

TYPE ScoreArray = ARRAY[1..10] OF REAL; VAR Score : ScoreArray;	TYPE ScoreArray = ARRAY[1..10] OF REAL; StudentRecord = RECORD Surname :STRING[30]; Forename:STRING[25]; Score :ScoreArray; END; StudentFileType = FILE OF StudentRecord; VAR Student : StudentRecord; StudentFile : StudentFileType;
FOR pos := 1 TO 10 DO WRITELN(Score[pos]);	FOR pos := 1 TO 10 DO WRITELN(Student.Score[pos]);
WRITE('Enter position to change: '); READLN(loc); WRITE('Enter new score: '); READLN(Score[loc]);	WRITE('Enter student: '); READLN(SearchName); RESET(StudentFile); REPEAT READ(StudentFile,Student); UNTIL Student.Surname = SearchName; Newscore := <<calculation>>; Student.Score[loc]:=Newscore; SEEK(StudentFile,FILEPOS(StudentFile)-1); WRITE(StudentFile,Student);

What's <<calculation>> and loc??

```
PROGRAM Parameters;
  USES CRT, SYSUTILS;
  TYPE
    SubjectArray = ARRAY[0..10] OF LONGINT;
    NameArray = ARRAY [0..10] OF STRING[30];
  VAR
    History, ICT, Physics : SubjectArray;
    HistName, ICTName, PhysName : NameArray;

  PROCEDURE Initialise;
    BEGIN
      History[0]:=47; HistName[0]:='Antlet';History[1]:=17; HistName[1]:='Berg';
      History[2]:=45; HistName[2]:='Cwag';  History[3]:=12; HistName[3]:='Drebble';
      History[4]:=78; HistName[4]:='Eek';   History[5]:=5;  HistName[5]:='Fijlip';
      History[6]:=99; HistName[6]:='Gjswik';History[7]:=25; HistName[7]:='Hquist';
      History[8]:=6;  HistName[8]:='Imbult'; History[9]:=77; HistName[9]:='Jokst';
      History[10]:=30; HistName[10]:='Kortwin';
    END;

  PROCEDURE Display(SubjectMark:SubjectArray; SubjectName:NameArray);
    VAR
      pos : LONGINT;
    BEGIN
      FOR pos := 0 to 10 DO
        WRITELN(SubjectMark[pos]:5,' by ', SubjectName[pos]);
      WRITELN;
    END;

  PROCEDURE Sort(VAR SubjectMark:SubjectArray; SubjectName:NameArray);
    VAR
      SwapMade : BOOLEAN;
      Position : LONGINT;
      ListLength : LONGINT;
      Temp : LONGINT;

    BEGIN
      SwapMade := TRUE; Listlength := 11;

      WHILE SwapMade DO
        BEGIN
          SwapMade := FALSE;
          FOR Position := 0 TO Listlength-2 DO
            BEGIN
              IF SubjectMark[Position] > SubjectMark[Position+1] THEN
                BEGIN
                  Temp := SubjectMark[Position];
                  SubjectMark[Position] := SubjectMark[Position+1];
                  SubjectMark[Position+1] := Temp;

                  SwapMade := TRUE;
                END;
            END;
        END;
    END;

  BEGIN
    Initialise;
    Display(History, HistName);
    Sort(History, HistName);
    Display(History, HistName);

   READLN;
  END.
```

- The program is in Teams> Files> Class Materials>ParameterSort.pas
- Make a copy in your area.
- Run the program.
- Identify its purpose; does it work?
- Remove `VAR` from the brackets of `PROCEDURE Sort`, what happens? Why?
- Add `VAR` to the brackets of `PROCEDURE Display`, what happens? Why?
- Make any corrections you perceive are required

Identify:

global variables	local variables	Note: Some terms are incorrect!
global parameters	local parameters	
formal variables	actual variables	
formal parameters	actual parameters	
pass-by-reference parameters	pass-by-value parameters	
pass-by-reference variables	pass-by-value variables	
scope of parameters	scope of variables	

Subprograms (Procedures and Functions), Parameters and local/global variables

```
procedure PTest1(x:byVal,y:byRef,z:byVal)
  p = x+7
  x = 0
  y = y*y
  q = z+p
  print(x)
  print(y)
endprocedure
```

```
function FTest2(a,b:byVal)
  temp = a.substring(b,2)
  return temp
endfunction
```

1. What are `p` and `q`?
2. Explain what will happen with the call:
   `PTest1(1,2,3)`
3. Explain what will happen with the calls:
   a) `FTest2("Hello",3)`
   b) `print(FTest2("Hello",3))`
   c) `j=FTest2("Hello",3)`
4. What data type is the variable j in 3c)?
5. Explain what will happen as a result of carrying out the following statements in the sequence given:
   ```
   k="go left, then right"
   m=FTest2(k,7)
   ```
6. Explain what will happen as a result of carrying out the following statements in the sequence given:
   ```
   e=1
   f=2
   g=3
   print(e)
   print(f)
   PTest1(e,f,g)
   print(e)
   print(f)
   ```
7. From all the questions and above, identify the formal and actual parameters.
8. Identify the scope and type of each variable used

Useful Pascal Library Routines

Procedures

`VAL(s1,n1,i1);`	One string parameter, `s1`; one numeric, `n1` (real or integer); and one integer, `i1`.
	Tries to convert `s1` into a number and stores it in `n1` if it is in a valid format (eg '23', '17.63', '0.02' etc). If `s1` is not a valid format, `n1` is undefined and `i1` is set to the position of the first error, (set to zero if no error);
	eg
	`READLN(Code);` {`Code` is a string eg '1234'}
	`VAL(Code,CodeNum,ErrCode);`
	`IF ErrCode <> 0 THEN`
	`BEGIN`
	`WRITELN('Error in data – must be valid number');`
	`WRITELN('Press any key');`
	`READKEY;`
	`END;`

`STR(n1,s1);`	Two parameters: one number, `n1` (real or integer) and one string variable, `s1`. Converts a number into its equivalent string format: eg `STR(No,NoStr);` (if `No` is integer and 27, `NoStr` is '27')(Could use `INTTOSTR`) `STR(Num,NumStr);` (If `Num` is string and 23.45, `NumStr` is ' 2.34500000000000E+0001') `STR(Num:5:2,NumStr);` (If `Num` is string and 23.45, `NumStr` is '23.45') `STR(Num:8:2,NumStr);` (If `Num` is string and 23.45, `NumStr` is '▽▽▽23.45') ▽=space
`TEXTCOLOR(p1);`	One parameter, `p1` which is a colour label or number. Changes the colour of the output text from the call onwards: eg `TEXTCOLOR(Yellow);` or `TEXTCOLOR(14);` changes the text to yellow 0-Black; 1-Blue; 2-Green; 3-Cyan; 4-Red; 5-Magenta; 6-Brown; 7-White; 8-Grey; 9-LightBlue; 10-Light Green; 11-Light Cyan; 12-Light Red; 13-Light Magenta; 14-Yellow; 15-High-intensity White
`TEXTBACKGROUND(p1);`	One parameter, `p1` which is the colour label or number. Changes the colour of the screen background from the call onwards: eg `TEXTBACKGROUND(Green);` or `TEXTBACKGROUND(2);` both give green. Can use colours 0-7 from above.
`DELETE(s1,i1,i2);`	Three parameters, one string, `s1` and two integers, `i1` and `i2`. Removes `i2` characters from `s1` starting at character `i1`: eg if Code contains 'hello over there' `DELETE(Code,3,6);` would change `Code` to 'heer there'.

Functions

`LENGTH(s1)`	One string parameter, `s1`.
	Returns the number of characters (length) of the string; eg
	`READLN(IdCode);`
	`IF LENGTH(IdCode) <> 5 THEN`
	`BEGIN`
	`WRITELN('Error in code – must be 5 characters long');`
	`WRITELN('Press any key');`
	`READKEY;`
	`END;`

`COPY(s1,i1,i2)`	One string parameter, `s1` and two integer parameters, `i1` and `i2`. Returns substring from `s1` starting at position `i1` for `i2` characters: eg `READLN(SeatId);` {eg 'A12'} `Row := COPY(SeatCode,1,1);` {Row becomes 'A'} `NumStr := COPY(SeatCode,2,1);` {NumStr becomes '1'} `NoStr := COPY(SeatCode,2,LENGTH(SeatCode)-1);` {copy from position 2. No of characters = length-1} {NoStr becomes '12'}

INTTOSTR(i1)	One integer parameter, i1. Returns the converted integer as a string: eg READLN(IdCode); {IdCode:STRING; eg '56'} READLN(Initials); {Initials:STRING; eg 'GM'} VAL(IdCode,Id,ErrCode); {Id = 56} IF ErrCode <> 0 THEN Id:=Id+1; {Id = 57} IdCode:=INTTOSTR(Id); {IdCode = '57'} FullId:=Initials+IdCode; {FullId = 'GM57'}
CHR(i1)	One integer parameter, i1. Returns the character with the passed ASCII (ordinal) value: eg WRITELN(CHR(156)); {displays '£'}
ORD(c1)	One character parameter, c1. Returns the ASCII (ordinal) value of the character passed: eg WRITELN(ORD('g')); {displays 103}
READKEY	No parameter. Returns the character pressed on the keyboard (and does not display it on the monitor): eg InputChar := READKEY;
UPCASE(c1)	One character parameter, c1. Returns the uppercase character of the variable passed as the parameter if it is a letter; other symbols are not converted: eg CapLetter := UPCASE(Letter); {if letter is 'a', CapLetter is 'A'} InputChar := UPCASE(READKEY); WRITELN(InputChar);
UPPERCASE(s1) LOWERCASE(s1)	One string parameter, s1. Returns uppercase, or lowercase, of the passed string: eg READLN(Surname); Surname := UPPERCASE(Surname); {eg 'Puet' becomes 'PUET'} ShowSurname := LOWERCASE(Surname); {eg 'PUET' ShowSurname[1] := UPCASE(Surname[1]); becomes 'Puet'}
POS(s1,s2)	Two string parameters, s1 and s2. Returns the integer value of the first position where s1 occurs within s2, or 0 otherwise: eg WRITELN(POS(Sub,Code)); will display 17 for Sub of '123' and Code of 'This is a test, 12345'; will display 0 for Sub of 'This is a test, 12345' and Code of '123'; will display 2 for Sub of 'X' and Code of 'AX BX CX'.
COMPARESTR(s1,s2)	Two string parameters, s1 and s2. Returns Integer result; 0 if the same, case sensitive, or difference of first ASCII values, can be >0 or <0): eg WRITELN(COMPARESTR(Entry1,Entry2)); will display 0 for Entry1 of 'abc' and Entry2 of 'abc'; will display 32 for Entry1 of 'abc' and Entry2 of 'ABC'; will display -32 for Entry1 of 'ABC' and Entry2 of 'abc'.
SAMETEXT(s1,s2)	Two string parameters, s1 and s2. Returns Boolean result; TRUE if the parameters are the same <u>without</u> case sensitivity, FALSE otherwise.
TRIM(s1)	One string parameter. Removes leading/trailing spaces & control characters
Also TRIMLEFT(s1), TRIMRIGHT(s1)	
RANDOM or RANDOM(i1)	If one integer parameter, then it returns a random integer ≥ 0 and < i1. If no parameter, it returns a value ≥ 0 and < 1 It needs an initial, single call to RANDOMIZE before use.

Date and Time features in Virtual Pascal

Within the SYSUTILS library there are date and time handling subroutines. A variable can be declared of the data type TDATETIME. This allows data to be stored in a format that can then use dates and times in calculations and comparisons. When two dates are subtracted the number of days is stored in the data type/format DOUBLE. (Double stores real numbers but with more bits than the REAL data type; similar to LONGINT and INTEGER.)

An example declaration is:

```
GetDateStr:STRING;
GetDate : TDATETIME;
GetTime : TDATETIME;
Difference : DOUBLE;
```

(Note: both dates and times are of the type TDATETIME)

In this case GetDateStr can accept a date in "normal" STRING format, but the programmer has to convert any data before calculations. Consider the program

```
PROGRAM DateAndTime;
  USES
    CRT, SYSUTILS;

  PROCEDURE CheckDateAndTime;
    VAR
      GetDateStr:STRING;
      GetDate : TDATETIME;
      GetTime : TDATETIME;
      Difference : DOUBLE;

    BEGIN
      GetDateStr := '12/03/2018';  {or READLN(GetDateStr);}
      WRITE(GetDateStr,'  : ');
      WRITELN(STRTODATE(GetDateStr));
      WRITELN;

      GetDate := STRTODATE(GetDateStr);

      WRITE(GetDate);
      WRITELN;

      WRITE('Today is: ',DATE,'   :  ');
      WRITELN(DATETOSTR(DATE));
      Difference := DATE - GetDate;
      WRITELN;
      WRITELN('Difference between the dates is: ',Difference:2:0,' days');
      WRITELN;

      GetTime := TIME;
      WRITELN(TIMETOSTR(GetTime),'  :  ',GetTime);
    END;


  BEGIN
    CheckDateAndTime;
    READLN;
  END.
```

DATE and TIME are reserved functions, so it's best not to use these names as variable identifiers.

```
VP debugging: c:\program files\vp21\out.W32\DateAndTime.exe
12/03/2018  :  4.31710000000000E+0004

 4.31710000000000E+0004
Today is:  4.50850000000000E+0004   :  08/06/2023

Difference between the dates is: 1914 days

10:18:24  :   4.29448530092593E-0001
```

We can extend the program to:

```
PROGRAM DateAndTime;
  USES
    CRT, SYSUTILS;

  PROCEDURE Checkdate;
    VAR
      OldDate : STRING;
      NewDate : TDATETIME;
      Difference : DOUBLE;

    BEGIN
      OldDate := '10/03/2018';
      WRITE(OldDate,'  : ');
      WRITELN(STRTODATE(OldDate));
      WRITELN;

      NewDate := STRTODATE('12'+DATESEPARATOR+'3'+DATESEPARATOR+'18');
      WRITE(NewDate);
      WRITELN;

      WRITE('Today is: ',DATE,'   :  ');
      WRITELN(DATETOSTR(DATE));
      Difference := DATE - STRTODATE(OldDate);
      WRITELN;
      WRITELN('Difference between the dates is: ',Difference:2:0,' days');
      WRITELN;
    END;

  PROCEDURE CheckTime;
    VAR
      OldTime : STRING;
      NewTime : TDATETIME;
      TimeDiff : TDATETIME;

    BEGIN
      OldTime := '08:32:17';
      WRITE(OldTime,'  : ');
      WRITELN(STRTOTIME(OldTime));
      WRITELN;

      NewTime := STRTOTIME('08'+TIMESEPARATOR+'32'+TIMESEPARATOR+'17');
      WRITE(NewTime);
      WRITELN;

      WRITE('The time now is: ',TIME,'  : ');
      WRITELN(TIMETOSTR(TIME));
      TimeDiff := TIME - NewTime;
      WRITELN;
      WRITELN('Difference between the times is: ',TimeDiff);
      WRITELN;
      WRITELN('Difference between the times is: ',TIMETOSTR(TimeDiff));
      WRITELN;
    END;
```

```
PROCEDURE ShowReservedSubroutines;
  CONST
    DayName : ARRAY[1..7] OF STRING[9] = ('Sunday', 'Monday', 'Tuesday',
                                'Wednesday', 'Thursday',
                                'Friday', 'Saturday');
  VAR
    Today, CurrentTime : TDATETIME;
    Day, Month, Year  : WORD;
    Hour, Minute, Second, HunSec : WORD;

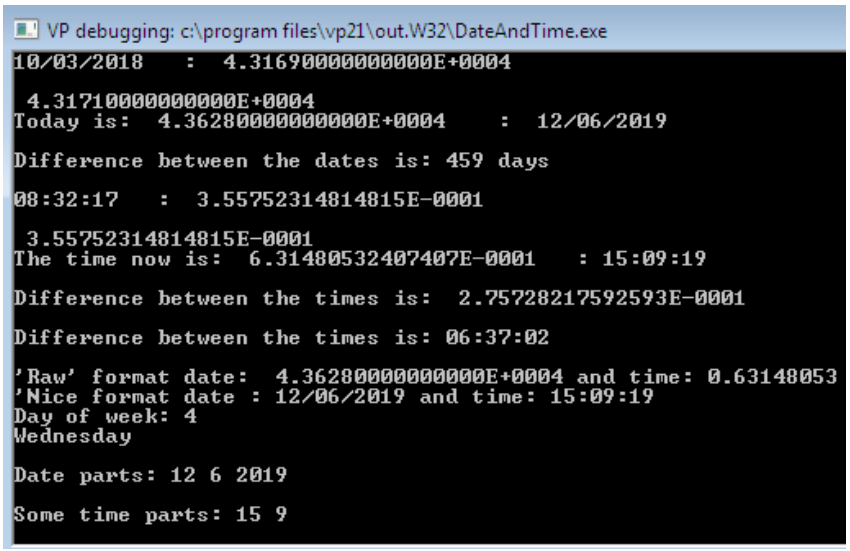
  BEGIN
    Today :=  DATE; //DATE - function returns a value in TDATETIME format
    CurrentTime := TIME; //TIME is a similar function
                  //There is also the function NOW

    WRITELN('''Raw'' format date: ',Today,' and time: ',CurrentTime:10:8);
                {Date returns the number of days since 30/12/1899  !!;
                Time returns the fraction of 24 hours so far}

    WRITELN('''Nice format date : ',DATETOSTR(Today),' and time: '
          TIMETOSTR(CurrentTime));
                             {Use of DATETOSTR and TIMETOSTR}
    WRITELN('Day of week: ',DAYOFWEEK(Today));
                  //DAYOFWEEK returns 1 for Sunday, 2 for Monday etc
    WRITELN(DayName[DAYOFWEEK(Today)]);
    WRITELN;
    DECODEDATE(Today,Year,Month,Day);
                //DECODEDATE procedure separates a variable of type TDATETIME
    WRITELN('Date parts: ',Day,' ',Month,' ',Year);
    WRITELN;

    DECODETIME(CurrentTime, Hour, Minute, Second, HunSec);
    WRITELN('Some time parts: ',Hour,' ',Minute);
  END;


BEGIN
  CheckDate;
  CheckTime;
  ShowReservedSubroutines;
  READLN;
END.
```



The reserved subroutines/data types used:

TDATETIME	DATE	TIME	NOW
DATETOSTR	TIMETOSTR	STRTODATE	STRTOTIME
DAYOFWEEK	DECODEDATE	DECODETIME	

Validation routines in Pascal.

It is always best to enter data from the keyboard into STRING data type variable. This means that the program will not crash due to a type mismatch error (eg entering "five" into a REAL or LONGINT variable). If numeric data is required, it can be converted from string after checking that it is in a valid format.

It is therefore assumed that all data input is into:

```
VAR
  Data : STRING;
```

Presence check. This tests that something/some data is entered into the variable at input:

```
REPEAT
  WRITELN('Enter Surname');
  READLN(Data);
  IF LENGTH(Data) = 0 THEN
    BEGIN
      WRITELN('Error – nothing entered');
      WRITELN('You need to enter something');
      WRITELN('Press any key to continue');
      READKEY;
    END;
UNTIL LENGTH(Data) <> 0;
```

LENGTH is a function that returns the number of characters in a STRING.

Test if an error has occurred.

Repeat the input until there is no error found

Type check. This tests that the data entered is of the expected data type, eg numeric. (in this case, assume that as well as Data, there is another variable Percent, that can be either REAL or LONGINT.)

```
REPEAT
  WRITELN('Enter Percentage');
  READLN(Data);
  VAL(Data,Percent,ErrCode);
  IF ErrCode <> 0 THEN
    BEGIN
      WRITELN('Error – invalid entry');
      WRITELN('You need to enter a number');
      WRITELN('Press any key to continue');
      READKEY;
    END;
UNTIL ErrCode = 0;
```

VAL is a procedure that converts the contents of STRING variable to a number if in a valid format. (See below.)

Test if an error has occurred.

Repeat the input until there is no error found

VAL procedure has three parameters: the STRING to be tested; a numeric (either REAL or LONGINT) variable to store the number if it is a valid one; and a LONGINT variable to store the error code, which will be 0 if the number is valid, or the position of the first character that is invalid.

Eg for VAL(Data, Percent, ErrCode); if:

Data	ErrCode (Percent:REAL)	ErrCode (Percent:LONGINT)
'97'	0	0
'18.36'	0	3
'3.4.5'	4	2
'1O'	2	2

Keeping a sorted file

A sorted file allows a binary search to be used, and is therefore generally faster to be able to find a required record, especially for larger files.

When a new record is added to the file, a merge or insertion sort method can be used.

For example with a main file of:

1487290	5681945	7255501	7447046	8754889
Napkins	Ready-bake breads	Dental Floss	Pancake/Waffle Mix	Red wine vinegar
£2.60	£2.13	£1.29	£4.90	£1.75
11	61	41	62	24
11	22	53	39	25
A3	B1	A3	B2	A3

If we want to add

6727436
Cauliflower
£4.28
9
43
A1

we need to "shuffle" the three end records down by one so that the new record can be added between "Ready-bake breads" and "Dental Floss"; this is because we start comparing from the end (right) and shuffling up each record until the new ID (barcode) is greater than a record in the file.

Stage 1:



1487290	5681945	7255501	7447046	8754889	8754889
Napkins	Ready-bake breads	Dental Floss	Pancake/Waffle Mix	Red wine vinegar	Red wine vinegar
£2.60	£2.13	£1.29	£4.90	£1.75	£1.75
11	61	41	62	24	24
11	22	53	39	25	25
A3	B1	A3	B2	A3	A3

"Red wine vinegar" shuffled up because 672743 is < 8754889

Stage 2:



1487290	5681945	7255501	7447046	7447046	8754889
Napkins	Ready-bake breads	Dental Floss	Pancake/Waffle Mix	Pancake/Waffle Mix	Red wine vinegar
£2.60	£2.13	£1.29	£4.90	£4.90	£1.75
11	61	41	62	62	24
11	22	53	39	39	25
A3	B1	A3	B2	B2	A3

"Pancake/Waffle mix" shuffled up because 672743 is < 7447046

Stage 3:



1487290	5681945	7255501	7255501	7447046	8754889
Napkins	Ready-bake breads	Dental Floss	Dental Floss	Pancake/Waffle Mix	Red wine vinegar
£2.60	£2.13	£1.29	£1.29	£4.90	£1.75
11	61	41	41	62	24
11	22	53	53	39	25
A3	B1	A3	A3	B2	A3

"Pancake/Waffle mix" shuffled up because 672743 is < 7255501

Stage 4:
The next record is read ("Ready-bake breads") and it is NOT shuffled up because 672743 is NOT < 5681945.
Therefore, we now add the new record to overwrite the original "Dental Floss" record:

1487290	5681945	6727436	7255501	7447046	8754889
Napkins	Ready-bake breads	Cauliflower	Dental Floss	Pancake/Waffle Mix	Red wine vinegar
£2.60	£2.13	£4.28	£1.29	£4.90	£1.75
11	61	9	41	62	24
11	22	43	53	39	25
A3	B1	A1	A3	B2	A3

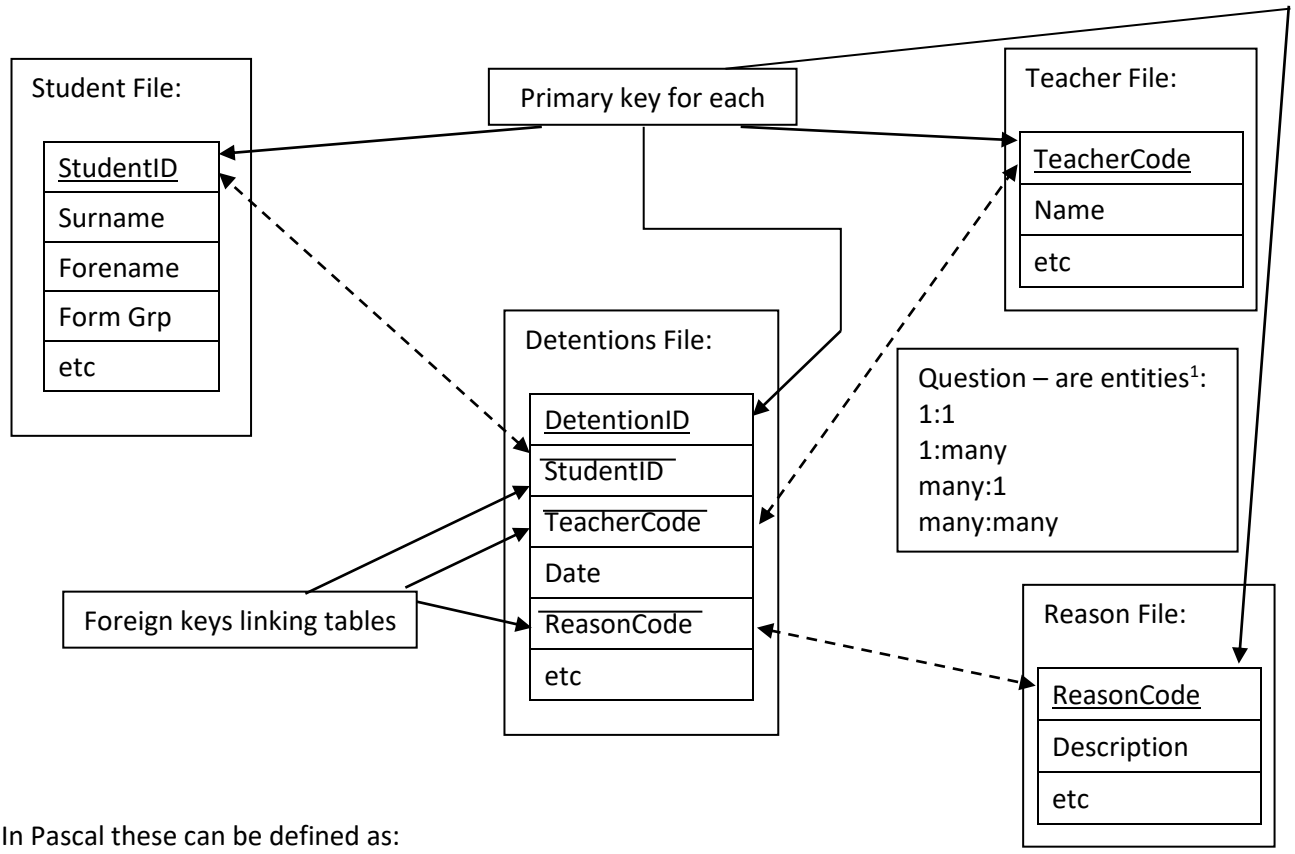
The file remains sorted.

Using Linked Files

When a database is created, the separate tables can be linked by one (or more) common field(s). The final list of tables can be determined by the process of normalisation.

The common field is likely to be the primary key in one of the tables, and used in another table (in which it is called the 'foreign' key).

Example: A detention storing system for a school. This can use three files (tables):



In Pascal these can be defined as:

```
TYPE
```

```
StudentRec = RECORD
               StudentId : STRING[5];
               Surname   : STRING[35];
               Forename  : STRING[30];
               Form      : STRING[5];
             END;
```

```
DetentionRec = RECORD
                 DetentionId : STRING[5];
                 StudentId   : STRING[5];
                 TeacherCode : STRING[3];
                 Date        : TDATETIME;
                 ReasonCode  : STRING[3];
               END;
```

```
TeacherRec = RECORD
               TeacherCode : STRING[3];
               TeacherName : STRING;
             END;
```

```
ReasonRec = RECORD
              ReasonCode  : STRING[3];
              Description : STRING;
            END;
```

[1]Entity: In a database, anything about which information can be stored; for example, a person, concept, physical object or event.

Use initial data of:

Student File

Fields	Record 1	Record 2	Record 3	Record 4
Student ID	00001	00002	00003	00004
Surname	Jenkins	Lermin	Yutrik	Banler
Forename	Phelmin	Jignaut	Almana	Gopan
Form Group	10ADD	9TFF	11SSW	10ADD

Teacher File

Fields	Record 1	Record 2	Record 3
Teacher Code	HMA	DDA	YTO
Name	Mrs H Makleb	Mr D Dankel	Ms Y Tonkmai

Detentions File

Fields	Record 1	Record 2	Record 3	Record 4	Record 5	Record 6	Record 7
Detention ID	A0001	A0002	A0003	A0004	A0005	A0006	A0007
Student ID	00002	00003	00002	00002	00003	00002	00003
Teacher ID	YTO	YTO	HMA	DDA	HMA	YTO	HMA
Date	7/9/2021	7/9/2021	9/9/2021	17/9/2021	17/9/2021	27/9/2021	5/10/2021
Reason	Late to class	Late to class	Rudeness	No Homework	Disrupting others' learning	No effort in class	No homework

To Display the detentions for one student:

- Open Student File
- Find student
    - Search by name or ID
    - Open Detentions File
    - Read each detention record
        - Test if detention student id = required student id {foreign field link}
        - If equal display detention details
            - Display date
            - Display Teacher Name
                - Open Teachers File
                - Read each teacher record
                - Test if teacher code = teacher code who set detention
                - If equal display full teacher name
                - Until teacher found
            - Display Detention Reason
                - Open Reasons File
                - Read each reason record
                - Test if reason code = reason code of the detention
                - If equal display full detention reason
                - Until reason found
    - Until end of detentions file

```
PROCEDURE ShowDetentions;
  VAR
    DetentionCount : LONGINT;

  BEGIN
    FindStudent;

    CLRSCR;
    WRITELN; WRITELN;
    WRITELN('   Detentions for:');
    WRITELN;
    WRITELN('                  ',Student.Forename,' ',Student.Surname);
    WRITELN;
    WRITELN('                  Form Group: ',Student.Form);
    WRITELN;

    DetentionCount:=0;
    RESET(DetentionFile);
    WHILE NOT EOF(DetentionFile) DO
      BEGIN
        READ(DetentionFile,Detention);
        IF Detention.StudentID = Student.StudentID THEN
          BEGIN
            DetentionCount := DetentionCount+1;
            WRITE(' ',DATETOSTR(Detention.Date),': for ');

            RESET(ReasonFile);
            REPEAT
              READ(ReasonFile,Reason);
            UNTIL Reason.ReasonCode = Detention.ReasonCode;

            WRITELN(Reason.Description);

            RESET(TeacherFile);
            REPEAT
              READ(TeacherFile,Teacher);
            UNTIL Teacher.TeacherCode = Detention.TeacherCode;

            WRITELN('          set by ',Teacher.TeacherName);
            WRITELN;
          END;
      END;

      WRITELN;
      IF DetentionCount = 0 THEN
        WRITELN('This student has no detentions')
      ELSE
        WRITELN('A total of ',DetentionCount,' detentions');

      WRITELN;
      WRITELN('Press any key to continue');
      READKEY;
  END;
```

What if not found?

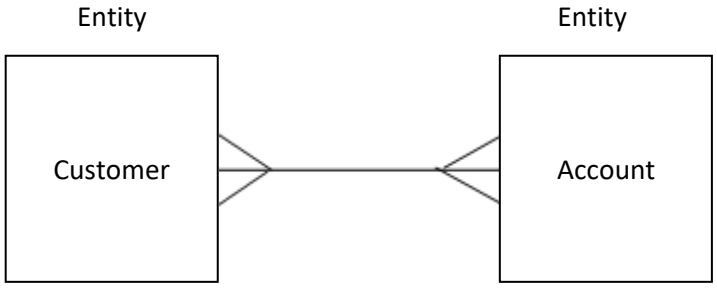
Could put into separate procedures,
eg `FindTeacher;` and `FindDetentionReason;`

Some files/tables are many:many. For example bank customers and their accounts.

1 customer may have more than one account (1:many)

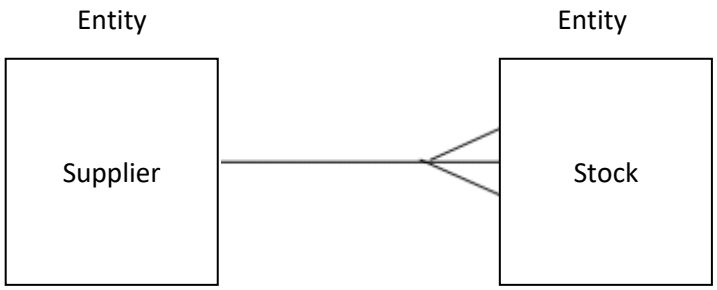
1 account may be owned by more than one customer (eg a joint account) (1:many)

so customer:account is many:many



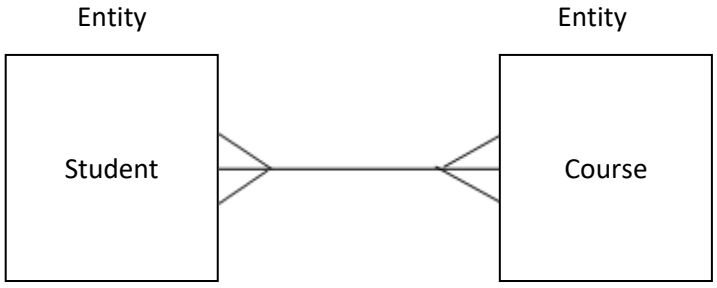
Note: a 1:many relationship would be shown by  (in the correct direction!)

eg:

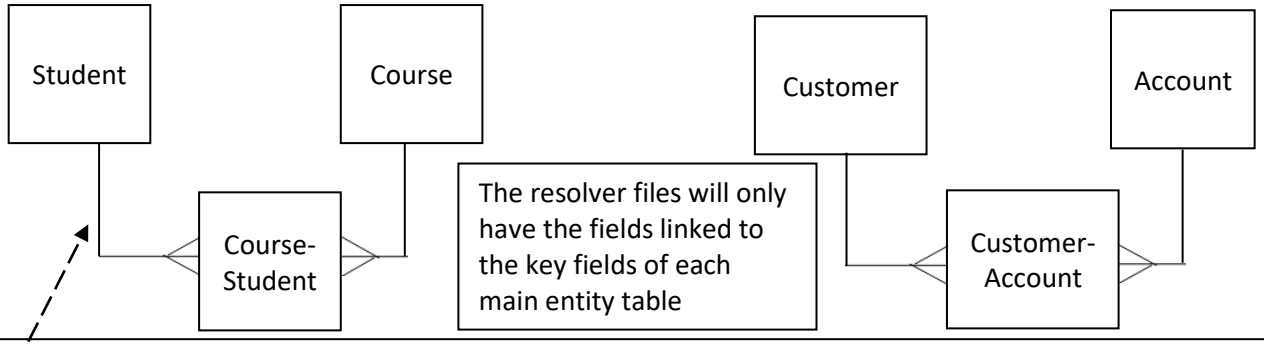


(ie one supplier can supply many different stock items/products; whereas one item of stock will be provided by one supplier. If an item of stock can be provided by more than one supplier then it would be shown by: 

Another many:many example would be a school's student entity/file and the courses that they provide entity/file (eg A level Computer Science course, A level Mathematics, GCSE English Literature etc). One student will/can be studying many (more than one) course; and one course will/can contain/have more than one student



We cannot program a many:many relationship, we need to add a third, intermediate <u>resolver</u> table/file:

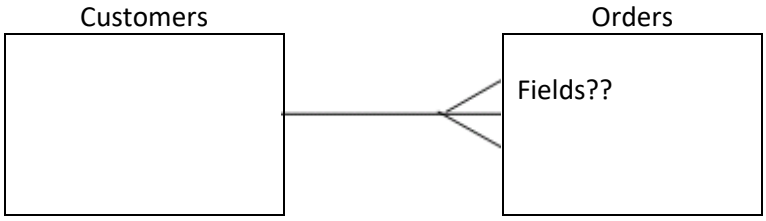


If 7 students each study 3 different courses, 5 students each study 4 different courses and 3 students each study 2 different courses, the resolver file will require 47 records (21 [7x3] + 20 [5x4] + 6 [3x3])

File examples of Amazon
Some files the company will need:
For selling: Customers:

Customer ID (unique) "Name" "Address" (for posting items) Payment "details" (to take money) Orders??

The problem is that one customer can make many orders (what is an order? – it is made up of (possibly) many items/products!!), so from Customer to Order it is 1:many; but one order will just be made by one customer so from order back to customer it is 1:1, we write this as:

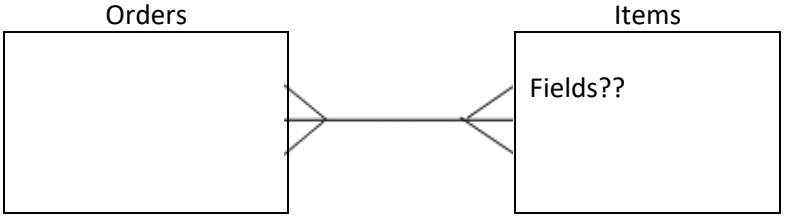


When we have a 1:many link between entities (in a database, an entity is anything about which information can be stored; for example, a person, concept, physical object or event) each entity becomes a file (a database table) linked by a foreign key (this is where the key field of one table/file is also stored in the other table/file to link them together). In this case the "many" table has a key field (eg an Order ID) and also has the key field from the other file (ie above each order record will also have the Customer ID who made the order). This means that if a customer, over time, has placed 37 orders, there will be an associated 37 records in the order file; and if another customer has made 14 orders, there will be another 14 associated records in the order file etc.

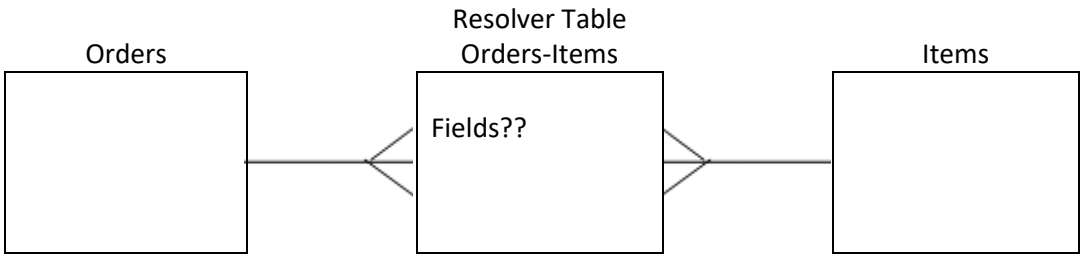
However, we also have to consider what fields we need for the order file:

Order ID (unique) Customer ID (need to know who made the order/where to send it) Date (to track order/chase any delays in despatch) Order completed (to keep "live" until completed) Products/Items in order??

Amazon will keep a File/Table of all the products they sell.
One order can be made up of many items, so from Order to Item it is 1:many. Also, an item can be sold to many people (not the same physical item obviously, but the details about an item (the record details) will be used for each customer who buys that item), so from Item to Order it also 1: many, which makes the link overall many:many, we write this as:



ie one order can be made up of many different items; and one item can be on many different orders.
We cannot program this link using the two files/tables as we can between Customers and Orders because it requires foreign fields in both files; so if we have a many:many link between tables, we need a third table, a resolver file/table:



The fields for the Item file/table would be the details needed about each item they stock, eg:

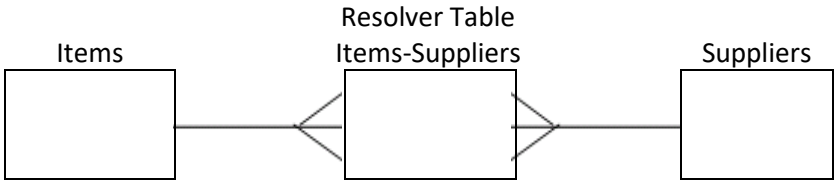
Item ID (unique) Description (to display on receipt) Price (to calculate pverall cost to customer) Number in stock (to identify when to order more) “Supplier” etc

The Resolver file/table Orders-Item would simply be the link between the two separate tables and store the key field of each record. So if one order was made up of four items, there would be four records in the resolver file associated with that order; and if an item had been bought on seven different orders, , there would be seven records in the resolver file associated with that product.

Another file required for this management system would be store details about suppliers. If one item is only supplied by one supplier, but one supplier may supply many different items then we have a 1:many link which is programmable. However, if one item can be supplied by more than one supplier (and one supplier can supply many different items) then we have a many:many relationship and we will need another resolver file:

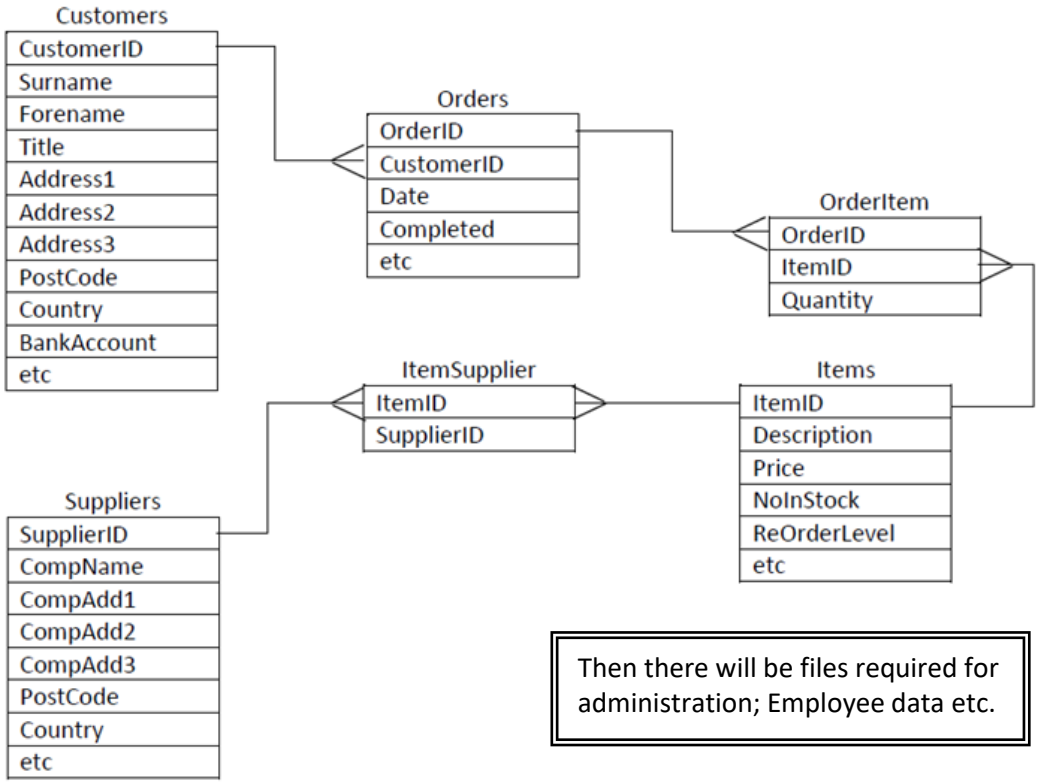


So:



The fields in the resolver file will again just be two; the Item ID and the associated Supplier ID. So if an item is available from three suppliers, then there will be three records in the resolver file associated with that item (if one item is only provided by one supplier, then there will only be one record in the resolver file associated with that item); if one supplier supplies 25 different items, then there will be 25 records in the resolver file associated with that supplier.

Entity-Relationship Diagram (ERD) of the system so far looks like:



Then there will be files required for administration; Employee data etc.

Some example data are:

Customers

CustomerID	CO1	CO2	CO3	CO4	CO5
Surname	Deftin	Hurma	Boldova	Yimnet	Memod
Forename	Filna	Lesta	Jawter	Aberst	Klean
Title	Miss	Mr	Mx	Mr	Mrs
Address1	23 High Street	17 Low Street	5A Mid Street	"Appledore"	Flat 17
Address2	Bradville	Loughton	Bradville	off Main Street	Gramwell
Address3	Milton Keynes	Milton Keynes	Milton Keynes	Woburn	Milton Keynes
PostCode	MK9 1TG	MK8 6KH	MK9 2YY	MK17 1DF	MK6 8DA
Country	UK	UK	UK	UK	UK
BankAccount	43221789	7664352	00342134	65388995	01020445

Orders

OrderID	OD1	OD2	OD3	OD4
CustomerID	CO1	CO4	CO1	CO2
Date	10/12/2020	10/12/2020	11/12/2020	11/12/2020
Completed	TRUE	FALSE	FALSE	TRUe

Items

ItemID	IT1	IT2	IT3	IT4	IT5	IT6
Description	Battery	Bulb	Ring	Ink	Pen	Scissors
Price	0.78	1.34	23.34	1.17	7.55	3.12
NoInStock	223	56	23	567	241	101
ReOrderLevel	100	40	15	300	200	100
SupplierID	SU4	SU4	SU2	SU1	SU1	SU1

OrderItem

OrderID	OD1	OD1	OD2	OD3	OD4	OD4	OD4	OD4	OD4
ItemID	IT1	IT2	IT1	IT5	IT1	IT2	IT4	IT5	IT6

Suppliers

SupplierID	SU1	SU2	SU3	SU4
CompName	Wilkin's	Beemson	Filey	Gorston's
CompAdd1	23 Givern	Unit 34	Killway	Lars Industrial Estate
CompAdd2		Trams Way	Stirling	Oft Road
CompAdd3	Kendal	Notts		Leicester
PostCode	LA8 9WX	NG1 1AU	FK7 0BE	LE7 1AE
Country	UK	UK	UK	UK

ItemSupplier

ItemID	IT1	IT1	IT1	IT2	IT3	IT4	IT4	IT5	IT6
SupplierID	SU4	SU2	SU1	SU4	SU3	SU4	SU2	SU2	SU4

To display all the orders, using data from the linked tables using the resolver tables:

```
RESET(OrderFile);
WHILE NOT EOF(OrderFile) DO
  BEGIN
    READ(OrderFile, Order);
    WRITELN(Order.OrderID);
    RESET(CustomerFile);
    REPEAT
      READ(CustomerFile,Customer);
    UNTIL Customer.CustomerID = Order.CustomerID);
    WITH Customer DO
      BEGIN
        WRITELN(CustomerID);
        WRITELN(Title, ' ',Forename, ' ',Surname);
        WRITELN(Address1); etc
      END;
    WHILE NOT EOF(OrderItemFile) DO
      BEGIN
        READ(OrderItemFile,OrderItem);
        IF OrderItem.OrderID = Order.OrderID THEN
          BEGIN
            RESET(ItemFile);
            REPEAT
              READ(ItemFile,Item);
            UNTIL Item.ItemID = OrderItem.ItemID;
            WRITELN(Item.Description);
            WRITELN(Item.Proce);
            WRITELN(OrderItem.Quantity);
            RESET(ItemSupplierFile);
            REPEAT
              READ(ItemSupplierFile,ItemSupplier);
            UNTIL ItemSupplier.ItemID = Item.ItemID;
            RESET(SupplierFile);
            REPEAT
              READ(SupplierFile,Supplier);
            UNTIL Supplier.SupplierID = ItemSupplier.SupplierID;
            WRITELN('Supplied by: ',Supplier.CompName);
            WRITELN(Supplier.CompAdd1); etc
          END;
      END;
  END;
```

Use each order record in turn linearly

Find the (unique) customer for each order

Display the customer's details

Find each item associated with the order (there may be many)

For each item ID found, find the (unique) item and display its details

For each item, find the (unique) supplier and display its name

Output too long for screen display.

If the output, eg a list of all products and related details, is too long for the output window then the earlier displays will be “lost” when scrolling back up the display window. In this case we can send the output to a text file on backing store which can then be opened using Word or Notepad and printed.

Declare a text file within the TYPE and VAR sections, eg:

```
TYPE
  :
  :
  ScreenDumpFileType = TEXT;

VAR
  :
  :
  ScreenDumpFile : ScreenDumpFileType;
```

In the procedure that initialises the other files (using the ASSIGN and RESET statements), you will need:

```
  ASSIGN(ScreenDumpFile, SelectedDrive + Directory+'ScreenDump.txt');
  REWRITE(ScreenDumpFile);
```

Then use a routine to WRITE/WRITELN to the text file. This is the same as a normal WRITE/WRITELN to the monitor, except that all data must be in string format. Therefore, if some data is another format (eg INTEGER/LONGINT, REAL, TDATETIME etc) you need to convert to STRING; eg:

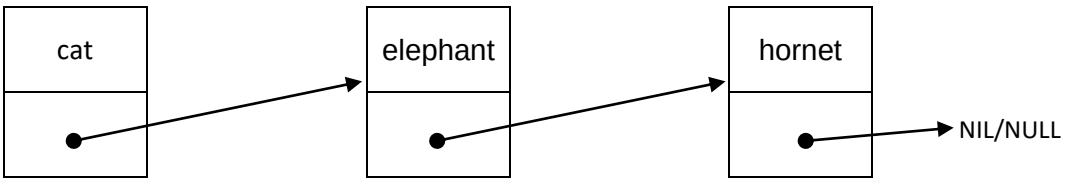
```
WITH Order DO
  BEGIN
    WRITELN(ScreenDumpFile,'Order ID: ':30,OrderId,' ');
    WRITELN(ScreenDumpFile,'Customer ID: ':30,CustomerId,' ');
    WRITELN(ScreenDumpFile,'Date: ':30, DATETOSTR(Date),' ');
    IF Completed THEN
       WRITELN(ScreenDumpFile,'Order completed':30,' ')
    ELSE
       WRITELN(ScreenDumpFile,'Order not completed':30,' ')
  END;
```

You simply add the filename at the start of the WRITE/WRITELN statement.

Pointers

Pointers are a data type available within Pascal. They are usually used as a field of a record to create a dynamic store, unlike arrays which are fixed in length (ie a static data structure).

For example: a record containing string data and a pointer field to the next record



Assuming a record type of:

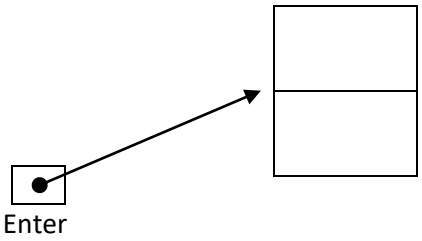
```
TYPE
   DataRec = RECORD
                Data : STRING[20];
                NextRec : Pointer;
             END;
```

Pointer is not a reserved word, it has to be declared in `TYPE` before the record definition: eg `Pointer = ^DataRec;` (in text it is written …↑`DataRec`)

Within the `VAR` section, pointer variables can be declared, eg: `Enter : Pointer;`

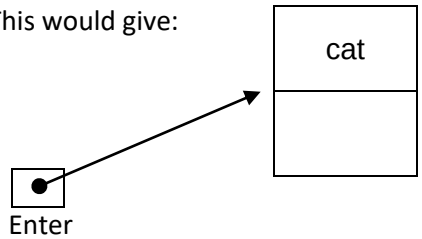
This has not yet created a record in which data can be entered/stored. To do this the Pascal command `NEW` is used, eg: `NEW(Enter);`

This now gives:



Data can be stored by: `Enter^.data := 'cat'` or `READLN(Enter^.data);`

This would give:



To continue the link we would need to create a new record. However `NEW(Enter);` would not work – why?

(It would <u>not</u> overwrite "cat", but there would be no pointer to "cat", so it would not be retrievable.)

```
PROGRAM Pointers;
   USES
      CRT, SYSUTILS;

   TYPE
      NextPerson = ^PersonRec;

      PersonRec = RECORD
                     Surname:STRING[30];
                     Forename:STRING[25];
                     Salary:REAL;
                     Next:NextPerson;
                  END;
   VAR
      Start, Current, Trace : Nextperson;

   PROCEDURE Test;
      BEGIN
         NEW(Start);
         Start^.Surname :='Jenkins';
         Start^.Forename:='Birs';
         Start^.Salary  := 43212.77;
         Start^.Next    :=NIL;
         Current := Start;

         NEW(Start);
         Start^.Surname :='Moing';
         Start^.Forename:='Mqist';
         Start^.Salary  := 16999.12;
         Start^.Next    :=Current;

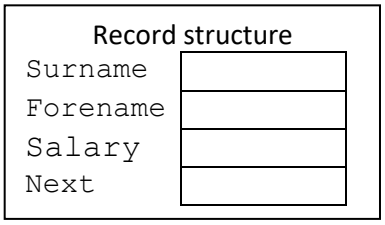
         Current := Start;

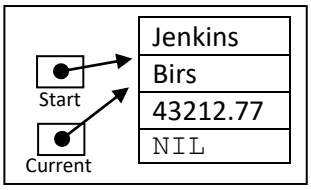
         NEW(Start);
         Start^.Surname :='Jiglert';
         Start^.Forename:='Grrst';
         Start^.Salary  := 36794.64;
         Start^.Next    :=Current;
      END;

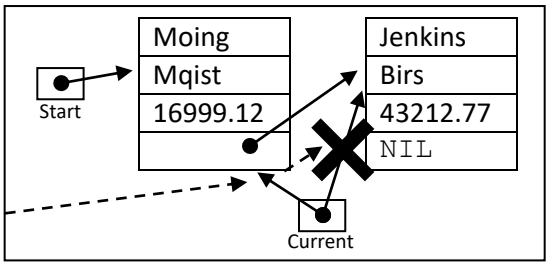
   PROCEDURE DisplayList;
      BEGIN
         Trace:=Start;
         WHILE Trace <> NIL DO
            BEGIN
               WRITELN(Trace^.Forename,' ',Trace^.Surname,': £',
                       Trace^.Salary:4:2);
               Trace:=Trace^.Next;
            END;
      END;

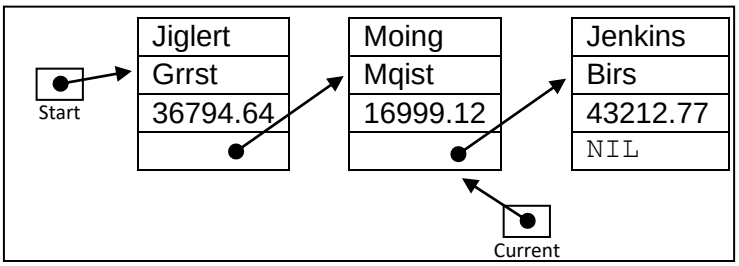
BEGIN
   Test;
   DisplayList;

   READKEY;
END.
```



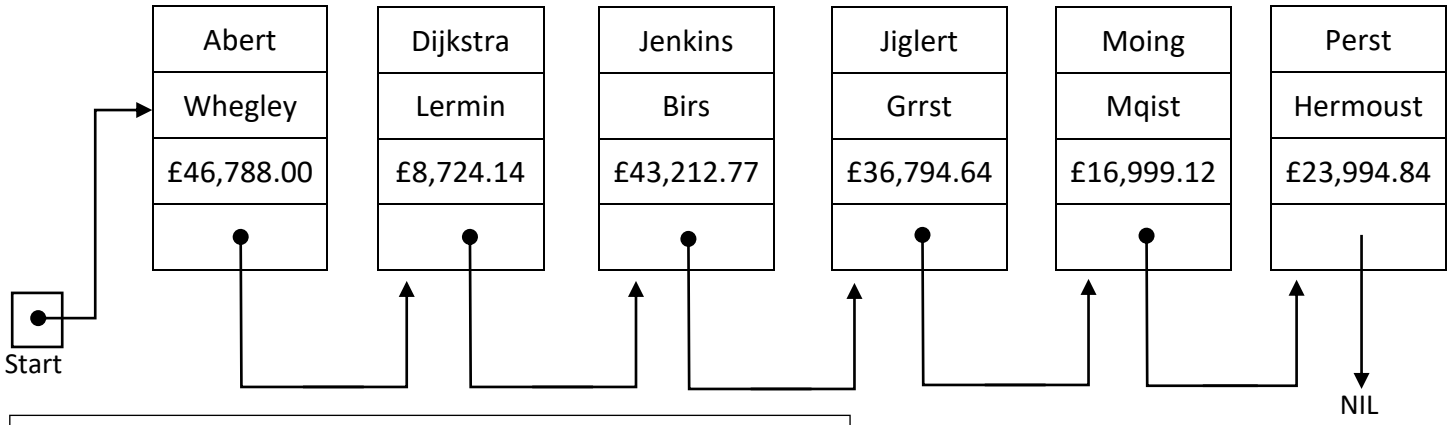




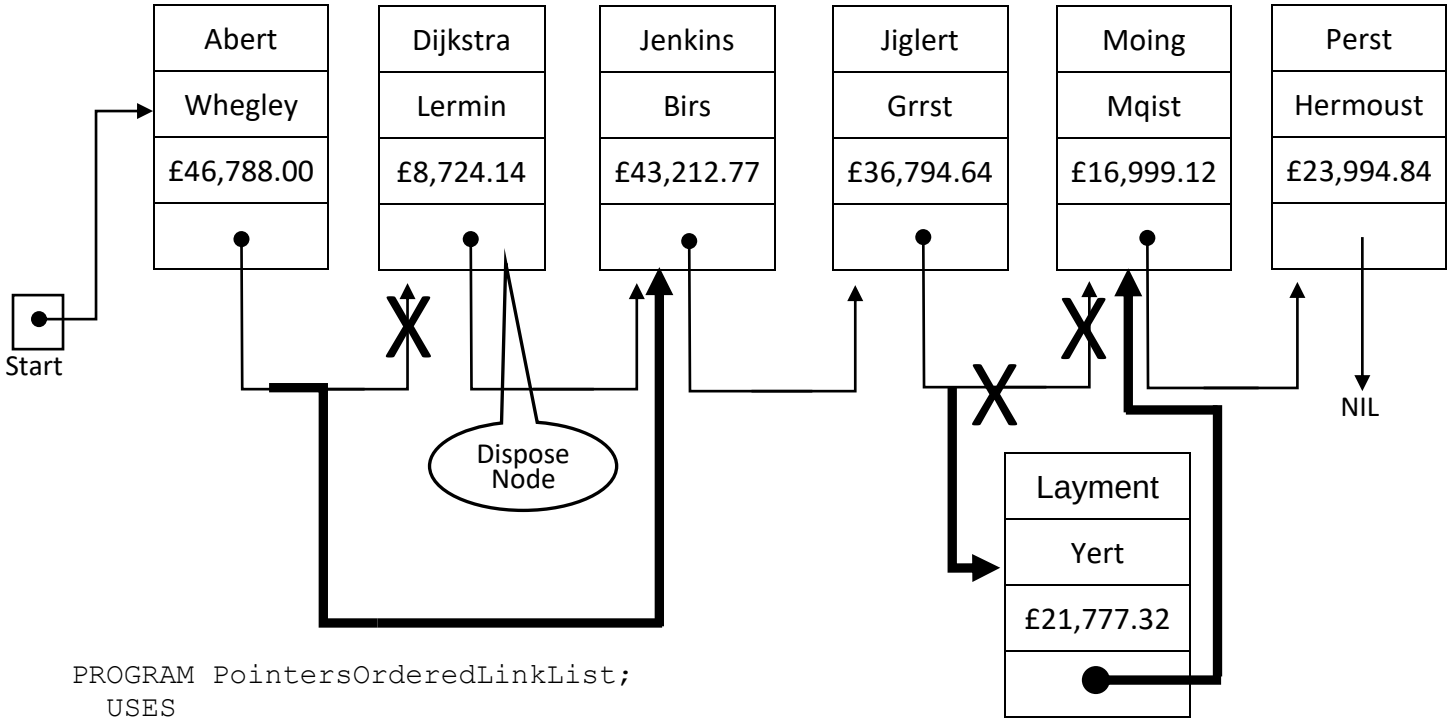



```
Grrst Jiglert: £36794.64
Mqist Moing: £16999.12
Birs Jenkins: £43212.77
```

Maintaining an ordered linked list using pointers



Adding: Layment, Yert, £21,777.32. Deleting: Dijkstra




```
PROGRAM PointersOrderedLinkList;
  USES
    CRT, SYSUTILS;

  TYPE
    NextPerson = ^PersonRec;

    PersonRec = RECORD
                  Surname:STRING[30];
                  Forename:STRING[25];
                  Salary:REAL;
                  Next:NextPerson;
                END;
  VAR
    Start, Prev, Current, NewNode : Nextperson;
```

```
PROCEDURE CreateList;
  BEGIN
    NEW(NewNode);
    NewNode^.Surname :='Abert';
    NewNode^.Forename:='Whegley';
    NewNode^.Salary  := 46788.00;
    NewNode^.Next    := NIL;

    Start:= NewNode;
    Current := NewNode;

    NEW(NewNode);
    NewNode^.Surname :='Dijkstra';
    NewNode^.Forename:='Lermin';
    NewNode^.Salary  := 8724.14;
    NewNode^.Next    := NIL;

    Current^.Next := NewNode;
    Current := Current^.Next;
       //or Current:=NewNode);}

    NEW(NewNode);
    NewNode^.Surname :='Jenkins';
    NewNode^.Forename:='Birs';
    NewNode^.Salary  := 43212.77;
    NewNode^.Next    := NIL;

    Current^.Next := NewNode;
    Current := Current^.Next;
      //or Current:=NewNode);

    NEW(NewNode);
    NewNode^.Surname :='Jiglert';
    NewNode^.Forename:='Grrst';
    NewNode^.Salary  := 36794.64;
    NewNode^.Next    := NIL;

    Current^.Next := NewNode;
    Current := Current^.Next;

    NEW(NewNode);
    NewNode^.Surname :='Moing';
    NewNode^.Forename:='Mqist';
    NewNode^.Salary  := 16999.12;
    NewNode^.Next    := NIL;

    Current^.Next := NewNode;
    Current := Current^.Next;

    NEW(NewNode);
    NewNode^.Surname :='Perst';
    NewNode^.Forename:='Hermoust';
    NewNode^.Salary  := 23944.84;
    NewNode^.Next    := NIL;

    Current^.Next := NewNode;
    Current := Current^.Next;
  END;
PROCEDURE AddPerson;
  BEGIN
```

OR with user input for data:

```
PROCEDURE CreateList;
  VAR
    Another : CHAR;
    BEGIN

    {Get first node}
      NEW(NewNode);
      WRITE('Surname: ');
      READLN(NewNode^.Surname);
      WRITE('Forename: ');
      READLN(NewNode^.Forename);
      WRITE('Salary: ');
      READLN(NewNode^.Salary);
      NewNode^.Next:= NIL;

      Start:= NewNode;
      Current := NewNode;

      {Get other nodes}
      WRITE('Add another node Y/N?: ');
      Another := UPCASE(READKEY);
      WRITELN(Another);


      WHILE Another = 'Y' DO
        BEGIN
          NEW(NewNode);
          WRITE('Surname: ');
          READLN(NewNode^.Surname);
          WRITE('Forename: ');
          READLN(NewNode^.Forename);
          WRITE('Salary: ');
          READLN(NewNode^.Salary);
          NewNode^.Next:= NIL;

          Current^.Next := NewNode;
          Current := Current^.Next;

          WRITE('Add another node Y/N?: ');
          Another := UPCASE(READKEY);
          WRITELN(Another);
        END;
    END;
```

```
NEW(NewNode);
WITH NewNode^ DO
  BEGIN
    WRITE('Enter new Forename: ');
    READLN(Forename);
    WRITE('Enter new Surname: ');
    READLN(Surname);
    WRITE('Enter new salary: ');
    READLN(Salary);
    Next := NIL;
  END;

  IF Newnode^.Surname <Start^.Surname THEN  {first node in list}
    BEGIN
      NewNode^.Next:=Start;
      Start:=NewNode;
    END
  ELSE  {follow pointers to find location in list}
    BEGIN
      Prev:=Start;
      Current:=Start^.Next;

      WHILE (Current<>NIL) AND (Current^.Surname<NewNode^.Surname) DO
        BEGIN
          Current:=Current^.Next;{or: Prev:=Current;}
          Prev:=Prev^.Next;{ Current:=Current^.Next }
        END;

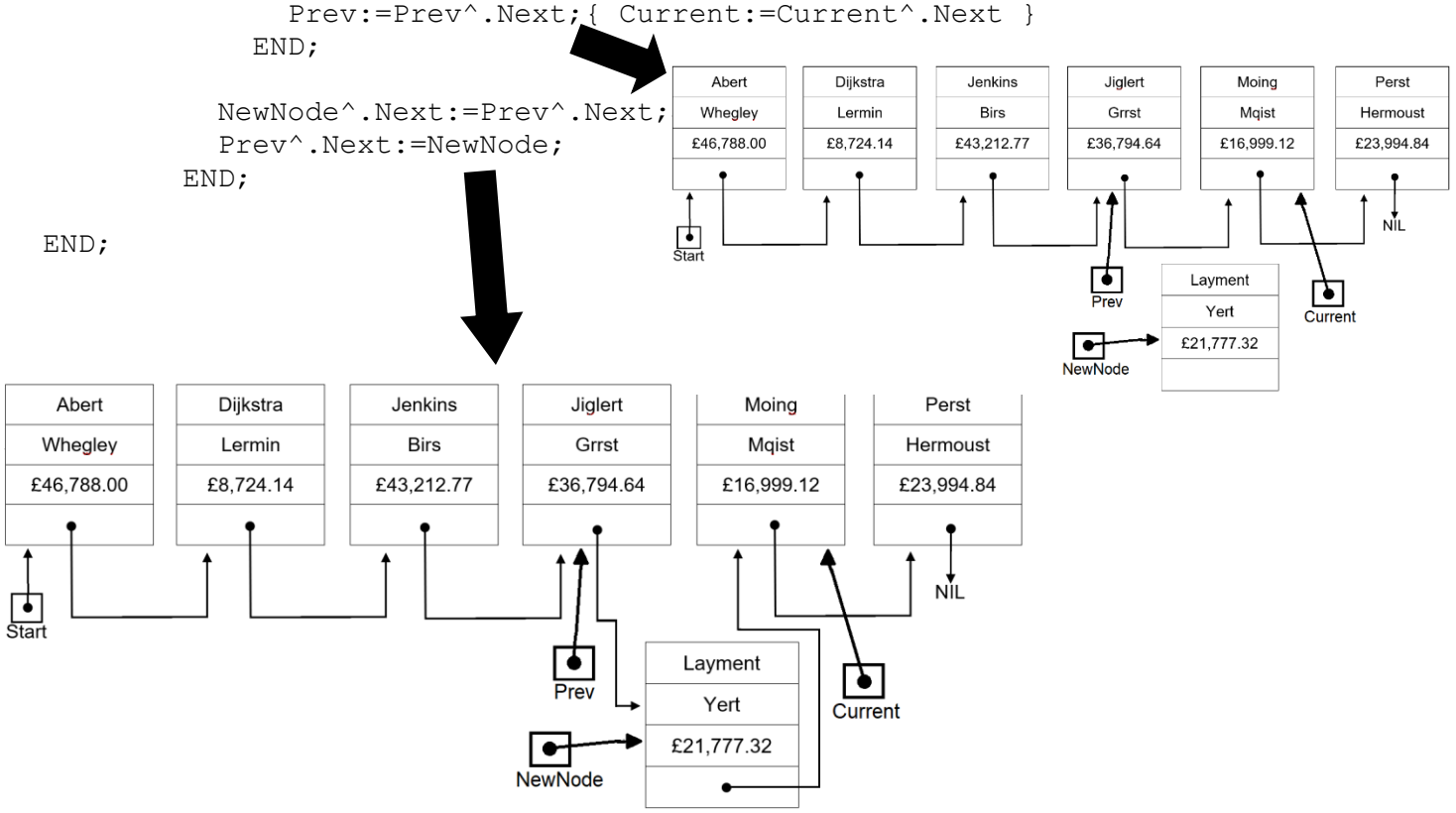
      NewNode^.Next:=Prev^.Next;
      Prev^.Next:=NewNode;
    END;

END;
```

Get data from user and store in memory pointed to by `NewNode` (a newly created location)

If alphabetically first name, add at start of list. New node points to current `Start`, the reset `Start` to be new node.

Keep two pointers updated `Prev` and `Current` so when location is found, `Current` points to node after one to add, and `Prev` is the node before it.



```
PROCEDURE DeletePerson;
  VAR
    DeleteSurname : STRING;
  BEGIN
    WRITE('Enter Surname of person to remove from list: ');
    READLN(DeleteSurname);
    DeleteSurname:=UPPERCASE(DeleteSurname);

    Current:=Start;
    {is it first name?}
    IF UPPERCASE(Current^.Surname)=DeleteSurname THEN
      BEGIN
        Start := Start^.Next;
        DISPOSE(Current);                  //Why not DISPOSE(Start);?

        WRITELN('Name removed from list');
        WRITELN('Press any key to continue');READKEY;
      END
    ELSE
      BEGIN {follow pointers to locate node}
        WHILE (Current<>NIL) AND (UPPERCASE(Current^.Surname)<DeleteSurname) DO
          BEGIN
            Prev:=Current;
            Current:=Current^.Next;
          END;

        IF UPPERCASE(Current^.Surname) = DeleteSurname THEN
          BEGIN{Readjust pointers}
            Prev^.Next := Current^.Next;
            DISPOSE(Current);
            WRITELN('Name removed from list');
          END
        ELSE
          BEGIN
            WRITELN('Entered name is not in the list');
          END;

        WRITELN('Press any key to continue');READKEY;
      END;
  END;

PROCEDURE DisplayList;
  BEGIN
    Current:=Start;

    WRITELN;
    WHILE Current <> NIL DO
      BEGIN
        WRITELN(Current^.Surname,'':12-LENGTH(Current^.Surname),
                Current^.Forename,'':15-LENGTH(Current^.Forename),
                ': £',Current^.Salary:4:2);
        Current:=Current^.Next;
      END;
    WRITELN;
  END;
```

`Current` points to node to delete,
`Prev` is the node before it.

Previous node points “around” the node to be deleted.

```
PROCEDURE MainMenu;
  VAR
    MenuResponse : CHAR;

  BEGIN
    REPEAT
      WRITELN('Using a Linked List in sorted order');
      WRITELN;
      WRITELN('1: Display List');
      WRITELN('2: Add a new person');
      WRITELN('3: Delete a person');
      WRITELN;
      WRITELN('X: eXit program');
      WRITELN; WRITELN;

      WRITE('Enter menu option: ');
      MenuResponse := UPCASE(READKEY);WRITELN(MenuResponse);

      CASE MenuResponse OF
        '1' : DisplayList;
        '2' : AddPerson;
        '3' : DeletePerson;
        'X' : {Do Nothing};
      ELSE
        BEGIN
            WRITELN('Error, enter 1, 2, 3 or X');
            WRITELN('Please re-enter');
        END;
      END;

    UNTIL MenuResponse = 'X';
  END;

BEGIN
  CreateList;
  DisplayList;

  MainMenu;
END.
```

OCR Programming Project

The project is a substantial piece of work which assesses a variety of different skills including the development and demonstration of computational thought processes.

Students will need to choose a well defined user-driven problem. The choice of project must enable students to:

- meet all assessment requirements as contained in the specification
- use an appropriate programming language which is non-trivial and has a substantial coded element
- solve a problem sensibly within the constraints of resources available
- facilitate the successful completion of a whole task from its definition to its acceptance and evaluation by that user
- involve all elements of the skills of analysis, design, development and evaluation.

Evidence Generation: it is important that students establish thorough and robust working practices at an early stage. The projects should contain as standard the following:

- Title
- Contents list
- Description of investigation
- Justification of investigation
- Analysis, design and methods used
- Evaluation
- Bibliography
- Pages numbered

Appropriate annotated evidence should be used to support the report – eg screen dumps or photographs taken of screen layouts. It is important that students generate their own individual, authentic evidence to show that they have met key assessment criteria. Any evidence submitted to demonstrate the development of solution must be able to be assessed without the use of any specific hardware or software.

Students will be expected to analyse, design, develop, test, evaluate and document a program written in a suitable programming language. The underlying approach to the project is to apply the principles of computational thinking to a practical coding problem. Students are expected to apply appropriate principles from an agile development approach to the project development.

While the project assessment criteria are organised into specific categories, it is anticipated the final report will document the agile development process, and elements for each of the assessment categories will appear throughout the report.

The programming project will be submitted in the form of a report that will contain the solution to a problem, selected by the student and school, written in a suitable programming language.

3.1. Analysis of the problem (10 marks)

3.1.1 Problem identification

- Describe and justify the features that make the problem solvable by computational methods
- Explain why the problem is amenable to a computational approach.

3.1.2 Stakeholders

- Identify and describe those who will have an interest in the solution explaining how the solution is appropriate to their needs (this may be named individuals, groups or persona that describes the target end user).

3.1.3 Research the problem

- Research the problem and solutions to similar problems to identify and justify suitable approaches to a solution.
- Describe the essential features of a computational solution explaining these choices.
- Explain the limitations of the proposed solution.

3.1.4 Specify the proposed solution

- Specify and justify the solution requirements including hardware and software configuration (if appropriate).
- Identify and justify measurable success criteria for the proposed solution.

3.2 Design of the solution (15 marks)

3.2.1 Decompose the problem

- Break down the problem into smaller parts suitable for computational solutions justifying any decisions made.

3.2.2 Describe the solution

- Explain and justify the structure of the solution.
- Describe the parts of the solution using algorithms justifying how these algorithms form a complete solution to the problem.
- Describe usability features to be included in the solution.
- Identify key variables / data structures / classes justifying choices and any necessary validation.

3.2.3 Describe the approach to testing

- Identify the test data to be used during the iterative development and post development phases and justify the choice of this test data.

3.3 Developing the solution (25 marks)

3.3.1 Iterative development process

- Provide annotated evidence of each stage of the iterative development process justifying any decision made.
- Provide annotated evidence of prototype solutions justifying any decision made.

3.3.2 Testing to inform development

- Provide annotated evidence for testing at each stage justifying the reason for the test.
- Provide annotated evidence of any remedial actions taken justifying the decision made.

3.4 Evaluation (20 marks)

3.4.1 Testing to inform evaluation

- Provide annotated evidence of testing the solution of robustness at the end of the development process.
- Provide annotated evidence of usability testing (user feedback).

3.4.2 Success of the solution

- Use the test evidence from the development and post development process to evaluate the solution against the success criteria from the analysis.

3.4.3 Describe the final product

- Provide annotated evidence of the usability features from the design, commenting on their effectiveness.

3.4.4 Maintenance and development

- Discuss the maintainability of the solution.
- Discuss potential further development of the solution.

3.1. Analysis of the problem (10 marks)

3.1.1 Problem identification

- Describe and justify the features that make the problem solvable by computational methods
- Explain why the problem is amenable to a computational approach.

3.1.2 Stakeholders

- Identify and describe those who will have an interest in the solution explaining how the solution is appropriate to their needs (this may be named individuals, groups or persona that describes the target end user).

3.1.3 Research the problem

- Research the problem and solutions to similar problems to identify and justify suitable approaches to a solution.
- Describe the essential features of a computational solution explaining these choices.
- Explain the limitations of the proposed solution.

3.1.4 Specify the proposed solution

- Specify and justify the solution requirements including hardware and software configuration (if appropriate).
- Identify and justify measurable success criteria for the proposed solution.

Analysis (maximum 10 marks)			
1–2 marks	3–5 marks	6–8 marks	9–10 marks
The candidate will have:			
Identified some features that make the problem solvable by computational methods.	Described the features that make the problem solvable by computational methods.	Described the features that make the problem solvable by computational methods and why it is amenable to a computational approach.	Described and justified the features that make the problem solvable by computational methods, explaining why it is amenable to a computational approach.
Identified suitable stakeholders for the project and described them and some of their requirements.	Identified suitable stakeholders for the project and described how they will make use of the proposed solution.	Identified suitable stakeholders for the project and described them and how they will make use of the proposed solution and why it is appropriate to their needs.	Identified suitable stakeholders for the project and described them explaining how they will make use of the proposed solution and why it is appropriate to their needs.
Identified some appropriate features to incorporate into their solution.	Researched the problem looking at existing solutions to similar problems identifying some appropriate features to incorporate into their solution.	Researched the problem in depth looking at existing solutions to similar problems identifying and describing suitable approaches based on this research.	Researched the problem in depth looking at existing solutions to similar problems, identifying and justifying suitable approaches based on this research.
Identified some features of the proposed computational solution.	Identified the essential features of the proposed computational solution.	Identified and described the essential features of the proposed computational solution.	Identified and described the essential features of the proposed computational solution explaining these choices.
Identified some limitations of the proposed solution.	Identified and described some limitations of the proposed solution.	Identified and explained any limitations of the proposed solution.	Identified and explained with justification any limitations of the proposed solution.
Identified some requirements for the solution.	Identified most requirements for the solution.	Specified the requirements for the solution including (as appropriate) any hardware and software requirements.	Specified and justified the requirements for the solution including (as appropriate) any hardware and software requirements.
Identified some success criteria for the proposed solution.	Identified some measurable success criteria for the proposed solution.	Identified all measurable success criteria for the proposed solution.	Identified and justified all measurable success criteria for the proposed solution.

3.2 Design of the solution (15 marks)

3.2.1 Decompose the problem

- Break down the problem into smaller parts suitable for computational solutions justifying any decisions made.

3.2.2 Describe the solution

- Explain and justify the structure of the solution.
- Describe the parts of the solution using algorithms justifying how these algorithms form a complete solution to the problem.
- Describe usability features to be included in the solution.
- Identify key variables / data structures / classes justifying choices and any necessary validation.

3.2.3 Describe the approach to testing

- Identify the test data to be used during the iterative development and post development phases and justify the choice of this test data.

<table>
<tr><td colspan="4">Design (maximum 15 marks)</td></tr>
<tr><td>1–4 marks</td><td>5–8 marks</td><td>9–12 marks</td><td>13–15 marks</td></tr>
<tr><td colspan="4">The candidate will have:</td></tr>
<tr><td></td><td>Broken the problem down systematically into a series of smaller problems suitable for computational solutions describing the process.</td><td>Broken the problem down systematically into a series of smaller problems suitable for computational solutions explaining the process.</td><td>Broken the problem down systematically into a series of smaller problems suitable for computational solutions, explaining and justifying the process.</td></tr>
<tr><td></td><td>Defined the structure of the solution to be developed.</td><td colspan="2">Defined in detail the structure of the solution to be developed.</td></tr>
<tr><td>Described elements of the solution using algorithms.</td><td>Described the solution fully using appropriate and accurate algorithms.</td><td>Described the solution fully using appropriate and accurate algorithms explaining how these algorithms form a complete solution to the problem.</td><td>Described the solution fully using appropriate and accurate algorithms justifying how these algorithms form a complete solution to the problem.</td></tr>
<tr><td>Described some usability features to be included in the solution.</td><td>Described the usability features to be included in the solution.</td><td>Described, explaining choices made, the usability features to be included in the solution.</td><td>Described, justifying choices made, the usability features to be included in the solution.</td></tr>
<tr><td>Identified the key variables / data structures / classes (as appropriate to the proposed solution).</td><td>Identified the key variables / data structures / classes (as appropriate to the proposed solution) and any necessary validation.</td><td>Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) explaining any necessary validation.</td><td>Identified and justified the key variables / data structures / classes (as appropriate to the proposed solution) justifying and explaining any necessary validation.</td></tr>
<tr><td rowspan="2">Identified some test data to be used during the iterative or post development phase of the process.</td><td>Identified the test data to be used during the iterative development of the solution.</td><td colspan="2">Identified and justified the test data to be used during the iterative development of the solution.</td></tr>
<tr><td>Identified any further data to be used in the post development phase.</td><td colspan="2">Identified and justified any further data to be used in the post development phase.</td></tr>
</table>

3.3 Developing the solution (25 marks)

3.3.1 Iterative development process

- Provide annotated evidence of each stage of the iterative development process justifying any decision made.
- Provide annotated evidence of prototype solutions justifying any decision made.

3.3.2 Testing to inform development

- Provide annotated evidence for testing at each stage justifying the reason for the test.
- Provide annotated evidence of any remedial actions taken justifying the decision made.

Iterative development of a coded solution (maximum 15 marks)			
1–4 marks	5–8 marks	9–12 marks	13–15 marks
The candidate will have:			
Provided evidence of some iterative development for a coded solution.	Provided evidence for most stages of the iterative development process for a coded solution describing what they did at each stage.	Provided evidence of each stage of the iterative development process for a coded solution relating this to the break down of the problem from the analysis stage and explaining what they did at each stage.	Provided evidence of each stage of the iterative development process for a coded solution relating this to the break down of the problem from the analysis stage and explaining what they did and justifying why.
Solution may be linear.		Provided evidence of some prototype versions of their solution.	Provided evidence of prototype versions of their solution for each stage of the process.
Code may be inefficient.	Solution will have some structure.	The solution will be modular in nature.	The solution will be well structured and modular in nature.
Code may not be annotated appropriately.	Code will be briefly annotated to explain key components.	Code will be well annotated to explain all key components.	Code will be well annotated to aid future maintenance of the system.
Variable names may be inappropriate.	Some variable and/or structure names will be largely appropriate.	Most variables and structures will be appropriately named.	All variables and structures will be appropriately named.
There will be little or no evidence of validation.	There will be evidence of some basic validation.	There will be evidence of validation for most key elements of the solution.	There will be evidence of validation for all key elements of the solution.
There will be little evidence of review during the development.	There will be evidence that the development was reviewed at some stage during the process.	The development will show review at most key stages in the process.	The development will show review at all key stages in the process.

Testing to inform development (maximum 10 marks)			
1–2 marks	3–5 marks	6–8 marks	9–10 marks
The candidate will have:			
Provided some evidence of testing during the iterative development process.	Provided some evidence of testing during the iterative development process.	Provided evidence of testing at most stages of the iterative development process.	Provided evidence of testing at each stage of the iterative development process.
	Provided evidence of some failed tests and the remedial actions taken.	Provided evidence of some failed tests and the remedial actions taken with some explanation of the actions taken.	Provided evidence of any failed tests and the remedial actions taken with full justification for any actions taken.

3.4 Evaluation (20 marks)

3.4.1 Testing to inform evaluation

- Provide annotated evidence of testing the solution of robustness at the end of the development process.
- Provide annotated evidence of usability testing (user feedback).

3.4.2 Success of the solution

- Use the test evidence from the development and post development process to evaluate the solution against the success criteria from the analysis.

3.4.3 Describe the final product

- Provide annotated evidence of the usability features from the design, commenting on their effectiveness.

3.4.4 Maintenance and development

- Discuss the maintainability of the solution.
- Discuss potential further development of the solution.

Testing to inform evaluation (maximum 5 marks)			
1 mark	2 marks	3–4 marks	5 marks
The candidate will have:			
Provided evidence of some post development testing.	Provided evidence of final product testing for function.	Provided annotated evidence of post development testing for function.	Provided annotated evidence of post development testing for function and robustness.
		Provided annotated evidence for usability testing.	

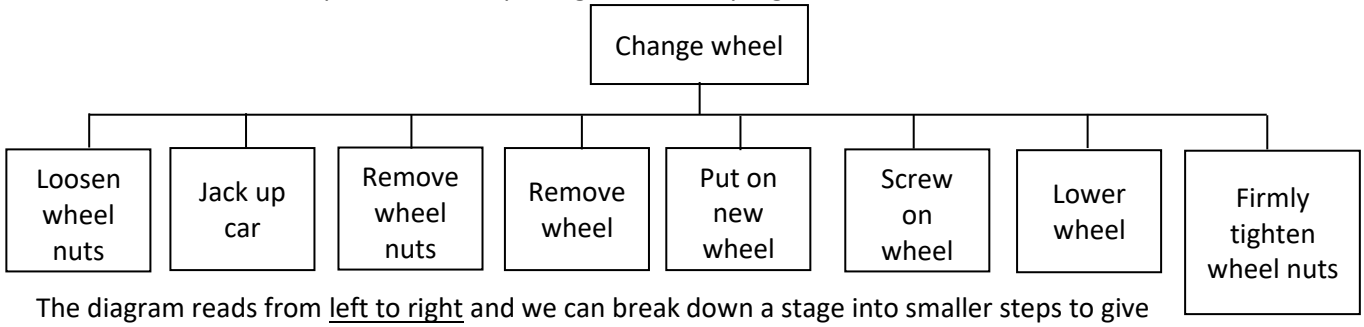
Evaluation of solution (maximum 15 marks)			
1–4 marks	5–8 marks	9–12 marks	13–15 marks
The candidate will have:			
Commented on the success or failure of the solution with some reference to test data.	Cross referenced some of the test evidence with the success criteria and commented on the success or otherwise of the solution.	Used the test evidence to cross reference with the success criteria to evaluate the solution identifying whether the criteria have been met, partially met or unmet.	Used the test evidence to cross reference with the success criteria to evaluate the solution explaining how the evidence shows that the criteria has been fully, partially or not met in each case.
		Provided comments on how any partially or not met criteria could be addressed in further development.	
	Provided evidence of usability features.		Provided evidence of the usability features justifying their success, partial success or failure as effective usability features.
The information is basic and communicated in an unstructured way. The information is supported by limited evidence and the relationship to the evidence may not be clear.			Provided comments on how any issues with partially or unmet usability features could be addressed in further development.
	Identified some limitations of the solution.	Considered maintenance issues and limitations of the solution.	
			Described how the program could be developed to deal with limitations of potential improvements / changes.
	The information has some relevance and is presented with limited structure. The information is supported by limited evidence.	There is a line of reasoning presented with some structure. The information presented is in the most part relevant and supported by some evidence.	There is a well developed line of reasoning which is clear and logically structured. The information presented is relevant and substantiated.

PLANNING WITH STRUCTURE DIAGRAMS AND PSEUDOCODE

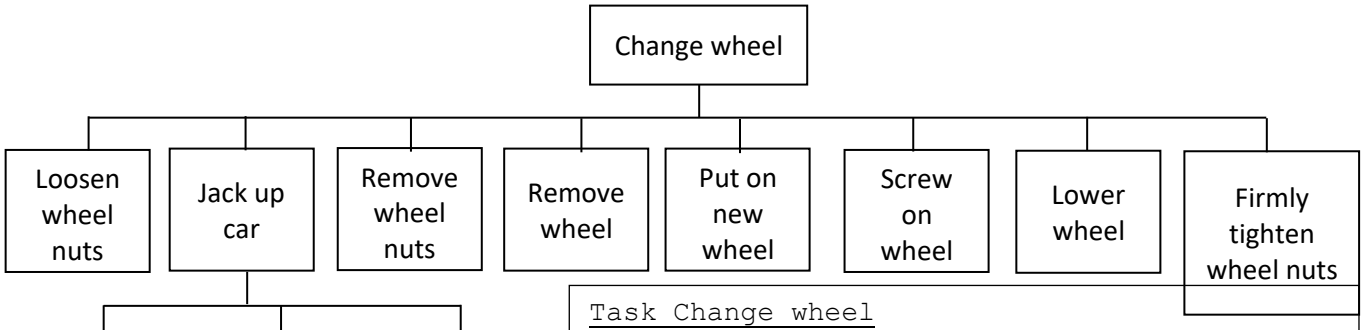
Computer programming is mostly concerned with problem solving. As with most problem-solving activities an organised approach is essential, ie we need to plan how to approach the problem before we attempt to solve it. An example would be the problem of changing a car wheel. If one was to start without planning, it is possible to get part way through and then find out that a necessary part is missing or a task has been done out of sequence thus wasting time (eg jacking up the car before loosening wheel nuts) etc. One way to specify the steps is simply by listing them as "pseudocode", eg:

```
Task Change wheel
      Loosen wheel nuts
      Jack up car
      Remove wheel nuts
      Remove wheel
      Put on new wheel
      Screw on wheel nuts
      Lower car
      Firmly tighten wheel nuts
End task
```

Another method is to represent the steps diagrammatically, eg:

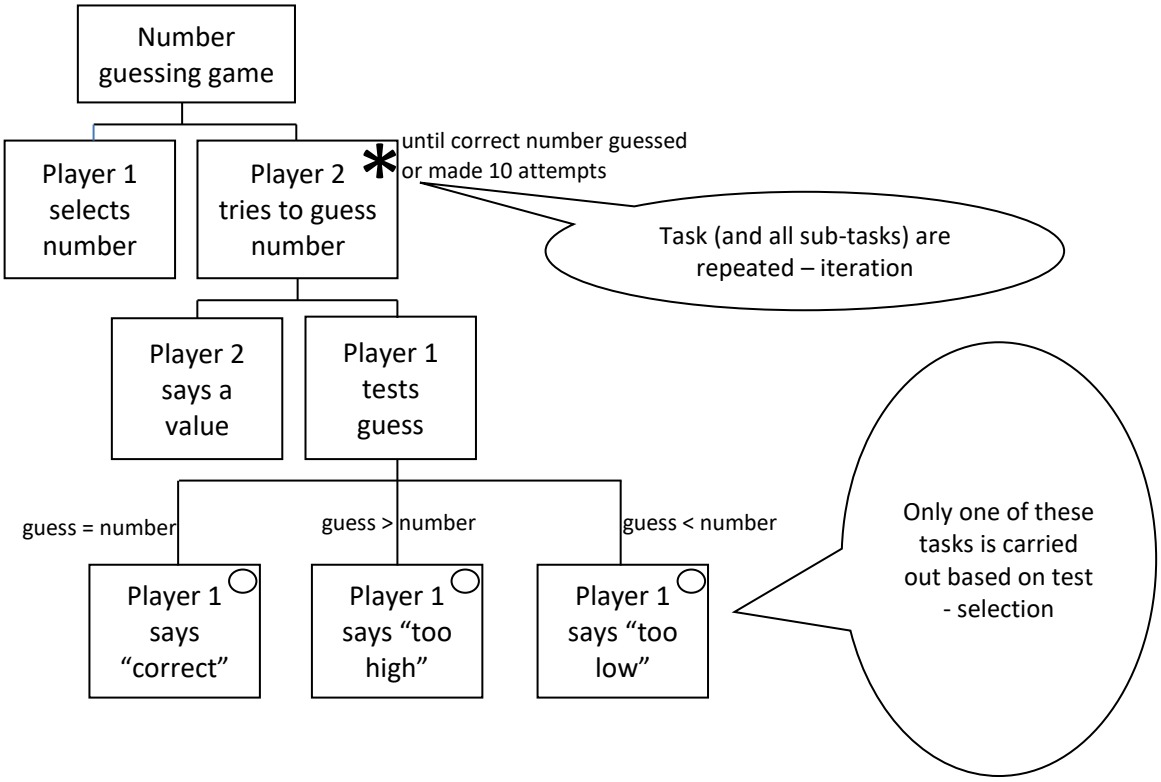


The diagram reads from left to right and we can break down a stage into smaller steps to give more detail. With this method of program design we try to solve the problem by starting at the top level and working down to lower levels, each of which is more detailed than the last. This is known as top down design and is probably the most widely used design strategy in computer programming. The point at which a task is not further refined is when it is completely understandable to the problem solver and/or others who might need to use it. For example "jack up car" might not be clear enough to another person and could be refined to:




```
Task Change wheel
      Loosen wheel nuts
      Jack up car
            locate specific point on car
            attach jack
            turn handle to raise car
      Remove wheel nuts
      Remove wheel
      Put on new wheel
      Screw on wheel nuts
      Lower car
      Firmly tighten wheel nuts
End task
```

This method of breaking down an activity into smaller and smaller steps is known as stepwise refinement or functional decomposition and is a very useful method of problem solving and therefore of computer program design. The stages above all show sequence, ie what task has to be carried out before another. The other two constructs are selection ie a task is only carried out as a result of a test which is either TRUE or FALSE, and iteration ie a task may be carried out/repeated more than once. For example the structure diagram and pseudocode for the game where one player chooses a number and the other player has up to ten attempts to guess the number; on each guess player 1 states whether it is higher or lower than the number until it guessed correctly or there are ten failed attempts.

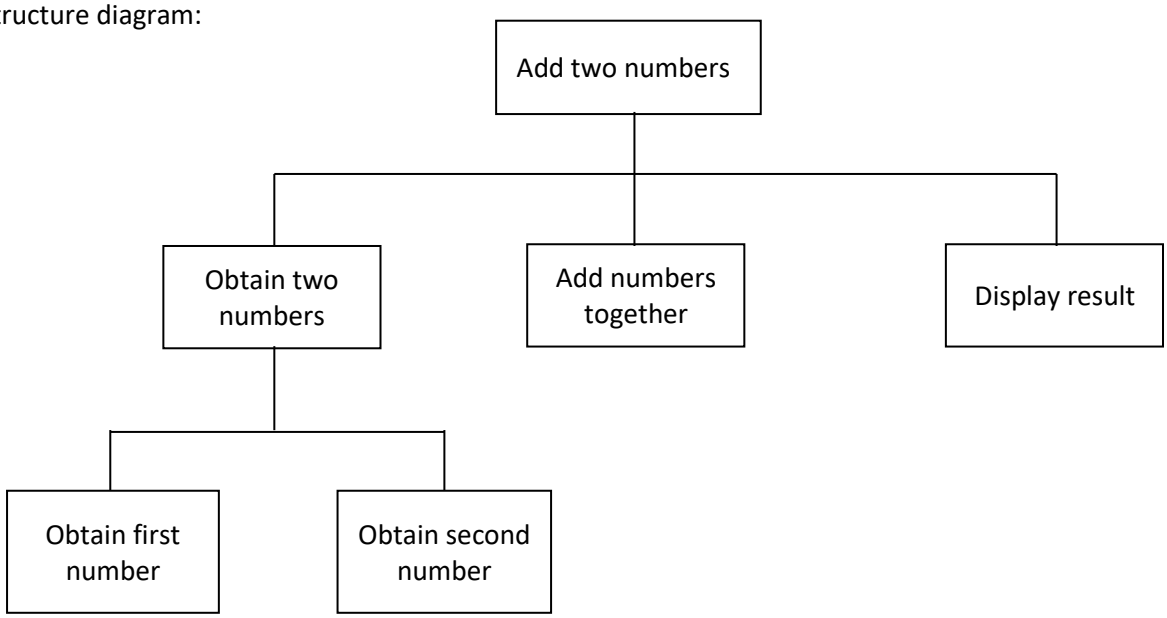



```
Task Number guessing game
  player 1 selects number
  repeat
    player 2 makes a guess
    if guess = number then
      player 1 says "correct"
    endif
    if guess > number then
      player 1 says "too high"
    endif
    if guess < number then
      player 1 says "too low"
    endif
  until correct number guessed or 10 attempts made
End task
```

```
if guess = number then
   player 1 says "correct"
else
   if guess > number then
      player 1 says "too high"
   else
      player 1 says "too low"
   endif
endif
```

A FIRST PASCAL PROGRAM

Structure diagram:




```
PROGRAM AddTwoNumbers;
  USES
    CRT, SYSUTILS;

  VAR
   Number1, Number2, Total : LONGINT;

  PROCEDURE Obtain;
     BEGIN
       WRITE('Please input the first number: ');
       READLN(Number1);
       WRITE('Please input the second number: ');
       READLN(Number2);
    END;

  PROCEDURE Add;
    BEGIN
      Total := Number1 + Number2;
    END;

  PROCEDURE Display;
    BEGIN
      WRITE('The total is ');
      WRITELN(Total);
    END;


  BEGIN
    Obtain;
    Add;
    Display;

    READLN;
  END.
```

Notice that the program name, AddTwoNumbers, corresponds to the top level of the structure diagram. The procedure names (Obtain, Add and Display) correspond to the next level of the design.

In the program the words in capital letters are reserved or key words, that is, they mean something special in Pascal. These words do not have to be in capital letters in your program, it is up to you to decide how to use upper- and lowercase letters in your programs. This is because Pascal is NOT a case-sensitive language (some languages are case-sensitive). To make programs easier to read it is good practice to distinguish between reserved words (eg all uppercase) and user-defined identifiers (eg lowercase/mixed case),

The program falls into 3 main parts; part 1 is the initialisation section:

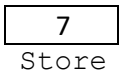
```
PROGRAM AddTwoNumbers
  USES
    CRT, SYSUTILS;

  VAR
    Number1, Number2, Total : LONGINT;
```

The USES section allows prewritten code/routines which are stored in units to be included within a program. In this case CRT and SYSUTILS are units that come with Pascal in a library of routines that can be used within the program. The CRT library has routines that help with the screen display and text formatting. For example it contains the code for the procedure called by CLRSCR to clear the screen and the code for the function READKEY to input a character pressed on the keyboard. SYSUTILS has routines to assist with file handling, string formatting and exception handling. We will be reviewing these later.

Each line in the program is generally a program statement, these must end with the semi-colon (;) as shown.

The next part of the initialisation section, VAR, contains details of any variables used in the program. The word VAR introduces this section and doesn't have a semi-colon, each statement in the VAR section does have one however. A variable reserves a location in the computer's memory (RAM) to store data and is used to store values in a program, it can be thought of as a memory cell or box:



In this case the memory cell or variable, called Store, contains the value 7. Variables have to be decalred with the data type that they will be used for, such as characters (CHAR), integers or whole numbers (LONGINT or INTEGER), real numbers or numbers that can have fractional "*decimal*" parts (REAL), text or strings, words or combinations of letters, numbers and other symbols (STRING) or Boolean, which are the values true or false (BOOLEAN). So Store could be declared as LONGINT, REAL, STRING, CHAR or BOOLEAN.

Declaration	Example data	Notes
VAR Store : REAL;	7.53 or 7	7 as a REAL is stored differently to 7 as an integer
VAR Store : CHAR;	'A' or '$'	We usually place single quotes around characters or strings
VAR Store : STRING;	'Hello' or 'G%9h'	
VAR Store : LONGINT;	7 or -32	
VAR Store : STRING;	'7.53'	This would be stored as a string NOT a number (so no calculations)
VAR Store : BOOLEAN;	FALSE	

A variable can only hold one value at any given time, and a variable name can only be one string (ie no spaces). So if you want a variable called 'Number 1', it must be declared as 'Number1' or 'Number_1' etc.

The program has to be told what names and types we wish to give to these variables, so that space can be reserved in memory (RAM). You should use meaningful (sensible) identifiers (names) so we can easily tell what each variable refers to. All the variables in this program will store whole numbers so we'll use the LONGINT data type.

The main body of the program contains special reserved Pascal commands which (when translated) tell the computer what to do, and the user-defined variable names and other identifiers.

Part 2 is the declaration section:

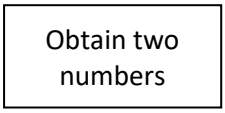
```
PROCEDURE Obtain;
  BEGIN
    WRITE('Please input the first number: ');
    READLN(Number1);
    WRITE('Please input the second number: ');
    READLN(Number2);
  END;

PROCEDURE Calculate;
  BEGIN
    Total := Number1 + Number2;
  END;

PROCEDURE Display;
  BEGIN
    WRITELN('The total is: ',Total);
  END;
```

This section contains the three main parts of the program outlined in the structure diagram. Each procedure that performs a self-contained task corresponding to one or more boxes in the structure diagram are declared here:

`Procedure Obtain;` is the procedure that obtains the two numbers to be added and is the coding for the structure diagram box:



Each Procedure has to have `BEGIN` and `END` statements as shown.

The `WRITE` or `WRITELN` statements are used as output to display details on the screen. Any data or message has to be inside brackets. To display a message, it has to be within single quotes, eg:

```
WRITE('Please input the first number: ');
```

Because this is a complete Pascal statement, it is terminated with a semi-colon.
If it to display the contents of a variable, then the variable identifier (name) is NOT in quotes, eg:

```
WRITELN(Total);
```

Messages and variables can be combined in one statement, eg:

```
WRITELN('The total is: ',Total);
```

After the output the next output will stay on the same line if `WRITE` is used, or the next line if `WRITELN` is used.
The `READLN` statement is used for input and causes the computer to pause and wait for the person using the program to type in something from the keyboard and press the <Return> or <Enter> key. When the user types in an integer (because this was the data type specified in the VAR section), it is put into the variable

Number1. There should always be a variable associated with this statement (unless it is used as a pause - see later), eg:

```
READLN(Number1);
```

(Again a semi-colon signals the end of a complete Pascal statement.)

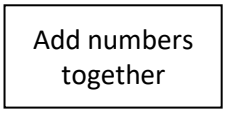
Whenever data is to be entered via the keyboard, it is usual to have a combination of WRITE/WRITELN (to give a prompt what to enter) and a READLN (to input the data).

Suppose the user types in the numbers 3 and 6, our variables in memory now contain:

3	6	
Number1	Number2	Total

Total is undefined, ie the memory location is reserved in RAM, but the programmer does not what is stored in it.

The next procedure is Procedure Calculate; which uses the values input into the variables Number1 and Number2 and adds them, storing the result in the variable total, which corresponds to the structure diagram box:




```
PROCEDURE Calculate;
  BEGIN
    Total := Number1 + Number2;
  END;
```

The symbol **:=** is called the assignment operator, it calculates the value on the right-hand side and places it in the variable on the left-hand side. So it uses the value stored in Number1 (eg 3) and adds it to the value stored in Number2 (eg 6) which gives 9 and this is stored in the variable Total (so it now it is not undefined).

Notes:

- These are illegal statements:
  - `Total + 2 := Number1 + Number2;` **wrong!**
  - `Number1 + Number2 := Total;` **wrong!**
- This is a legal statement
  - `Number1 := Number1+5;`
    - It takes the current value in Number1 (eg 3), adds 5 to it (ie to give 8) and then stores it back in Number1 (overwriting the value that was there, so 3 is overwritten/replaced by 8).

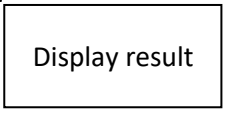
Only a single variable can go on the left.

An expression can include all mathematical operations, eg combinations of addition (+), subtraction (-), multiplication (*), division (/) etc.

In the three variables we now have:

3	6	9
Number1	Number2	Total

The next procedure, Procedure Display; is used to output the result of the calculation on the terminal (monitor) screen.



```
PROCEDURE Display;
  BEGIN
    WRITE('The total is ');
    WRITELN(Total);
  END;
```

We have already seen that WRITE/WRITELN can display a message specified between single quotes, eg:

```
WRITE('The total is ');
```

If quotes are not used, then WRITE/WRITELN displays the contents of a variable, eg:

```
WRITELN(Total);
```

will not display the word 'total', but the contents of the variable called total, ie 9. (If `WRITELN('Total');` was used, the word "Total" would be output, not the contents of the variable `Total`.)

As stated before, text/messages and variables can be combined in one output statement so the following are valid statements:

```
WRITELN('The total is: ',Total); or
WRITELN('The answer is ',Total, ' when adding ',Number1, ' and ',Number2);
```

WRITELN and WRITE.

The only difference is that WRITELN moves onto a new line ready for the next output. For example:

```
WRITELN('Hello');
WRITELN('Goodbye');
```

would display

Hello
Goodbye

Whereas

```
WRITE('Hello');
WRITELN('Goodbye');
```

would display

HelloGoodbye

Assuming that the following variables contain the given values:

15	2
Age	Height

then

```
WRITELN('My age is',age,'and my height is',height,'metres');
```

would display

My age is15and my height is2metres

Note the lack of spaces. However, because everything inside the quotes is printed exactly as it appears we can include the wanted spaces there:

```
WRITELN('My age is ',age,' and my height is ',height,' metres');
```

and we would then see

My age is 15 and my height is 2 metres

Note that:

```
WRITELN('My age is age and my height is height metres');
```

would output

My age is age and my height is height metres

because everything in quotes is written exactly as given. Variable names must be outside of the quotes (and separated by commas).

Part 3 is the main program

```
BEGIN
  Obtain;
  Calculate;
  Display;

  READLN;
END.
```

As a general rule, the main program will correspond exactly to the top level of your structure diagram and will only contain the names of procedures. When a procedure name appears on its own, as above, it is said to be called. Procedures may be called more than once if neccessary. When a procedure is called the program carries out the instructions (or statements) in that procedure.

The instructions in a procedure are not carried out until the procedure is called.

The READLN statement following the procedure calls makes the program pause so that you can see the output. (It's actually waiting for the <return> key to be pressed. No data will be stored because there is no variable stated.)

The last line signals the end of the program. Notice that in this one case there is a full stop after the END rather than a semi-colon.

(Although the statements in the program before an END have a semi-colon after them, this is just to maintain consistency and is not required by Pascal. All instructions between a BEGIN and an END are treated as one statement by Pascal. Hence BEGIN does not have a semi-colon after it because it is not a complete statement in its own right (similar to VAR), but END does because it is the finish of a statement.)

Take one last look at the program and notice the way the text is laid out with indentations in the VAR section and between the BEGIN and END statement. The compiler doesn't need to have a program laid out this way, the idea is to make the program easier to read. If you follow this, or a similar way, of laying out programs, you will find that both you and other programmers will find your programs easier to follow.

The compiler is the program that converts the Pascal source code into a machine code program for the computer to execute (run).

Any messages between { } are comments. These are ignored by the compiler, but are helpful for you to explain what certain parts of the program are doing. Another comment indicator is the statement //. Any text after this until the end of the line is regarded as a comment. Similar to good indentation it will help both you and other programmers to understand your program. The last line tells the compiler that there is no more program to be compiled, notice that in this one case there is a full stop after the END rather than a semi-colon.

SUMMARY

- A Pascal program follows the structure:

```
PROGRAM name;      //identifier must be one word, no spaces
  USES
    Library names/identifiers;
                          separated by commas,(eg CRT, SYSUTILS) ; to end
  VAR
    variable name/identifier: data type;     // identifiers one word

  <Subprograms>; (procedures and functions)

  BEGIN
    Pascal statements;
  END.
```

- A semi-colon (;) is required at the end of each complete Pascal statement (optional before an `END`).

- Input into the computer via the keyboard is by the Pascal statement `READLN(variable);`.

- Output from the program of either a text string (message) and/or variable contents is by the `WRITE` or `WRITELN` statements.

- Calculations are carried out by the assignment symbol `:=` and the result of the calculation (specified on the right hand side) must be placed into a single variable named on the left hand side.
    - This can also be used to store a value in a variable, eg `Total:=0;`

- Indentation showing the start and finish of specific sections (eg `VAR`, between `BEGIN` and `END`) improves the readability of a program, as does the use of blank lines.

- Any comments to be used by the programmer, but to be ignored by the compiler are placed within `{}` or after `//` (which comments until the end of the line).

WORKSHEET – First program

1. In the spaces provided, show what would be produced by the `WRITELN` statements, assuming variables with values as indicated by the boxes. Be careful with the spaces.

216	6	9	'John'
Amount	Number1	Number2	Name

   a. `WRITELN('I owe Name ',Amount,' pounds');`

   b. `WRITELN('Number1 is not equal to',number2);`

   c. `WRITELN(name,' is my best friend!');`

   d. `WRITELN('number2, is three more than, number1');`

2. Place the correct letter from the list below into the corresponding box:

REAL	STRING	LONGINT (INTEGER)	CHAR

   a. 'Hi there, genius'
   b. -98
   c. 'p'
   d. 7.2345

3. Put a cross next to any of the following Pascal statements which you think are wrong:
   a. Total := number1 + number2; [ ]
   b. total + 12 := number; [ ]
   c. amount := 56.89 * 13; [ ]
   d. Amount = Subtotal + Amount_to_pay; [ ]
   e. AMOUNT := AMOUNT + 1; [ ]
   f. Begin := Previous * 44.56; [ ]
   g. Procedure := possible - probable [ ]
   h. PROC:=POSS+PROB; [ ]

4. What is produced on the monitor by the following sequence of statements?

```
WRITE('I am not very happy');
WRITELN('about how my course');
WRITELN('is shaping up.');
```

Simple Arithmetic Operators

These are:

+	add
-	subtract
*	multiply
/	divide

All of them can be used with numeric (`LONGINT`, `INTEGER` and `REAL`) variables to do calculations such as:

- `Total := Number1 + Number2 + Number3;`
- `Average := (Number1 + Number2 + Number3) / 3;`

(`Average` must be declared as `REAL` since after a division the result may not be an integer. `Number1`, `Number2` and `Number3` can be `REAL` or `LONGINT`, or mixed.)

Brackets can be used to force the addition before the division as above as in "normal" maths - BIDMAS. (Numbers or variables either side of the operator are called operands.) The order that calculations are carried out depend on operator precedence , ie

```
Average := Number1 + Number2 + Number3 / 3;
```

is not the same as the last expression (and is incorrect!).

If powers are required, eg `Area := Pi*Radius`^{2} then in Virtual Pascal, the pre-written routine `POWER` can be used which is in the library `MATH`, so must be included in the `USES` section.

```
PROGRAM CircleArea;
  USES
    CRT, SYSUTILS, MATH;
  VAR
    Area, Pi, Radius : REAL;

  PROCEDURE ObtainRadius;
    BEGIN
      WRITE('Enter radius of circle in cm: ');
      READLN(Radius);
    END;

  PROCEDURE CalcAreaAndDisplay;
    BEGIN
      Pi := 3.142;
      Area := Pi*(POWER(Radius,2));
      WRITELN('Area of circle with radius ',Radius,'cm is ',
              Area:4:2,'sq cm');
    END;

  BEGIN
    ObtainRadius;
    CalcAreaAndDisplay;

    READLN;
  END.
```

or
`Area:=3.142*Radius*Radius`
//No need for `MATH` unit or `pi` variable

In addition the + sign can be used with `STRING` types to concatenate them (join together).

```
Name1 := 'Richard M. ';
Name2 := 'Nixon';
Name3 := string1 + string2;
```

`Name3` now has the value 'Richard M. Nixon'. Or we could have used:

```
Name1 := Name1 + Name2;
```

and `Name1` now has the value 'Richard M. Nixon' (having overwritten the original contents). Or:

```
Name2 := Name2 + ', ' + Name1;
```

which would overwrite the contents of `Name2` with the value 'Nixon, Richard M. '

Notice the trailing space in the last example.

WORKSHEET – Arithmetic Operators

1. Some variables have values assigned as follows:

```
number1 := 12.5;
number2 :=  2.5;
number3 := 2;
number4 := 4.0;
total  := 0;
average := 0;

text1 := ' teaching computing is';
text2 := 'mildy amusing at times';
text3 := 'That''s all folks';
```

What values do the following operations produce:

a. `total := number1 + number2 * 2;` .....

b. `average := (number1 + number2) / 2.0;` .....

c. `total := number1 * 2.0 - number2 * 2.0;` .....

d. `average := number1 + number2 / 2.0;`
`average := average - number1 / 2.0;` .....

e. `total := POWER(number4,number3);` .....

f. `text3 := text2 + text1;`

.....

2. Design and write a program to calculate and display the cost of dress material, given the cost per square metre and the number of square metres required.

SUMMARY

- Mathematical expressions in Pascal can use:
  - brackets
  - powers (`POWER`)
  - multiplication (*), division (/)
  - addition (+), subtraction (-)
- Operator precedence means that the order of calculation is not necessarily always from left to right. The precedence of each operator is in the order given above, ie brackets before powers etc.
- String manipulation also uses the symbol + to mean concatenation.